

```
1  ---
2  ## 文件: `AlbumWindow.ts`
3  ``typescript
4  import { _decorator, Button, Color, Component, instantiate, Label, Node, Prefab, Sprite, SpriteFrame, tween,
5  Tween, UITransform, Vec3 } from 'cc';
6  import { RoundBox } from '../RoundBox';
7  import { CardSkinLibrary } from './Config/CardSkinLibrary';
8  import { getSkillConfigBySkinId } from './Config/Skills';
9  import { DataManager } from './Manager/DataManager';
10 import { ToastManager } from './Manager/ToastManager';
11 import { EVENT_ENUM, AUDIO_EFFECT_ENUM } from './Utils/Enum';
12 import { EQUIPPED_SKIN_CHANGE, PLAY_AUDIO } from './Utils/Event';
13 import { applySpriteFrameToUINode, clearSpriteFrameOnUINode, setNodeUIOpacity } from './Utils/Utils';
14 const { ccclass, property } = _decorator;
15 const SKIN_DISPLAY_ORDER: number[] = [
16     0, 101, 102, 103, 104, 106, 107, 203, 202, 201, 205, 204, 206, 207, 301, 302, 303, 304, 401, 402,
17 ];
18 const CARDS_PER_PAGE = 20;
19 const GRID_COLS = 4;
20 const ALBUM_GRID_CARD_SCALE = 1.0;
21 const GRID_GAP_X = 30;
22 const GRID_GAP_Y = 25;
23 const ALBUM_GRID_LAYOUT_OFFSET_X = 0;
24 const ALBUM_GRID_LAYOUT_OFFSET_Y = 5;
25 const UNOWNED_CARD_ROOT_OPACITY = Math.round(255 * 0.6);
26 const UNOWNED_SKILL_ICON_OPACITY = Math.round((255 * 0.3 * 255) / UNOWNED_CARD_ROOT_OPACITY);
27 const ALBUM_FLIP_HALF_DURATION = 0.375;
28 const ALBUM_MOVE_ENLARGE_DURATION = 0.6;
29 const ALBUM_FLIP_THIN_X = 0.05;
30 const ALBUM_SELECTED_SCALE_MULT = 4;
31 const ALBUM_NAV_DIM_OPACITY = Math.round(255 * 0.3);
32 const ALBUM_SKILL_LEVEL_COLORS: Color[] = [
33     new Color(0x37, 0x1e, 0x1a), // 0 褐 #371e1a
34     new Color(0x00, 0x1f, 0x55), // 1 蓝 #001f55
35     new Color(0x3f, 0x00, 0x55), // 2 紫 #3f0055
36     new Color(0x55, 0x00, 0x00), // 3 红 #550000
37     new Color(0x00, 0x00, 0x00), // 4 炫彩 #000000
38 ];
39 const ALBUM_BACK_BTN_BG_BY_SKILL_LEVEL: Color[] = [
40     new Color(0xf0, 0xde, 0xdb), // 0 褐 #f0dedb
41     new Color(0xe9, 0xf1, 0xff), // 1 蓝 #e9f1ff
42     new Color(0xf9, 0xe9, 0xff), // 2 紫 #f9e9ff
43     new Color(0xff, 0xe9, 0xe9), // 3 红 #ffe9e9
44     new Color(0xff, 0xe9, 0xf8), // 4 粉色 #ffe9f8
45 ];
46 @ccclass('AlbumWindow')
47 export class AlbumWindow extends Component {
48     @property(Node)
49     closeButtonNode: Node = null;
50     @property(Node)
51     showCardsNode: Node = null;
52     @property(Prefab)
53     cardDisplayPrefab: Prefab = null;
54     @property(CardSkinLibrary)
55     cardSkinLibrary: CardSkinLibrary = null;
56     @property(Node)
57     prevPageButtonNode: Node = null;
58     @property(Node)
```

```

59     nextPageButtonNode: Node = null;
60     @property(ToastManager)
61     toastManager: ToastManager = null;
62     private _pageIndex = 0;
63     private _albumFlipAnimating = false;
64     private _albumFrontShownCards = new WeakSet<Node>();
65     private _albumEnlargePending = new Map<
66         Node,
67         { origPos: Vec3; origScale: Vec3; skinId: number; origSiblingIndex: number }
68     >();
69     private _albumFrontUnbind = new Map<Node, () => void>();
70     private _albumGridOwnedTouchUnbind = new Map<Node, () => void>();
71     private _albumCardSkinId = new WeakMap<Node, number>();
72     private _albumCardBgDesignSize = new WeakMap<Node, { w: number; h: number }>();
73     private _albumNavInputBlocked = false;
74     private _albumBackdropBgNode: Node | null = null;
75     private _albumFlipDriver = { u: 0 };
76     private _albumMoveEnlargeDriver = { t: 0 };
77     private _albumFlipScratchBase = new Vec3();
78     private _albumFlipScratchScale = new Vec3();
79     private _albumFlipScratchPos = new Vec3();
80     get pageIndex(): number {
81         return this._pageIndex;
82     }
83     private get maxPageIndex(): number {
84         const n = SKIN_DISPLAY_ORDER.length;
85         return Math.max(0, Math.ceil(n / CARDS_PER_PAGE) - 1);
86     }
87     protected onLoad(): void {
88         this._albumBackdropBgNode = this.node.getChildByName('bg') ?? null;
89         if (this._albumBackdropBgNode) {
90             this._albumBackdropBgNode.on(Node.EventType.TOUCH_END, this.onCloseButtonClick, this);
91         }
92         if (this.closeButtonNode) {
93             this.closeButtonNode.on(Node.EventType.TOUCH_END, this.onCloseButtonClick, this);
94         }
95         if (this.prevPageButtonNode) {
96             this.prevPageButtonNode.on(Node.EventType.TOUCH_END, this.onPrevPageButtonClick, this);
97         }
98         if (this.nextPageButtonNode) {
99             this.nextPageButtonNode.on(Node.EventType.TOUCH_END, this.onNextPageButtonClick, this);
100         }
101     }
102     protected onDisable(): void {
103         this.applyAlbumNavChromeState('normal');
104     }
105     protected onDestroy(): void {
106         if (this._albumBackdropBgNode?.isValid) {
107             this._albumBackdropBgNode.off(Node.EventType.TOUCH_END, this.onCloseButtonClick, this);
108         }
109         this._albumBackdropBgNode = null;
110         if (this.closeButtonNode && this.closeButtonNode.isValid) {
111             this.closeButtonNode.off(Node.EventType.TOUCH_END, this.onCloseButtonClick, this);
112         }
113         if (this.prevPageButtonNode && this.prevPageButtonNode.isValid) {
114             this.prevPageButtonNode.off(Node.EventType.TOUCH_END, this.onPrevPageButtonClick, this);
115         }
116         if (this.nextPageButtonNode && this.nextPageButtonNode.isValid) {

```

```
117         this.nextPageButtonNode.off(Node.EventType.TOUCH_END, this.onNextPageButtonClick, this);
118     }
119 }
120 private collectAlbumNavButtonsForSlot(navBoundNode: Node): Button[] {
121     if (!navBoundNode?.isValid) {
122         return [];
123     }
124     const seen = new Set<Button>();
125     const out: Button[] = [];
126     const add = (b: Button | null | undefined) => {
127         if (b?.node?.isValid && !seen.has(b)) {
128             seen.add(b);
129             out.push(b);
130         }
131     };
132     const childBtns = navBoundNode.getComponentsInChildren(Button);
133     for (let i = 0; i < childBtns.length; i++) {
134         add(childBtns[i]);
135     }
136     add(navBoundNode.getComponent(Button));
137     add(navBoundNode.parent?.getComponent(Button) ?? null);
138     return out;
139 }
140 private applyAlbumNavChromeState(mode: 'normal' | 'flipAnimating' | 'enlarged'): void {
141     this._albumNavInputBlocked = mode !== 'normal';
142     const navs = [this.closeButtonNode, this.prevPageButtonNode, this.nextPageButtonNode];
143     const interactable = mode === 'normal';
144     const opacity = mode === 'enlarged' ? ALBUM_NAV_DIM_OPACITY : 255;
145     for (let i = 0; i < navs.length; i++) {
146         const n = navs[i];
147         if (!n?.isValid) {
148             continue;
149         }
150         setNodeUIOpacity(n, opacity);
151         const buttons = this.collectAlbumNavButtonsForSlot(n);
152         for (let j = 0; j < buttons.length; j++) {
153             buttons[j].interactable = interactable;
154         }
155     }
156 }
157 resetToFirstPage(): void {
158     this._pageIndex = 0;
159     this.refreshPageContent();
160 }
161 private onCloseButtonClick(): void {
162     if (this._albumNavInputBlocked) {
163         return;
164     }
165     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
166     this.node.active = false;
167 }
168 private onPrevPageButtonClick(): void {
169     if (this._albumNavInputBlocked) {
170         return;
171     }
172     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
173     if (this._pageIndex <= 0) {
174         return;
```

```
175     }
176     this._pageIndex--;
177     this.refreshPageContent();
178 }
179 private onNextPageButtonClick(): void {
180     if (this._albumNavInputBlocked) {
181         return;
182     }
183     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
184     if (this._pageIndex >= this.maxPageIndex) {
185         return;
186     }
187     this._pageIndex++;
188     this.refreshPageContent();
189 }
190 private refreshPageContent(): void {
191     if (!this.showCardsNode || !this.cardDisplayPrefab) {
192         return;
193     }
194     if (!this.cardSkinLibrary) {
195         console.warn('[AlbumWindow] 请绑定 cardSkinLibrary');
196         return;
197     }
198     this.applyAlbumNavChromeState('normal');
199     this._albumFlipAnimating = false;
200     const existing = [...this.showCardsNode.children];
201     for (let i = 0; i < existing.length; i++) {
202         const ex = existing[i];
203         this.unbindAlbumFrontActions(ex);
204         this.unbindAlbumGridOwnedCardOpen(ex);
205     }
206     this._albumEnlargePending.clear();
207     this.showCardsNode.removeAllChildren();
208     const start = this._pageIndex * CARDS_PER_PAGE;
209     const slice = SKIN_DISPLAY_ORDER.slice(start, start + CARDS_PER_PAGE);
210     const prefabRoot = this.cardDisplayPrefab.data;
211     const layoutUt =
212         prefabRoot.getChildByName('CardBack')?.getComponent(UITransform) ??
213         prefabRoot.getComponent(UITransform);
214     const baseW = layoutUt?.width ?? 120;
215     const baseH = layoutUt?.height ?? 160;
216     const cellW = baseW * ALBUM_GRID_CARD_SCALE;
217     const cellH = baseH * ALBUM_GRID_CARD_SCALE;
218     const gapX = GRID_GAP_X;
219     const gapY = GRID_GAP_Y;
220     const cols = GRID_COLS;
221     const rows = Math.ceil(CARDS_PER_PAGE / cols);
222     const contentW = cols * cellW + (cols - 1) * gapX;
223     const contentH = rows * cellH + (rows - 1) * gapY;
224     const originX = -contentW / 2 + cellW / 2;
225     const originY = contentH / 2 - cellH / 2;
226     const owned = DataManager.instance.ownedSkins;
227     for (let i = 0; i < slice.length; i++) {
228         const skinId = slice[i];
229         const col = i % cols;
230         const row = Math.floor(i / cols);
231         const x = originX + col * (cellW + gapX);
232         const y = originY - row * (cellH + gapY);
```

```
233         const card = instantiate(this.cardDisplayPrefab);
234         card.setParent(this.showCardsNode);
235         card.setPosition(x + ALBUM_GRID_LAYOUT_OFFSET_X, y + ALBUM_GRID_LAYOUT_OFFSET_Y, 0);
236         card.setScale(ALBUM_GRID_CARD_SCALE, ALBUM_GRID_CARD_SCALE, 1);
237         this._albumCardSkinId.set(card, skinId);
238         this.applyAlbumCardEnlargeOverlayHidden(card);
239         this.applyPreviewSkinToCard(card, skinId);
240         const isOwned = owned.includes(skinId);
241         this.applyAlbumCardOpacity(card, isOwned);
242         this.setAlbumCardButtonsInteractable(card, isOwned);
243         this.applyAlbumGridOwnedRootButtonOff(card, isOwned);
244         if (isOwned) {
245             this.bindAlbumGridOwnedCardOpen(card, skinId);
246         }
247         this.applyAlbumGridRootHitBoxToCardBack(card);
248     }
249 }
250 private applyAlbumCardOpacity(card: Node, isOwned: boolean): void {
251     const skillNode = this.resolveSkillIconNode(card);
252     if (isOwned) {
253         setNodeUIOpacity(card, 255);
254         if (skillNode) {
255             setNodeUIOpacity(skillNode, 255);
256         }
257         return;
258     }
259     setNodeUIOpacity(card, UNOWNED_CARD_ROOT_OPACITY);
260     if (skillNode) {
261         setNodeUIOpacity(skillNode, UNOWNED_SKILL_ICON_OPACITY);
262     }
263 }
264 private resolveSkillIconNode(card: Node): Node | null {
265     const cardBack = card.getChildByName('CardBack');
266     return cardBack?.getChildByName('Skill') ?? card.getChildByName('Skill') ?? null;
267 }
268 private applyAlbumGridRootHitBoxToCardBack(card: Node): void {
269     const rootUt = card.getComponent(UITransform);
270     if (!rootUt?.isValid) {
271         return;
272     }
273     const backUt = card.getChildByName('CardBack')?.getComponent(UITransform);
274     if (backUt?.isValid) {
275         rootUt.setContentSize(backUt.width, backUt.height);
276     } else {
277         rootUt.setContentSize(120, 160);
278     }
279 }
280 private applyAlbumCardRootHitBoxFromPrefabTemplate(card: Node): void {
281     const rootUt = card.getComponent(UITransform);
282     const tpl = this.cardDisplayPrefab?.data?.getComponent(UITransform);
283     if (!rootUt?.isValid || !tpl?.isValid) {
284         return;
285     }
286     rootUt.setContentSize(tpl.width, tpl.height);
287 }
288 private setAlbumCardButtonsInteractable(card: Node, interactable: boolean): void {
289     const buttons = card.getComponentsInChildren(Button);
290     for (let i = 0; i < buttons.length; i++) {
```

```

291         buttons[i].interactable = interactable;
292     }
293 }
294 private onOwnedCardClick(skinId: number, card: Node): void {
295     if (this._albumFlipAnimating || !card?.isValid) {
296         return;
297     }
298     if (this._albumFrontShownCards.has(card)) {
299         return;
300     }
301     if (this._albumEnlargePending.size > 0 && !this._albumEnlargePending.has(card)) {
302         return;
303     }
304     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
305     this.playAlbumSelectFlipAndEquip(skinId, card);
306 }
307 private albumFlipMulAtU(u: number): { mx: number; my: number } {
308     if (u <= 0.5) {
309         const t = u * 2;
310         return {
311             mx: 1 + (ALBUM_FLIP_THIN_X - 1) * t,
312             my: 1,
313         };
314     }
315     const t = (u - 0.5) * 2;
316     return {
317         mx: ALBUM_FLIP_THIN_X + (1 - ALBUM_FLIP_THIN_X) * t,
318         my: 1,
319     };
320 }
321 private albumApplyFlipOnlyFrame(card: Node, origPos: Vec3, origScale: Vec3, u: number): void {
322     if (!card?.isValid) {
323         return;
324     }
325     card.setPosition(origPos);
326     const { mx, my } = this.albumFlipMulAtU(u);
327     this._albumFlipScratchScale.set(origScale.x * mx, origScale.y * my, origScale.z);
328     card.setScale(this._albumFlipScratchScale);
329 }
330 private albumApplyMoveEnlargeFrame(
331     card: Node,
332     origPos: Vec3,
333     origScale: Vec3,
334     endScale: Vec3,
335     t: number,
336 ): void {
337     if (!card?.isValid) {
338         return;
339     }
340     Vec3.lerp(this._albumFlipScratchPos, origPos, Vec3.ZERO, t);
341     Vec3.lerp(this._albumFlipScratchBase, origScale, endScale, t);
342     card.setPosition(this._albumFlipScratchPos);
343     card.setScale(this._albumFlipScratchBase);
344 }
345 private albumApplyMoveShrinkFrame(
346     card: Node,
347     startPos: Vec3,
348     origPos: Vec3,

```

```
349         startScale: Vec3,
350         origScale: Vec3,
351         t: number,
352     ): void {
353         if (!card?.isValid) {
354             return;
355         }
356         Vec3.lerp(this._albumFlipScratchPos, startPos, origPos, t);
357         Vec3.lerp(this._albumFlipScratchBase, startScale, origScale, t);
358         card.setPosition(this._albumFlipScratchPos);
359         card.setScale(this._albumFlipScratchBase);
360     }
361     private playAlbumSelectFlipAndEquip(skinId: number, card: Node): void {
362         const lib = this.cardSkinLibrary;
363         if (!lib || !this.showCardsNode) {
364             return;
365         }
366         this._albumFlipAnimating = true;
367         this.applyAlbumNavChromeState('flipAnimating');
368         this.setAllAlbumCardButtonsInteractable(false);
369         Tween.stopAllByTarget(card);
370         Tween.stopAllByTarget(this._albumFlipDriver);
371         Tween.stopAllByTarget(this._albumMoveEnlargeDriver);
372         const origSiblingIndex = card.getSiblingIndex();
373         card.setSiblingIndex(this.showCardsNode.children.length - 1);
374         this.applyAlbumCardRootHitBoxFromPrefabTemplate(card);
375         const half = ALBUM_FLIP_HALF_DURATION;
376         const origPos = card.position.clone();
377         const origScale = card.scale.clone();
378         this._albumEnlargePending.set(card, {
379             origPos: origPos.clone(),
380             origScale: origScale.clone(),
381             skinId,
382             origSiblingIndex,
383         });
384         const endScale = origScale.clone().multiplyScalar(ALBUM_SELECTED_SCALE_MULT);
385         this._albumFlipDriver.u = 0;
386         const applyFlipOnly = (): void => {
387             this.albumApplyFlipOnlyFrame(card, origPos, origScale, this._albumFlipDriver.u);
388         };
389         applyFlipOnly();
390         const finishEnlargeOpen = (): void => {
391             this._albumMoveEnlargeDriver.t = 1;
392             this.albumApplyMoveEnlargeFrame(card, origPos, origScale, endScale, 1);
393             this._albumFlipAnimating = false;
394             if (!card.isValid) {
395                 this._albumEnlargePending.delete(card);
396                 this.applyAlbumNavChromeState('normal');
397                 this.applyAlbumGridButtonsByOwnership();
398                 return;
399             }
400             this.applyAlbumNavChromeState('enlarged');
401             this.applyAlbumGridButtonsWhileOneEnlarged(card);
402             this.applyAlbumCardEnlargeOverlayVisible(card, skinId);
403             this._albumFrontShownCards.add(card);
404             this.bindAlbumFrontActions(card, skinId);
405         };
406         tween(this._albumFlipDriver)
```

```

407         .to(half, { u: 0.5 }, { easing: 'linear', onUpdate: applyFlipOnly })
408         .call(() => {
409             this._albumFlipDriver.u = 0.5;
410             applyFlipOnly();
411             if (card.isValid) {
412                 this.showAlbumCardFrontForSkin(card, skinId);
413             }
414         })
415         .to(half, { u: 1 }, { easing: 'linear', onUpdate: applyFlipOnly })
416         .call(() => {
417             this._albumFlipDriver.u = 1;
418             applyFlipOnly();
419             this._albumMoveEnlargeDriver.t = 0;
420             const applyMoveEnlarge = (): void => {
421                 this.albumApplyMoveEnlargeFrame(
422                     card,
423                     origPos,
424                     origScale,
425                     endScale,
426                     this._albumMoveEnlargeDriver.t,
427                 );
428             };
429             applyMoveEnlarge();
430             tween(this._albumMoveEnlargeDriver)
431                 .to(
432                     ALBUM_MOVE_ENLARGE_DURATION,
433                     { t: 1 },
434                     { easing: 'sineOut', onUpdate: applyMoveEnlarge },
435                 )
436                 .call(finishEnlargeOpen)
437                 .start();
438         })
439         .start();
440     }
441     private setAlbumCardFrontBarButtonsInteractable(card: Node, interactable: boolean): void {
442         const cf = card.getChildByName('CardFront');
443         if (!cf) {
444             return;
445         }
446         for (const name of ['back', 'equip']) {
447             const n = cf.getChildByName(name);
448             if (!n) {
449                 continue;
450             }
451             const bs = n.getComponentsInChildren(Button);
452             for (let i = 0; i < bs.length; i++) {
453                 bs[i].interactable = interactable;
454             }
455         }
456     }
457     private unbindAlbumFrontActions(card: Node): void {
458         const fn = this._albumFrontUnbind.get(card);
459         if (fn) {
460             fn();
461             this._albumFrontUnbind.delete(card);
462         }
463     }
464     private bindAlbumGridOwnedCardOpen(card: Node, skinId: number): void {

```



```
465         this.unbindAlbumGridOwnedCardOpen(card);
466         const target = card.getChildByName('CardBack') ?? card;
467         const onOpen = (): void => {
468             this.onOwnedCardClick(skinId, card);
469         };
470         target.on(Node.EventType.TOUCH_END, onOpen, this);
471         this._albumGridOwnedTouchUnbind.set(card, () => {
472             if (target.isValid) {
473                 target.off(Node.EventType.TOUCH_END, onOpen, this);
474             }
475         });
476     }
477     private unbindAlbumGridOwnedCardOpen(card: Node): void {
478         const fn = this._albumGridOwnedTouchUnbind.get(card);
479         if (fn) {
480             fn();
481             this._albumGridOwnedTouchUnbind.delete(card);
482         }
483     }
484     private applyAlbumGridOwnedRootButtonOff(card: Node, isOwned: boolean): void {
485         if (!isOwned) {
486             return;
487         }
488         const rootBtn = card.getComponent(Button);
489         if (rootBtn) {
490             rootBtn.interactable = false;
491         }
492     }
493     private bindAlbumFrontActions(card: Node, skinId: number): void {
494         this.unbindAlbumFrontActions(card);
495         const cf = card.getChildByName('CardFront');
496         const backNode = cf?.getChildByName('back');
497         const equipNode = cf?.getChildByName('equip');
498         const bgNode = card.getChildByName('BG');
499         const onBack = (): void => {
500             this.onAlbumCardBackClick(card);
501         };
502         const onEquip = (): void => {
503             this.onAlbumCardEquipClick(card, skinId);
504         };
505         backNode?.on(Node.EventType.TOUCH_END, onBack, this);
506         equipNode?.on(Node.EventType.TOUCH_END, onEquip, this);
507         bgNode?.on(Node.EventType.TOUCH_END, onBack, this);
508         this._albumFrontUnbind.set(card, () => {
509             backNode?.off(Node.EventType.TOUCH_END, onBack, this);
510             equipNode?.off(Node.EventType.TOUCH_END, onEquip, this);
511             bgNode?.off(Node.EventType.TOUCH_END, onBack, this);
512         });
513     }
514     private onAlbumCardBackClick(card: Node): void {
515         if (this._albumFlipAnimating || !card?.isValid) {
516             return;
517         }
518         if (!this._albumFrontShownCards.has(card) || !this._albumEnlargePending.has(card)) {
519             return;
520         }
521         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
522         this.playAlbumFlipBackToGrid(card);
```

```
523     }
524     private onAlbumCardEquipClick(card: Node, skinId: number): void {
525         if (this._albumFlipAnimating || !card?.isValid) {
526             return;
527         }
528         if (!this._albumFrontShownCards.has(card)) {
529             return;
530         }
531         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
532         DataManager.instance.equippedSkinId = skinId;
533         EQUIPPED_SKIN_CHANGE.emit(EVENT_ENUM.EQUIPPED_SKIN_CHANGE, skinId);
534         this.showAlbumEquipToast(skinId);
535     }
536     private playAlbumFlipBackToGrid(card: Node): void {
537         const pending = this._albumEnlargePending.get(card);
538         if (!pending || !this.showCardsNode) {
539             return;
540         }
541         this.unbindAlbumFrontActions(card);
542         this.applyAlbumCardEnlargeOverlayHidden(card);
543         this._albumFlipAnimating = true;
544         this.applyAlbumNavChromeState('flipAnimating');
545         this.setAllAlbumCardButtonsInteractable(false);
546         Tween.stopAllByTarget(card);
547         Tween.stopAllByTarget(this._albumFlipDriver);
548         Tween.stopAllByTarget(this._albumMoveEnlargeDriver);
549         const half = ALBUM_FLIP_HALF_DURATION;
550         const { origPos, origScale, skinId, origSiblingIndex } = pending;
551         const enlargedPos = card.position.clone();
552         const enlargedScale = card.scale.clone();
553         this._albumFlipDriver.u = 0;
554         const applyFlipOnly = (): void => {
555             this.albumApplyFlipOnlyFrame(card, enlargedPos, enlargedScale, this._albumFlipDriver.u);
556         };
557         applyFlipOnly();
558         const finishShrinkToGrid = (): void => {
559             this._albumMoveEnlargeDriver.t = 1;
560             this.albumApplyMoveShrinkFrame(card, enlargedPos, origPos, enlargedScale, origScale, 1);
561             this._albumFlipAnimating = false;
562             this._albumEnlargePending.delete(card);
563             this._albumFrontShownCards.delete(card);
564             if (!card.isValid) {
565                 this.applyAlbumNavChromeState('normal');
566                 return;
567             }
568             this.applyAlbumNavChromeState('normal');
569             this.applyAlbumGridButtonsByOwnership();
570             const maxIdx = Math.max(0, this.showCardsNode.children.length - 1);
571             card.setSiblingIndex(Math.min(origSiblingIndex, maxIdx));
572             this.applyAlbumGridRootHitBoxToCardBack(card);
573         };
574         tween(this._albumFlipDriver)
575             .to(half, { u: 0.5 }, { easing: 'linear', onUpdate: applyFlipOnly })
576             .call(() => {
577                 this._albumFlipDriver.u = 0.5;
578                 applyFlipOnly();
579                 if (card.isValid) {
580                     this.applyPreviewSkinToCard(card, skinId);
```

```

581         }
582     })
583     .to(half, { u: 1 }, { easing: 'linear', onUpdate: applyFlipOnly })
584     .call(() => {
585         this._albumFlipDriver.u = 1;
586         applyFlipOnly();
587         this._albumMoveEnlargeDriver.t = 0;
588         const applyMoveShrink = (): void => {
589             this.albumApplyMoveShrinkFrame(
590                 card,
591                 enlargedPos,
592                 origPos,
593                 enlargedScale,
594                 origScale,
595                 this._albumMoveEnlargeDriver.t,
596             );
597         };
598         applyMoveShrink();
599         tween(this._albumMoveEnlargeDriver)
600             .to(
601                 ALBUM_MOVE_ENLARGE_DURATION,
602                 { t: 1 },
603                 { easing: 'sineIn', onUpdate: applyMoveShrink },
604             )
605             .call(finishShrinkToGrid)
606             .start();
607     })
608     .start();
609 }
610 private albumEquipToastBgWidth(skillName: string): number {
611     const n = [...skillName].length;
612     if (n <= 2) {
613         return 340;
614     }
615     if (n === 3) {
616         return 360;
617     }
618     return 420;
619 }
620 private showAlbumEquipToast(skinId: number): void {
621     if (!this.toastManager?.isValid) {
622         return;
623     }
624     const cfg = getSkillConfigBySkinId(skinId);
625     const name = cfg?.name?.trim() ?? '';
626     const w = this.albumEquipToastBgWidth(name);
627     const msg = name.length > 0 ? `「${name}」装备成功` : '装备成功';
628     this.toastManager.show(msg, w, 450);
629 }
630 private showAlbumCardFrontForSkin(cardNode: Node, displaySkinId: number): void {
631     const lib = this.cardSkinLibrary;
632     if (!lib) {
633         return;
634     }
635     const textureIndex = Math.floor(displaySkinId / 100);
636     const fronts = lib.cardFrontSkinFrames;
637     const maxIdx = Math.max(0, fronts.length - 1);
638     const clamped = Math.max(0, Math.min(textureIndex, maxIdx));

```

```

639         const frontSf = fronts[clamped] ?? fronts[0];
640         if (!frontSf) {
641             return;
642         }
643         const cardFront = cardNode.getChildByName('CardFront');
644         const cardBack = cardNode.getChildByName('CardBack');
645         if (cardBack) {
646             cardBack.active = false;
647         }
648         if (cardFront) {
649             cardFront.active = true;
650             applySpriteFrameToUINode(cardFront, frontSf);
651             this.applyCardShowFrontSkillAndLabels(cardFront, displaySkinId, lib);
652             this.applyAlbumEquipButtonStyle(cardNode, displaySkinId);
653         }
654     }
655     private applyCardShowFrontSkillAndLabels(cardFront: Node, displaySkinId: number, lib: CardSkinLibrary):
656     void {
657         const frontSkill = cardFront.getChildByName('Skill');
658         const skillsf = this.resolveLibSkillSpriteFrame(lib, displaySkinId);
659         if (skillsf && frontSkill) {
660             frontSkill.active = true;
661             applySpriteFrameToUINode(frontSkill, skillsf);
662         } else {
663             clearSpriteFrameOnUINode(frontSkill ?? undefined);
664         }
665         const cfg = getSkillConfigBySkinId(displaySkinId);
666         const textColor = cfg ? this.colorForAlbumSkillLevel(cfg.skillLevel) : ALBUM_SKILL_LEVEL_COLORS[0];
667         const setLabel = (childName: string, text: string, outlineSameAsFill: boolean): void => {
668             const n = cardFront.getChildByName(childName);
669             const lab = n?.getComponent(Label);
670             if (!lab) {
671                 return;
672             }
673             lab.string = text;
674             lab.color = textColor;
675             if (outlineSameAsFill) {
676                 lab.enableOutline = true;
677                 lab.outlineColor = textColor;
678             }
679         };
680         if (cfg) {
681             setLabel('Name', cfg.name, false);
682             setLabel('Kind', cfg.isActive ? '（主动技能）' : '（被动技能）', true);
683             setLabel('Effect', cfg.effect, true);
684             setLabel('Intro', cfg.intro, true);
685         } else {
686             setLabel('Name', '', false);
687             setLabel('Kind', '', true);
688             setLabel('Effect', '', true);
689             setLabel('Intro', '', true);
690         }
691     }
692     private colorForAlbumSkillLevel(level: number): Color {
693         const i = Math.max(0, Math.min(4, Math.floor(level)));
694         return ALBUM_SKILL_LEVEL_COLORS[i];
695     }
696     private resolveLibSkillSpriteFrame(lib: CardSkinLibrary, skinId: number): SpriteFrame | null {

```

```
697         if (skinId < 100) {
698             return null;
699         }
700         const ids = lib.skillSpriteIds;
701         const frames = lib.skillSpriteFrames;
702         const idx = ids.indexOf(skinId);
703         if (idx < 0) {
704             return null;
705         }
706         return frames[idx] ?? null;
707     }
708     private setAllAlbumCardButtonsInteractable(interactable: boolean): void {
709         if (!this.showCardsNode) {
710             return;
711         }
712         const buttons = this.showCardsNode.getComponentsInChildren(Button);
713         for (let i = 0; i < buttons.length; i++) {
714             buttons[i].interactable = interactable;
715         }
716     }
717     private applyAlbumGridButtonsByOwnership(): void {
718         if (!this.showCardsNode) {
719             return;
720         }
721         const owned = DataManager.instance.ownedSkins;
722         const ch = this.showCardsNode.children;
723         for (let i = 0; i < ch.length; i++) {
724             const node = ch[i];
725             const sid = this._albumCardSkinId.get(node);
726             if (typeof sid !== 'number') {
727                 continue;
728             }
729             const isOwned = owned.includes(sid);
730             this.setAlbumCardButtonsInteractable(node, isOwned);
731             this.applyAlbumGridOwnedRootButtonOff(node, isOwned);
732         }
733     }
734     private applyAlbumGridButtonsWhileOneEnlarged(enlargedCard: Node): void {
735         if (!this.showCardsNode) {
736             return;
737         }
738         const ch = this.showCardsNode.children;
739         for (let i = 0; i < ch.length; i++) {
740             this.setAlbumCardButtonsInteractable(ch[i], false);
741         }
742         if (enlargedCard?.isValid) {
743             this.setAlbumCardButtonsInteractable(enlargedCard, false);
744             this.setAlbumCardFrontBarButtonsInteractable(enlargedCard, true);
745         }
746     }
747     private applyAlbumCardEnlargeOverlayVisible(card: Node, skinId: number): void {
748         if (!card?.isValid) {
749             return;
750         }
751         this.applyAlbumBackButtonStyle(card, skinId);
752         const bg = card.getChildByName('BG');
753         if (bg) {
754             this.applyAlbumBgContentSizeForEnlargedRootScale(card, bg);
```

```

755         bg.active = true;
756     }
757     const back = card.getChildByName('CardFront')?.getChildByName('back');
758     if (back) {
759         back.active = true;
760     }
761 }
762 private applyAlbumCardEnlargeOverlayHidden(card: Node): void {
763     if (!card?.isValid) {
764         return;
765     }
766     const bg = card.getChildByName('BG');
767     if (bg) {
768         this.restoreAlbumBgContentSize(card, bg);
769         bg.active = false;
770     }
771     const back = card.getChildByName('CardFront')?.getChildByName('back');
772     if (back) {
773         back.active = false;
774     }
775 }
776 private applyAlbumBgContentSizeForEnlargedRootScale(card: Node, bg: Node): void {
777     const ut = bg.getComponent(UITransform);
778     if (!ut?.isValid) {
779         return;
780     }
781     const mult = ALBUM_SELECTED_SCALE_MULT;
782     if (mult <= 0) {
783         return;
784     }
785     let sz = this._albumCardBgDesignSize.get(card);
786     if (!sz) {
787         sz = { w: ut.width, h: ut.height };
788         this._albumCardBgDesignSize.set(card, sz);
789     }
790     ut.setContentSize(sz.w / mult, sz.h / mult);
791 }
792 private restoreAlbumBgContentSize(card: Node, bg: Node): void {
793     const ut = bg.getComponent(UITransform);
794     const sz = this._albumCardBgDesignSize.get(card);
795     if (!ut?.isValid || !sz) {
796         return;
797     }
798     ut.setContentSize(sz.w, sz.h);
799 }
800 private resolveAlbumBackButtonIconFrame(lib: CardSkinLibrary, skillLevel: number): SpriteFrame | null {
801     const frames = lib.cardBackButtonSkinFrames;
802     if (!frames?.length) {
803         return null;
804     }
805     const i = Math.max(0, Math.min(4, Math.floor(skillLevel)));
806     const maxIdx = frames.length - 1;
807     const idx = Math.min(i, maxIdx);
808     return frames[idx] ?? frames[0] ?? null;
809 }
810 private resolveAlbumEquipButtonIconFrame(lib: CardSkinLibrary, skillLevel: number): SpriteFrame | null {
811     const frames = lib.cardEquipButtonSkinFrames;
812     if (!frames?.length) {

```

```
813         return null;
814     }
815     const i = Math.max(0, Math.min(4, Math.floor(skillLevel)));
816     const maxIdx = frames.length - 1;
817     const idx = Math.min(i, maxIdx);
818     return frames[idx] ?? frames[0] ?? null;
819 }
820 private applyAlbumEquipButtonStyle(card: Node, skinId: number): void {
821     const lib = this.cardSkinLibrary;
822     if (!lib || !card?.isValid) {
823         return;
824     }
825     const cfg = getSkillConfigBySkinId(skinId);
826     const lv = cfg ? Math.max(0, Math.min(4, Math.floor(cfg.skillLevel))) : 0;
827     const textColor = this.colorForAlbumSkillLevel(lv);
828     const equip = card.getChildByName('CardFront')?.getChildByName('equip');
829     const sf = this.resolveAlbumEquipButtonIconFrame(lib, lv);
830     if (equip && sf) {
831         applySpriteFrameToUINode(equip, sf);
832     }
833     const lab = equip?.getChildByName('Label')?.getComponent(Label);
834     if (lab) {
835         lab.color = textColor;
836         lab.enableOutline = true;
837         lab.outlineColor = textColor;
838     }
839 }
840 private applyAlbumBackButtonStyle(card: Node, skinId: number): void {
841     const lib = this.cardSkinLibrary;
842     if (!lib || !card?.isValid) {
843         return;
844     }
845     const cfg = getSkillConfigBySkinId(skinId);
846     const lv = cfg ? Math.max(0, Math.min(4, Math.floor(cfg.skillLevel))) : 0;
847     const tint = ALBUM_BACK_BTN_BG_BY_SKILL_LEVEL[lv] ?? ALBUM_BACK_BTN_BG_BY_SKILL_LEVEL[0];
848     const back = card.getChildByName('CardFront')?.getChildByName('back');
849     const bgNode = back?.getChildByName('bg');
850     if (bgNode) {
851         const rb = bgNode.getComponent(RoundBox);
852         if (rb) {
853             rb.color = tint;
854         } else {
855             const spr = bgNode.getComponent(Sprite);
856             if (spr) {
857                 spr.color = tint;
858             }
859         }
860     }
861     const iconNode = back?.getChildByName('icon');
862     const sf = this.resolveAlbumBackButtonIconFrame(lib, lv);
863     if (iconNode && sf) {
864         applySpriteFrameToUINode(iconNode, sf);
865     }
866 }
867 private applyPreviewSkinToCard(cardNode: Node, displaySkinId: number): void {
868     const lib = this.cardSkinLibrary;
869     if (!lib) {
870         return;
```

```

871     }
872     const textureIndex = Math.floor(displaySkinId / 100);
873     const backs = lib.cardBackSkinFrames;
874     const backLen = backs.length;
875     const maxIdx = Math.max(0, backLen - 1);
876     const clamped = Math.max(0, Math.min(textureIndex, maxIdx));
877     const backSf = backs[clamped] ?? backs[0];
878     if (!backSf) {
879         return;
880     }
881     const cardFront = cardNode.getChildByName('CardFront');
882     const cardBack = cardNode.getChildByName('CardBack');
883     if (cardFront) {
884         cardFront.active = false;
885     }
886     if (cardBack) {
887         cardBack.active = true;
888     }
889     applySpriteFrameToUINode(cardBack ?? undefined, backSf);
890     const skillNode = this.resolveSkillIconNode(cardNode);
891     if (displaySkinId >= 100) {
892         const skillIds = lib.skillSpriteIds;
893         const skillFrames = lib.skillSpriteFrames;
894         const skillIdx = skillIds.indexOf(displaySkinId);
895         const skillSf = skillIdx >= 0 ? skillFrames[skillIdx] : null;
896         if (skillSf) {
897             if (skillNode) {
898                 skillNode.active = true;
899             }
900             applySpriteFrameToUINode(skillNode ?? undefined, skillSf);
901         } else {
902             clearSpriteFrameOnUINode(skillNode ?? undefined);
903         }
904     } else {
905         clearSpriteFrameOnUINode(skillNode ?? undefined);
906     }
907 }
908 }
909 ```
910 ---
911 ## 文件: `BannerDrawWindow.ts`
912 ```typescript
913 import {
914     _decorator,
915     Button,
916     Color,
917     Component,
918     instantiate,
919     Label,
920     Node,
921     Prefab,
922     Sprite,
923     SpriteFrame,
924     tween,
925     Tween,
926     UITransform,
927     Vec3,
928 } from 'cc';

```



```
929 import { RoundBox } from '../RoundBox';
930 import { CardSkinLibrary } from './Config/CardSkinLibrary';
931 import { getSkillConfigBySkinId } from './Config/Skills';
932 import { DataManager } from './Manager/DataManager';
933 import { ToastManager } from './Manager/ToastManager';
934 import { EVENT_ENUM, AUDIO_EFFECT_ENUM } from './Utils/Enum';
935 import { COIN_UPDATE, EQUIPPED_SKIN_CHANGE, PLAY_AUDIO } from './Utils/Event';
936 import { applySpriteFrameToUINode, clearSpriteFrameOnUINode, setNodeUIOpacity } from './Utils/Utils';
937 const { ccclass, property } = _decorator;
938 const SKIN_DISPLAY_ORDER: number[] = [
939     0, 101, 102, 103, 104, 106, 107, 205, 203, 201, 202, 204, 206, 207, 301, 302, 303, 304, 401, 402,
940 ];
941 const BANNER_GRID_COLS = 4;
942 const BANNER_GRID_ROWS = 2;
943 const BANNER_GRID_CARD_SCALE = 1.0;
944 const GRID_GAP_X = 30;
945 const GRID_GAP_Y = 25;
946 const BANNER_GRID_LAYOUT_OFFSET_X = 0;
947 const BANNER_GRID_LAYOUT_OFFSET_Y = 5;
948 const BANNER_BEFORE_SPREAD_PAUSE_SEC = 0.25;
949 const BANNER_IDLE_PREVIEW_ROOT_OPACITY = Math.round(255 * 1.0);
950 const BANNER_SPREAD_MERGE_DURATION = 0.75;
951 const BANNER_AFTER_SPREAD_PAUSE_SEC = 0.75;
952 const BANNER_ROUND_PAUSE_SEC = 0.5;
953 const BANNER_DRAW_GOLD_COST = 10;
954 const BANNER_WON_ROTATE_DURATION = 0.6;
955 const BANNER_WON_SELECTED_SCALE_MULT = 4;
956 const BANNER_FLIP_HALF_DURATION = 0.375;
957 const BANNER_FLIP_THIN_X = 0.05;
958 const BANNER_MOVE_ENLARGE_DURATION = 0.75;
959 const BANNER_DRAW_BUTTON_DIM_OPACITY = Math.round(255 * 0.3);
960 const DEFAULT_BANNER_DRAW_WEIGHTS: Record<number, number> = {
961     101: 30,
962     102: 30,
963     103: 30,
964     104: 30,
965     106: 30,
966     107: 30,
967     201: 20,
968     202: 20,
969     203: 20,
970     204: 20,
971     205: 20,
972     206: 20,
973     207: 20,
974     301: 5,
975     302: 5,
976     303: 5,
977     304: 3,
978     401: 2,
979     402: 2,
980 };
981 const BANNER_SKILL_LEVEL_COLORS: Color[] = [
982     new Color(0x37, 0x1e, 0x1a),
983     new Color(0x00, 0x1f, 0x55),
984     new Color(0x3f, 0x00, 0x55),
985     new Color(0x55, 0x00, 0x00),
986     new Color(0x00, 0x00, 0x00),
```

```
987 ];
988 const BANNER_BACK_BTN_BG_BY_SKILL_LEVEL: Color[] = [
989     new Color(0xf0, 0xde, 0xdb),
990     new Color(0xe9, 0xf1, 0xff),
991     new Color(0xf9, 0xe9, 0xff),
992     new Color(0xff, 0xe9, 0xe9),
993     new Color(0xff, 0xe9, 0xf8),
994 ];
995 @ccclass('BannerDrawWindow')
996 export class BannerDrawWindow extends Component {
997     @property(Node)
998     showCardsNode: Node = null;
999     @property(Prefab)
1000     cardDisplayPrefab: Prefab = null;
1001     @property(CardSkinLibrary)
1002     cardSkinLibrary: CardSkinLibrary = null;
1003     @property	ToastManager
1004     toastManager: ToastManager = null;
1005     @property(Node)
1006     drawButtonNode: Node = null;
1007     @property(Label)
1008     goldCoinLabel: Label = null;
1009     private _cardSkinId = new WeakMap<Node, number>();
1010     private _bannerStackSiblingIndex = new WeakMap<Node, number>();
1011     private _cardGridPos = new WeakMap<Node, Vec3>();
1012     private _stackCenter = new Vec3(0, 0, 0);
1013     private _drawFlowActive = false;
1014     private _spreadMergeBusy = false;
1015     private _idleSession = 0;
1016     private _flipDriver = { u: 0 };
1017     private _moveEnlargeDriver = { t: 0 };
1018     private _flipScratchBase = new Vec3();
1019     private _flipScratchScale = new Vec3();
1020     private _flipScratchPos = new Vec3();
1021     private _enlargePending: {
1022         origPos: Vec3;
1023         origScale: Vec3;
1024         skinId: number;
1025         origSiblingIndex: number;
1026     } | null = null;
1027     private _frontUnbind: (() => void) | null = null;
1028     private _congratsShown = false;
1029     private _albumCardBgDesignSize: { w: number; h: number } | null = null;
1030     protected onLoad(): void {
1031         if (this.drawButtonNode) {
1032             this.drawButtonNode.on(Node.EventType.TOUCH_END, this.onDrawButtonClick, this);
1033         }
1034     }
1035     protected onEnable(): void {
1036         this.refreshGoldLabels();
1037         this.liftDrawButtonAboveCards();
1038         if (!this._drawFlowActive) {
1039             const s = this._idleSession;
1040             this.scheduleOnce(() => {
1041                 if (s !== this._idleSession || !this.node?.activeInHierarchy) {
1042                     return;
1043                 }
1044                 this.kickIdleLoop();
1045             });
1046         }
1047     }
1048 }
```

```
1045         }, 0);
1046     }
1047 }
1048 protected onDisable(): void {
1049     this._idleSession++;
1050     this.stopAllAnimTargets();
1051     this.unbindWonFrontActions();
1052     this.unscheduleAllCallbacks();
1053     this._spreadMergeBusy = false;
1054     this._drawFlowActive = false;
1055     this._enlargePending = null;
1056     this._congratsShown = false;
1057     if (this.showCardsNode?.isValid) {
1058         this.showCardsNode.removeAllChildren();
1059     }
1060     this.setDrawButtonInteractable(true);
1061 }
1062 protected onDestroy(): void {
1063     if (this.drawButtonNode?.isValid) {
1064         this.drawButtonNode.off(Node.EventType.TOUCH_END, this.onDrawButtonClick, this);
1065     }
1066 }
1067 onBannerShown(): void {
1068     this.refreshGoldLabels();
1069     this.liftDrawButtonAboveCards();
1070     if (!this._drawFlowActive) {
1071         const s = this._idleSession;
1072         this.scheduleOnce(() => {
1073             if (s !== this._idleSession || !this.node?.activeInHierarchy) {
1074                 return;
1075             }
1076             this.kickIdleLoop();
1077         }, 0);
1078     }
1079 }
1080 private refreshGoldLabels(): void {
1081     const g = DataManager.instance.goldCoin;
1082     if (this.goldCoinLabel) {
1083         this.goldCoinLabel.string = String(g);
1084     }
1085     COIN_UPDATE.emit(EVENT_ENUM.COIN_UPDATE);
1086 }
1087 private liftDrawButtonAboveCards(): void {
1088     const n = this.drawButtonNode;
1089     if (!n?.isValid || !n.parent) {
1090         return;
1091     }
1092     const p = n.parent;
1093     n.setSiblingIndex(Math.max(0, p.children.length - 1));
1094 }
1095 private releaseIdleSpreadMergeLock(): void {
1096     this._spreadMergeBusy = false;
1097     if (!this._drawFlowActive && this.node?.activeInHierarchy) {
1098         this.setDrawButtonInteractable(true);
1099     }
1100 }
1101 private interruptBannerAnimsAndSchedules(): void {
1102     this.stopAllAnimTargets();
```

```
1103         this.unscheduleAllCallbacks();
1104         this._idleSession++;
1105         this._spreadMergeBusy = false;
1106     }
1107     private setDrawButtonInteractable(on: boolean): void {
1108         if (!this.drawButtonNode?.isValid) {
1109             return;
1110         }
1111         setNodeUIOpacity(this.drawButtonNode, on ? 255 : BANNER_DRAW_BUTTON_DIM_OPACITY);
1112         const bs = this.drawButtonNode.getComponentsInChildren(Button);
1113         for (let i = 0; i < bs.length; i++) {
1114             bs[i].interactable = on;
1115         }
1116         const root = this.drawButtonNode.getComponent(Button);
1117         if (root) {
1118             root.interactable = on;
1119         }
1120     }
1121     private kickIdleLoop(): void {
1122         if (!this.node?.activeInHierarchy || this._drawFlowActive) {
1123             return;
1124         }
1125         this.runIdleRound();
1126     }
1127     private runIdleRound(): void {
1128         if (!this.node?.activeInHierarchy || this._drawFlowActive || this._spreadMergeBusy) {
1129             return;
1130         }
1131         if (!this.showCardsNode || !this.cardDisplayPrefab || !this.cardSkinLibrary) {
1132             console.warn('[BannerDrawWindow] 请 绑 定 showCardsNode / cardDisplayPrefab /
1133 cardSkinLibrary');
1134             return;
1135         }
1136         this._spreadMergeBusy = true;
1137         const seven = this.pickSevenWeightedDistinctFromLibrary();
1138         const stackBottomToTop = [...seven, 0];
1139         this.rebuildEightCards(stackBottomToTop, true);
1140         const beforeSpreadSessionIdle = this._idleSession;
1141         this.scheduleOnce(() => {
1142             if (
1143                 beforeSpreadSessionIdle !== this._idleSession ||
1144                 !this.node?.activeInHierarchy ||
1145                 this._drawFlowActive
1146             ) {
1147                 this.releaseIdleSpreadMergeLock();
1148                 return;
1149             }
1150             this.playSpreadTween(() => {
1151                 this.restoreBannerStackSiblingOrder();
1152                 const spreadPauseSession = this._idleSession;
1153                 this.scheduleOnce(() => {
1154                     if (
1155                         spreadPauseSession !== this._idleSession ||
1156                         !this.node?.activeInHierarchy ||
1157                         this._drawFlowActive
1158                     ) {
1159                         this.releaseIdleSpreadMergeLock();
1160                         return;
```

```

1161         }
1162         this.playMergeTween(() => {
1163             this._spreadMergeBusy = false;
1164             if (!this.node?.activeInHierarchy || this._drawFlowActive) {
1165                 return;
1166             }
1167             const pauseSession = this._idleSession;
1168             this.scheduleOnce(() => {
1169                 if (pauseSession !== this._idleSession || !this.node?.activeInHierarchy ||
1170 this._drawFlowActive) {
1171                 }
1172                 return;
1173                 this.runIdleRound();
1174             }, BANNER_ROUND_PAUSE_SEC);
1175         });
1176         }, BANNER_AFTER_SPREAD_PAUSE_SEC);
1177     });
1178     }, BANNER_BEFORE_SPREAD_PAUSE_SEC);
1179 }
1180 private onDrawButtonClick(): void {
1181     if (this._drawFlowActive) {
1182         return;
1183     }
1184     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
1185     const cost = BANNER_DRAW_GOLD_COST;
1186     if (!DataManager.instance.consumeGoldCoin(cost)) {
1187         this.toastManager?.show('「金贝贝」数量不足', 360, 450);
1188         return;
1189     }
1190     this.interruptBannerAnimsAndSchedules();
1191     this.refreshGoldLabels();
1192     this._spreadMergeBusy = false;
1193     this._drawFlowActive = true;
1194     this.setDrawButtonInteractable(false);
1195     const seven = this.pickSevenWeightedDistinctFromLibrary();
1196     const stackBottomToTop = [...seven, 0];
1197     const wonIndex = Math.floor(Math.random() * 8);
1198     const wonSkin = stackBottomToTop[wonIndex];
1199     DataManager.instance.addSkin(wonSkin);
1200     this.rebuildEightCards(stackBottomToTop, false);
1201     this._congratsShown = false;
1202     const beforeSpreadSessionDraw = this._idleSession;
1203     this.scheduleOnce(() => {
1204         if (
1205             beforeSpreadSessionDraw !== this._idleSession ||
1206             !this.node?.activeInHierarchy ||
1207             !this._drawFlowActive
1208         ) {
1209             return;
1210         }
1211         this.playSpreadTween(() => {
1212             this.restoreBannerStackSiblingOrder();
1213             const spreadPauseSession = this._idleSession;
1214             this.scheduleOnce(() => {
1215                 if (
1216                     spreadPauseSession !== this._idleSession ||
1217                     !this.node?.activeInHierarchy ||
1218                     !this._drawFlowActive

```

```

1219         ){
1220             return;
1221         }
1222         this.promoteBannerWonCardToTopSibling(wonSkin);
1223         this.playMergeTween(() => {
1224             this.beginWonCardShowcase(wonSkin);
1225         });
1226         }, BANNER_AFTER_SPREAD_PAUSE_SEC);
1227     });
1228     }, BANNER_BEFORE_SPREAD_PAUSE_SEC);
1229 }
1230 private getDrawPoolFromLibrary(): number[] {
1231     const lib = this.cardSkinLibrary;
1232     if (!lib?.skillSpriteIds?.length) {
1233         return [];
1234     }
1235     const idSet = new Set(lib.skillSpriteIds);
1236     return SKIN_DISPLAY_ORDER.filter((id) => id > 0 && idSet.has(id));
1237 }
1238 private weightForSkin(skinId: number): number {
1239     const w = DEFAULT_BANNER_DRAW_WEIGHTS[skinId];
1240     return typeof w === 'number' && w > 0 ? w : 5;
1241 }
1242 private pickSevenWeightedDistinctFromLibrary(): number[] {
1243     const pool = [...this.getDrawPoolFromLibrary()];
1244     if (pool.length === 0) {
1245         console.warn('[BannerDrawWindow] 抽卡池为空, 请检查 CardSkinLibrary.skillSpriteIds');
1246         return [101, 102, 103, 106, 107, 201, 202];
1247     }
1248     const out: number[] = [];
1249     for (let n = 0; n < 7; n++) {
1250         if (pool.length === 0) {
1251             pool.push(...this.getDrawPoolFromLibrary());
1252         }
1253         let total = 0;
1254         for (let i = 0; i < pool.length; i++) {
1255             total += this.weightForSkin(pool[i]);
1256         }
1257         let r = Math.random() * total;
1258         let pickIdx = pool.length - 1;
1259         for (let i = 0; i < pool.length; i++) {
1260             r -= this.weightForSkin(pool[i]);
1261             if (r <= 0) {
1262                 pickIdx = i;
1263                 break;
1264             }
1265         }
1266         const picked = pool[pickIdx];
1267         out.push(picked);
1268         pool.splice(pickIdx, 1);
1269     }
1270     return out;
1271 }
1272 private computeGridLayout(): {
1273     cellW: number;
1274     cellH: number;
1275     positions: Vec3[];
1276 }{

```

```

1277     const prefabRoot = this.cardDisplayPrefab!.data;
1278     const layoutUt =
1279         prefabRoot.getChildByName('CardBack')?.getComponent(UITransform) ??
1280         prefabRoot.getComponent(UITransform);
1281     const baseW = layoutUt?.width ?? 120;
1282     const baseH = layoutUt?.height ?? 160;
1283     const cellW = baseW * BANNER_GRID_CARD_SCALE;
1284     const cellH = baseH * BANNER_GRID_CARD_SCALE;
1285     const gapX = GRID_GAP_X;
1286     const gapY = GRID_GAP_Y;
1287     const cols = BANNER_GRID_COLS;
1288     const rows = BANNER_GRID_ROWS;
1289     const contentW = cols * cellW + (cols - 1) * gapX;
1290     const contentH = rows * cellH + (rows - 1) * gapY;
1291     const originX = -contentW / 2 + cellW / 2;
1292     const originY = contentH / 2 - cellH / 2;
1293     const positions: Vec3[] = [];
1294     for (let i = 0; i < cols * rows; i++) {
1295         const col = i % cols;
1296         const row = Math.floor(i / cols);
1297         const x = originX + col * (cellW + gapX) + BANNER_GRID_LAYOUT_OFFSET_X;
1298         const y = originY - row * (cellH + gapY) + BANNER_GRID_LAYOUT_OFFSET_Y;
1299         positions.push(new Vec3(x, y, 0));
1300     }
1301     return { cellW, cellH, positions };
1302 }
1303 private rebuildEightCards(stackBottomToTop: number[], isIdlePreview: boolean): void {
1304     this.showCardsNode!.removeAllChildren();
1305     const { positions } = this.computeGridLayout();
1306     const rootOpacity = isIdlePreview ? BANNER_IDLE_PREVIEW_ROOT_OPACITY : 255;
1307     for (let i = 0; i < 8; i++) {
1308         const skinId = stackBottomToTop[i];
1309         const card = instantiate(this.cardDisplayPrefab!);
1310         card.setParent(this.showCardsNode!);
1311         card.setPosition(this._stackCenter);
1312         card.setScale(BANNER_GRID_CARD_SCALE, BANNER_GRID_CARD_SCALE, 1);
1313         this._cardSkinId.set(card, skinId);
1314         this._cardGridPos.set(card, positions[i].clone());
1315         this.applyPreviewSkinToCard(card, skinId);
1316         setNodeUIOpacity(card, rootOpacity);
1317         this.setAllCardButtonsInteractable(card, false);
1318         this.applyGridRootHitBoxToCardBack(card);
1319         this.applyBannerGridOverlayHidden(card);
1320         this._bannerStackSiblingIndex.set(card, i);
1321         card.setSiblingIndex(i);
1322     }
1323 }
1324 private restoreBannerStackSiblingOrder(): void {
1325     const p = this.showCardsNode;
1326     if (!p?.isValid || p.children.length === 0) {
1327         return;
1328     }
1329     const ch = [...p.children];
1330     ch.sort(
1331         (a, b) =>
1332             (this._bannerStackSiblingIndex.get(a) ?? 99) - (this._bannerStackSiblingIndex.get(b) ?? 99),
1333     );
1334     for (let i = 0; i < ch.length; i++) {

```

```
1335         ch[i].setSiblingIndex(i);
1336     }
1337 }
1338 private applyBannerGridOverlayHidden(card: Node): void {
1339     if (!card?.isValid) {
1340         return;
1341     }
1342     const bg = card.getChildByName('BG');
1343     if (bg) {
1344         bg.active = false;
1345     }
1346     const back = card.getChildByName('CardFront')?.getChildByName('back');
1347     if (back) {
1348         back.active = false;
1349     }
1350 }
1351 private playSpreadTween(done: () => void): void {
1352     const ch = [...this.showCardsNode!.children];
1353     let remaining = ch.length;
1354     const check = (): void => {
1355         remaining--;
1356         if (remaining <= 0) {
1357             done();
1358         }
1359     };
1360     if (ch.length === 0) {
1361         done();
1362         return;
1363     }
1364     for (let i = 0; i < ch.length; i++) {
1365         const card = ch[i];
1366         const target = this._cardGridPos.get(card);
1367         if (!target) {
1368             check();
1369             continue;
1370         }
1371         Tween.stopAllByTarget(card);
1372         tween(card)
1373             .to(BANNER_SPREAD_MERGE_DURATION, { position: target.clone() }, { easing: 'sineOut' })
1374             .call(check)
1375             .start();
1376     }
1377 }
1378 private playMergeTween(done: () => void): void {
1379     const ch = [...this.showCardsNode!.children];
1380     let remaining = ch.length;
1381     const check = (): void => {
1382         remaining--;
1383         if (remaining <= 0) {
1384             done();
1385         }
1386     };
1387     if (ch.length === 0) {
1388         done();
1389         return;
1390     }
1391     for (let i = 0; i < ch.length; i++) {
1392         const card = ch[i];
```



```
1393         Tween.stopAllByTarget(card);
1394         tween(card)
1395             .to(BANNER_SPREAD_MERGE_DURATION, { position: this._stackCenter.clone() }, { easing:
1396 'sineIn' })
1397             .call(check)
1398             .start();
1399     }
1400 }
1401 private findCardBySkinId(skinId: number): Node | null {
1402     const ch = this.showCardsNode?.children ?? [];
1403     for (let i = 0; i < ch.length; i++) {
1404         if (this._cardSkinId.get(ch[i]) === skinId) {
1405             return ch[i];
1406         }
1407     }
1408     return null;
1409 }
1410 private promoteBannerWonCardToTopSibling(wonSkin: number): void {
1411     const p = this.showCardsNode;
1412     if (!p?.isValid) {
1413         return;
1414     }
1415     const card = this.findCardBySkinId(wonSkin);
1416     if (!card?.isValid) {
1417         return;
1418     }
1419     card.setSiblingIndex(Math.max(0, p.children.length - 1));
1420 }
1421 private beginWonCardShowcase(wonSkin: number): void {
1422     const card = this.findCardBySkinId(wonSkin);
1423     if (!card?.isValid || !this.showCardsNode) {
1424         this.finishDrawFlowAndRestartIdle();
1425         return;
1426     }
1427     const ch = [...this.showCardsNode.children];
1428     for (let i = 0; i < ch.length; i++) {
1429         const c = ch[i];
1430         if (c !== card) {
1431             c.active = false;
1432         }
1433     }
1434     card.setPosition(this._stackCenter);
1435     card.setScale(BANNER_GRID_CARD_SCALE, BANNER_GRID_CARD_SCALE, 1);
1436     this.applyCardRootHitBoxFromPrefabTemplate(card);
1437     const startEuler = card.eulerAngles.clone();
1438     const endEuler = new Vec3(startEuler.x, startEuler.y, startEuler.z + 360);
1439     tween(card)
1440         .to(BANNER_WON_ROTATE_DURATION, { eulerAngles: endEuler }, { easing: 'quadOut' })
1441         .call(() => {
1442             card.eulerAngles = startEuler;
1443             this.playWonFlipAndEnlarge(card, wonSkin);
1444         })
1445         .start();
1446 }
1447 private playWonFlipAndEnlarge(card: Node, skinId: number): void {
1448     Tween.stopAllByTarget(card);
1449     Tween.stopAllByTarget(this._flipDriver);
1450     Tween.stopAllByTarget(this._moveEnlargeDriver);
```

```
1451     const origSiblingIndex = card.getSiblingIndex();
1452     const half = BANNER_FLIP_HALF_DURATION;
1453     const origPos = card.position.clone();
1454     const origScale = card.scale.clone();
1455     const endScale = origScale.clone().multiplyScalar(BANNER_WON_SELECTED_SCALE_MULT);
1456     this._enlargePending = {
1457         origPos: origPos.clone(),
1458         origScale: origScale.clone(),
1459         skinId,
1460         origSiblingIndex,
1461     };
1462     this._flipDriver.u = 0;
1463     const applyFlipOnly = (): void => {
1464         this.applyFlipOnlyFrame(card, origPos, origScale, this._flipDriver.u);
1465     };
1466     applyFlipOnly();
1467     const finishEnlargeOpen = (): void => {
1468         this._moveEnlargeDriver.t = 1;
1469         this.applyMoveEnlargeFrame(card, origPos, origScale, endScale, 1);
1470         if (!card.isValid) {
1471             this._enlargePending = null;
1472             return;
1473         }
1474         this.applyWonOverlayVisible(card, skinId);
1475         this.bindWonFrontActions(card, skinId);
1476         if (!this._congratsShown) {
1477             this._congratsShown = true;
1478             this.showCongratsToast(skinId);
1479         }
1480     };
1481     tween(this._flipDriver)
1482         .to(half, { u: 0.5 }, { easing: 'linear', onUpdate: applyFlipOnly })
1483         .call(() => {
1484             this._flipDriver.u = 0.5;
1485             applyFlipOnly();
1486             if (card.isValid) {
1487                 this.showCardFrontForSkin(card, skinId);
1488             }
1489         })
1490         .to(half, { u: 1 }, { easing: 'linear', onUpdate: applyFlipOnly })
1491         .call(() => {
1492             this._flipDriver.u = 1;
1493             applyFlipOnly();
1494             this._moveEnlargeDriver.t = 0;
1495             const applyMoveEnlarge = (): void => {
1496                 this.applyMoveEnlargeFrame(
1497                     card,
1498                     origPos,
1499                     origScale,
1500                     endScale,
1501                     this._moveEnlargeDriver.t,
1502                 );
1503             };
1504             applyMoveEnlarge();
1505             tween(this._moveEnlargeDriver)
1506                 .to(
1507                     BANNER_MOVE_ENLARGE_DURATION,
1508                     { t: 1 },
```

```
1509             { easing: 'sineOut', onUpdate: applyMoveEnlarge },
1510         )
1511         .call(finishEnlargeOpen)
1512         .start();
1513     })
1514     .start();
1515 }
1516 private applyFlipMulAtU(u: number): { mx: number; my: number } {
1517     if (u <= 0.5) {
1518         const t = u * 2;
1519         return { mx: 1 + (BANNER_FLIP_THIN_X - 1) * t, my: 1 };
1520     }
1521     const t = (u - 0.5) * 2;
1522     return { mx: BANNER_FLIP_THIN_X + (1 - BANNER_FLIP_THIN_X) * t, my: 1 };
1523 }
1524 private applyFlipOnlyFrame(card: Node, origPos: Vec3, origScale: Vec3, u: number): void {
1525     if (!card?.isValid) {
1526         return;
1527     }
1528     card.setPosition(origPos);
1529     const { mx, my } = this.applyFlipMulAtU(u);
1530     this._flipScratchScale.set(origScale.x * mx, origScale.y * my, origScale.z);
1531     card.setScale(this._flipScratchScale);
1532 }
1533 private applyMoveEnlargeFrame(
1534     card: Node,
1535     origPos: Vec3,
1536     origScale: Vec3,
1537     endScale: Vec3,
1538     t: number,
1539 ): void {
1540     if (!card?.isValid) {
1541         return;
1542     }
1543     Vec3.lerp(this._flipScratchPos, origPos, Vec3.ZERO, t);
1544     Vec3.lerp(this._flipScratchBase, origScale, endScale, t);
1545     card.setPosition(this._flipScratchPos);
1546     card.setScale(this._flipScratchBase);
1547 }
1548 private onWonCardBackClick(card: Node): void {
1549     if (!this._enlargePending || !card?.isValid) {
1550         return;
1551     }
1552     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
1553     this.playWonFlipBackToStack(card);
1554 }
1555 private onWonCardEquipClick(card: Node, skinId: number): void {
1556     if (!this._enlargePending || !card?.isValid) {
1557         return;
1558     }
1559     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
1560     DataManager.instance.equippedSkinId = skinId;
1561     EQUIPPED_SKIN_CHANGE.emit(EVENT_ENUM.EQUIPPED_SKIN_CHANGE, skinId);
1562     this.showEquipToast(skinId);
1563 }
1564 private playWonFlipBackToStack(card: Node): void {
1565     const pending = this._enlargePending;
1566     if (!pending || !this.showCardsNode) {
```

```
1567         return;
1568     }
1569     this.unbindWonFrontActions();
1570     this.applyWonOverlayHidden(card);
1571     Tween.stopAllByTarget(card);
1572     Tween.stopAllByTarget(this._flipDriver);
1573     Tween.stopAllByTarget(this._moveEnlargeDriver);
1574     const { origPos, origScale, origSiblingIndex } = pending;
1575     if (card.isValid) {
1576         card.setPosition(origPos);
1577         card.setScale(origScale);
1578         const maxIdx = Math.max(0, this.showCardsNode.children.length - 1);
1579         card.setSiblingIndex(Math.min(origSiblingIndex, maxIdx));
1580     }
1581     this.setWonFrontBarButtonsInteractable(card, false);
1582     this._enlargePending = null;
1583     this.finishDrawFlowAndRestartIdle();
1584 }
1585 private finishDrawFlowAndRestartIdle(): void {
1586     this.unbindWonFrontActions();
1587     this._enlargePending = null;
1588     this._congratsShown = false;
1589     this._drawFlowActive = false;
1590     if (this.showCardsNode?.isValid) {
1591         this.showCardsNode.removeAllChildren();
1592     }
1593     this.refreshGoldLabels();
1594     this.setDrawButtonInteractable(true);
1595     if (this.node?.activeInHierarchy) {
1596         const s = this._idleSession;
1597         this.scheduleOnce(() => {
1598             if (s !== this._idleSession || !this.node?.activeInHierarchy || this._drawFlowActive) {
1599                 return;
1600             }
1601             this.runIdleRound();
1602         }, 0);
1603     }
1604 }
1605 private congratsToastBgWidth(skillName: string): number {
1606     const n = [...skillName].length;
1607     if (n <= 2) {
1608         return 340;
1609     }
1610     if (n === 3) {
1611         return 360;
1612     }
1613     return 420;
1614 }
1615 private showCongratsToast(skinId: number): void {
1616     if (!this.toastManager?.isValid) {
1617         return;
1618     }
1619     const cfg = getSkillConfigBySkinId(skinId);
1620     const name = cfg?.name?.trim() ?? '';
1621     const w = this.congratsToastBgWidth(name);
1622     const msg = name.length > 0 ? `恭喜获得「${name}」` : '恭喜获得新卡牌';
1623     this.toastManager.show(msg, w, 450);
1624 }
```

```
1625     private showEquipToast(skinId: number): void {
1626         if (!this.toastManager?.isValid) {
1627             return;
1628         }
1629         const cfg = getSkillConfigBySkinId(skinId);
1630         const name = cfg?.name?.trim() ?? '';
1631         const n = [...name].length;
1632         let w = 420;
1633         if (n <= 2) {
1634             w = 340;
1635         } else if (n === 3) {
1636             w = 360;
1637         }
1638         const msg = name.length > 0 ? `「${name}」装备成功` : '装备成功';
1639         this.toastManager.show(msg, w, 450);
1640     }
1641     private bindWonFrontActions(card: Node, skinId: number): void {
1642         this.unbindWonFrontActions();
1643         const cf = card.getChildByName('CardFront');
1644         const backNode = cf?.getChildByName('back');
1645         const equipNode = cf?.getChildByName('equip');
1646         const bgNode = card.getChildByName('BG');
1647         const onBack = (): void => {
1648             this.onWonCardBackClick(card);
1649         };
1650         const onEquip = (): void => {
1651             this.onWonCardEquipClick(card, skinId);
1652         };
1653         backNode?.on(Node.EventType.TOUCH_END, onBack, this);
1654         equipNode?.on(Node.EventType.TOUCH_END, onEquip, this);
1655         bgNode?.on(Node.EventType.TOUCH_END, onBack, this);
1656         this.setWonFrontBarButtonsInteractable(card, true);
1657         this._frontUnbind = () => {
1658             backNode?.off(Node.EventType.TOUCH_END, onBack, this);
1659             equipNode?.off(Node.EventType.TOUCH_END, onEquip, this);
1660             bgNode?.off(Node.EventType.TOUCH_END, onBack, this);
1661         };
1662     }
1663     private unbindWonFrontActions(): void {
1664         if (this._frontUnbind) {
1665             this._frontUnbind();
1666             this._frontUnbind = null;
1667         }
1668     }
1669     private setWonFrontBarButtonsInteractable(card: Node, interactable: boolean): void {
1670         const cf = card.getChildByName('CardFront');
1671         if (!cf) {
1672             return;
1673         }
1674         for (const name of ['back', 'equip']) {
1675             const n = cf.getChildByName(name);
1676             if (!n) {
1677                 continue;
1678             }
1679             const bs = n.getComponentsInChildren(Button);
1680             for (let i = 0; i < bs.length; i++) {
1681                 bs[i].interactable = interactable;
1682             }
1683         }
1684     }
```

```
1683     }
1684   }
1685   private applyWonOverlayVisible(card: Node, skinId: number): void {
1686     if (!card?.isValid) {
1687       return;
1688     }
1689     this.applyBackButtonStyle(card, skinId);
1690     const bg = card.getChildByName('BG');
1691     if (bg) {
1692       this.applyBgContentSizeForEnlargedRootScale(card, bg);
1693       bg.active = true;
1694     }
1695     const back = card.getChildByName('CardFront')?.getChildByName('back');
1696     if (back) {
1697       back.active = true;
1698     }
1699   }
1700   private applyWonOverlayHidden(card: Node): void {
1701     if (!card?.isValid) {
1702       return;
1703     }
1704     const bg = card.getChildByName('BG');
1705     if (bg) {
1706       this.restoreBgContentSize(card, bg);
1707       bg.active = false;
1708     }
1709     const back = card.getChildByName('CardFront')?.getChildByName('back');
1710     if (back) {
1711       back.active = false;
1712     }
1713   }
1714   private applyBgContentSizeForEnlargedRootScale(card: Node, bg: Node): void {
1715     const ut = bg.getComponent(UITransform);
1716     if (!ut?.isValid) {
1717       return;
1718     }
1719     const mult = BANNER_WON_SELECTED_SCALE_MULT;
1720     if (mult <= 0) {
1721       return;
1722     }
1723     if (!this._albumCardBgDesignSize) {
1724       this._albumCardBgDesignSize = { w: ut.width, h: ut.height };
1725     }
1726     const sz = this._albumCardBgDesignSize;
1727     ut.setContentSize(sz.w / mult, sz.h / mult);
1728   }
1729   private restoreBgContentSize(_card: Node, bg: Node): void {
1730     const ut = bg.getComponent(UITransform);
1731     const sz = this._albumCardBgDesignSize;
1732     if (!ut?.isValid || !sz) {
1733       return;
1734     }
1735     ut.setContentSize(sz.w, sz.h);
1736   }
1737   private resolveSkillIconNode(card: Node): Node | null {
1738     const cardBack = card.getChildByName('CardBack');
1739     return cardBack?.getChildByName('Skill') ?? card.getChildByName('Skill') ?? null;
1740   }
```

```
1741     private applyGridRootHitBoxToCardBack(card: Node): void {
1742         const rootUt = card.getComponent(UITransform);
1743         if (!rootUt?.isValid) {
1744             return;
1745         }
1746         const backUt = card.getChildByName('CardBack')?.GetComponent(UITransform);
1747         if (backUt?.isValid) {
1748             rootUt.setContentSize(backUt.width, backUt.height);
1749         } else {
1750             rootUt.setContentSize(120, 160);
1751         }
1752     }
1753     private applyCardRootHitBoxFromPrefabTemplate(card: Node): void {
1754         const rootUt = card.getComponent(UITransform);
1755         const tpl = this.cardDisplayPrefab?.data?.GetComponent(UITransform);
1756         if (!rootUt?.isValid || !tpl?.isValid) {
1757             return;
1758         }
1759         rootUt.setContentSize(tpl.width, tpl.height);
1760     }
1761     private setAllCardButtonsInteractable(card: Node, interactable: boolean): void {
1762         const buttons = card.getComponentsInChildren(Button);
1763         for (let i = 0; i < buttons.length; i++) {
1764             buttons[i].interactable = interactable;
1765         }
1766     }
1767     private applyPreviewSkinToCard(cardNode: Node, displaySkinId: number): void {
1768         const lib = this.cardSkinLibrary;
1769         if (!lib) {
1770             return;
1771         }
1772         const textureIndex = Math.floor(displaySkinId / 100);
1773         const backs = lib.cardBackSkinFrames;
1774         const backLen = backs.length;
1775         const maxIdx = Math.max(0, backLen - 1);
1776         const clamped = Math.max(0, Math.min(textureIndex, maxIdx));
1777         const backSf = backs[clamped] ?? backs[0];
1778         if (!backSf) {
1779             return;
1780         }
1781         const cardFront = cardNode.getChildByName('CardFront');
1782         const cardBack = cardNode.getChildByName('CardBack');
1783         if (cardFront) {
1784             cardFront.active = false;
1785         }
1786         if (cardBack) {
1787             cardBack.active = true;
1788         }
1789         applySpriteFrameToUINode(cardBack ?? undefined, backSf);
1790         const skillNode = this.resolveSkillIconNode(cardNode);
1791         if (displaySkinId >= 100) {
1792             const skillIds = lib.skillSpriteIds;
1793             const skillFrames = lib.skillSpriteFrames;
1794             const skillIdx = skillIds.indexOf(displaySkinId);
1795             const skillSf = skillIdx >= 0 ? skillFrames[skillIdx] : null;
1796             if (skillSf) {
1797                 if (skillNode) {
1798                     skillNode.active = true;
```

```

1799         }
1800         applySpriteFrameToUINode(skillNode ?? undefined, skillSf);
1801     } else {
1802         clearSpriteFrameOnUINode(skillNode ?? undefined);
1803     }
1804 } else {
1805     clearSpriteFrameOnUINode(skillNode ?? undefined);
1806 }
1807 }
1808 private showCardFrontForSkin(cardNode: Node, displaySkinId: number): void {
1809     const lib = this.cardSkinLibrary;
1810     if (!lib) {
1811         return;
1812     }
1813     const textureIndex = Math.floor(displaySkinId / 100);
1814     const fronts = lib.cardFrontSkinFrames;
1815     const maxIdx = Math.max(0, fronts.length - 1);
1816     const clamped = Math.max(0, Math.min(textureIndex, maxIdx));
1817     const frontSf = fronts[clamped] ?? fronts[0];
1818     if (!frontSf) {
1819         return;
1820     }
1821     const cardFront = cardNode.getChildByName('CardFront');
1822     const cardBack = cardNode.getChildByName('CardBack');
1823     if (cardBack) {
1824         cardBack.active = false;
1825     }
1826     if (cardFront) {
1827         cardFront.active = true;
1828         applySpriteFrameToUINode(cardFront, frontSf);
1829         this.applyCardShowFrontSkillAndLabels(cardFront, displaySkinId, lib);
1830         this.applyEquipButtonStyle(cardNode, displaySkinId);
1831     }
1832 }
1833 private applyCardShowFrontSkillAndLabels(cardFront: Node, displaySkinId: number, lib: CardSkinLibrary):
1834 void {
1835     const frontSkill = cardFront.getChildByName('Skill');
1836     const skillSf = this.resolveLibSkillSpriteFrame(lib, displaySkinId);
1837     if (skillSf && frontSkill) {
1838         frontSkill.active = true;
1839         applySpriteFrameToUINode(frontSkill, skillSf);
1840     } else {
1841         clearSpriteFrameOnUINode(frontSkill ?? undefined);
1842     }
1843     const cfg = getSkillConfigBySkinId(displaySkinId);
1844     const textColor = cfg ? this.colorForSkillLevel(cfg.skillLevel) : BANNER_SKILL_LEVEL_COLORS[0];
1845     const setLabel = (childName: string, text: string, outlineSameAsFill: boolean): void => {
1846         const n = cardFront.getChildByName(childName);
1847         const lab = n?.getComponent(Label);
1848         if (!lab) {
1849             return;
1850         }
1851         lab.string = text;
1852         lab.color = textColor;
1853         if (outlineSameAsFill) {
1854             lab.enableOutline = true;
1855             lab.outlineColor = textColor;
1856         }
1857     }

```



```
1857     };
1858     if (cfg) {
1859         setLabel('Name', cfg.name, false);
1860         setLabel('Kind', cfg.isActive ? '（主动技能）' : '（被动技能）', true);
1861         setLabel('Effect', cfg.effect, true);
1862         setLabel('Intro', cfg.intro, true);
1863     } else {
1864         setLabel('Name', '', false);
1865         setLabel('Kind', '', true);
1866         setLabel('Effect', '', true);
1867         setLabel('Intro', '', true);
1868     }
1869 }
1870 private colorForSkillLevel(level: number): Color {
1871     const i = Math.max(0, Math.min(4, Math.floor(level)));
1872     return BANNER_SKILL_LEVEL_COLORS[i];
1873 }
1874 private resolveLibSkillSpriteFrame(lib: CardSkinLibrary, skinId: number): SpriteFrame | null {
1875     if (skinId < 100) {
1876         return null;
1877     }
1878     const ids = lib.skillSpriteIds;
1879     const frames = lib.skillSpriteFrames;
1880     const idx = ids.indexOf(skinId);
1881     if (idx < 0) {
1882         return null;
1883     }
1884     return frames[idx] ?? null;
1885 }
1886 private resolveBackButtonIconFrame(lib: CardSkinLibrary, skillLevel: number): SpriteFrame | null {
1887     const frames = lib.cardBackButtonSkinFrames;
1888     if (!frames?.length) {
1889         return null;
1890     }
1891     const i = Math.max(0, Math.min(4, Math.floor(skillLevel)));
1892     const maxIdx = frames.length - 1;
1893     const idx = Math.min(i, maxIdx);
1894     return frames[idx] ?? frames[0] ?? null;
1895 }
1896 private resolveEquipButtonIconFrame(lib: CardSkinLibrary, skillLevel: number): SpriteFrame | null {
1897     const frames = lib.cardEquipButtonSkinFrames;
1898     if (!frames?.length) {
1899         return null;
1900     }
1901     const i = Math.max(0, Math.min(4, Math.floor(skillLevel)));
1902     const maxIdx = frames.length - 1;
1903     const idx = Math.min(i, maxIdx);
1904     return frames[idx] ?? frames[0] ?? null;
1905 }
1906 private applyEquipButtonStyle(card: Node, skinId: number): void {
1907     const lib = this.cardSkinLibrary;
1908     if (!lib || !card?.isValid) {
1909         return;
1910     }
1911     const cfg = getSkillConfigBySkinId(skinId);
1912     const lv = cfg ? Math.max(0, Math.min(4, Math.floor(cfg.skillLevel))) : 0;
1913     const textColor = this.colorForSkillLevel(lv);
1914     const equip = card.getChildByName('CardFront')?.getChildByName('equip');
```

```

1915         const sf = this.resolveEquipButtonIconFrame(lib, lv);
1916         if (equip && sf) {
1917             applySpriteFrameToUINode(equip, sf);
1918         }
1919         const lab = equip?.getChildByName('Label')?.getComponent(Label);
1920         if (lab) {
1921             lab.color = textColor;
1922             lab.enableOutline = true;
1923             lab.outlineColor = textColor;
1924         }
1925     }
1926     private applyBackButtonStyle(card: Node, skinId: number): void {
1927         const lib = this.cardSkinLibrary;
1928         if (!lib || !card?.isValid) {
1929             return;
1930         }
1931         const cfg = getSkillConfigBySkinId(skinId);
1932         const lv = cfg ? Math.max(0, Math.min(4, Math.floor(cfg.skillLevel))) : 0;
1933         const tint = BANNER_BACK_BTN_BG_BY_SKILL_LEVEL[lv] ??
1934         BANNER_BACK_BTN_BG_BY_SKILL_LEVEL[0];
1935         const back = card.getChildByName('CardFront')?.getChildByName('back');
1936         const bgNode = back?.getChildByName('bg');
1937         if (bgNode) {
1938             const rb = bgNode.getComponent(RoundBox);
1939             if (rb) {
1940                 rb.color = tint;
1941             } else {
1942                 const spr = bgNode.getComponent(Sprite);
1943                 if (spr) {
1944                     spr.color = tint;
1945                 }
1946             }
1947         }
1948         const iconNode = back?.getChildByName('icon');
1949         const sf = this.resolveBackButtonIconFrame(lib, lv);
1950         if (iconNode && sf) {
1951             applySpriteFrameToUINode(iconNode, sf);
1952         }
1953     }
1954     private stopAllAnimTargets(): void {
1955         Tween.stopAllByTarget(this._flipDriver);
1956         Tween.stopAllByTarget(this._moveEnlargeDriver);
1957         if (this.showCardsNode?.isValid) {
1958             const ch = this.showCardsNode.children;
1959             for (let i = 0; i < ch.length; i++) {
1960                 Tween.stopAllByTarget(ch[i]);
1961             }
1962         }
1963     }
1964 }
1965 ```
1966 ---
1967 ## 文件: `CardMove.ts`
1968 ```typescript
1969 import { _decorator, Component, Node, tween, Vec3 } from 'cc';
1970 const { ccclass, property } = _decorator;
1971 @ccclass('CardMove')
1972 export class CardMove extends Component {

```

```

1973     @property([Node])
1974     movePointsTrans: Node[] = [];
1975     private cardFront: Node = null;
1976     private cardBack: Node = null;
1977     private isShowingFront: boolean = false;
1978     start() {
1979         this.cardFront = this.node.getChildByName('CardFront');
1980         this.cardBack = this.node.getChildByName('CardBack');
1981         if(this.movePointsTrans.length>0){
1982             this.moveAlongPath();
1983         }
1984     }
1985     moveAlongPath(){
1986         let points=this.movePointsTrans.map(point=>point.getPosition());
1987         let currentIndex=0;
1988         let isFirstMove = true; // 标记是否是第一次移动
1989         const moveToNextPoint=()=>{
1990             let targetPos=points[currentIndex];
1991             currentIndex=(currentIndex+1)%points.length;
1992             this.flipCardAndMove(targetPos, moveToNextPoint, isFirstMove);
1993             isFirstMove = false; // 第一次后设为 false
1994         }
1995         moveToNextPoint();
1996     }
1997     flipCardAndMove(targetPos: Vec3, callback: () => void, isFirstMove: boolean = false) {
1998         const moveDuration = 1.5; // 总移动时间
1999         tween(this.node)
2000             .to(moveDuration, { position: targetPos }, { easing: 'smooth' })
2001             .delay(0.5) // 停顿 0.5 秒
2002             .call(callback)
2003             .start();
2004         if (isFirstMove) {
2005             return;
2006         }
2007         const flipDuration = 0.75; // 翻牌时间（一半）
2008         const currentScale = this.node.scale.clone();
2009         tween(this.node)
2010             .to(flipDuration, {
2011                 scale: new Vec3(0.05, currentScale.y, currentScale.z)
2012             }, { easing: 'linear' })
2013             .call(() => {
2014                 this.switchCardFace();
2015             })
2016             .to(flipDuration, {
2017                 scale: currentScale
2018             }, { easing: 'linear' })
2019             .start();
2020     }
2021     switchCardFace() {
2022         this.isShowingFront = !this.isShowingFront;
2023         if (this.cardFront) {
2024             this.cardFront.active = this.isShowingFront;
2025         }
2026         if (this.cardBack) {
2027             this.cardBack.active = !this.isShowingFront;
2028         }
2029     }
2030     update(deltaTime: number) {

```

```
2031     }
2032 }
2033 ...
2034 ---
2035 ## 文件: `CardRandNum.ts`
2036 ```typescript
2037 import { _decorator, Component, Label, Sprite, SpriteFrame } from "cc";
2038 import { RoundBox } from "../RoundBox";
2039 import { CardSkinLibrary } from "../Config/CardSkinLibrary";
2040 import { DataManager } from "../Manager/DataManager";
2041 import { EVENT_ENUM } from "../Utils/Enum";
2042 import { EQUIPPED_SKIN_CHANGE } from "../Utils/Event";
2043 import { applySpriteFrameToUINode, clearSpriteFrameOnUINode, shuffle } from "../Utils/Utils";
2044 const { ccclass, property } = _decorator;
2045 @ccclass("CardRandNum")
2046 export class CardRandNum extends Component {
2047     @property
2048     cardCount: number = 8;
2049     @property(CardSkinLibrary)
2050     cardSkinLibrary: CardSkinLibrary = null;
2051     protected onLoad(): void {
2052         EQUIPPED_SKIN_CHANGE.on(EVENT_ENUM.EQUIPPED_SKIN_CHANGE, this.onEquippedSkinChange,
2053 this);
2054     }
2055     protected onDestroy(): void {
2056         EQUIPPED_SKIN_CHANGE.off(EVENT_ENUM.EQUIPPED_SKIN_CHANGE, this.onEquippedSkinChange,
2057 this);
2058     }
2059     start() {
2060         this.generateRandomNumbers();
2061     }
2062     private onEquippedSkinChange(): void {
2063         this.applyEquippedSkinTextures();
2064     }
2065     private effectiveFrontFrames(): SpriteFrame[] {
2066         return this.cardSkinLibrary?.cardFrontSkinFrames ?? [];
2067     }
2068     private effectiveBackFrames(): SpriteFrame[] {
2069         return this.cardSkinLibrary?.cardBackSkinFrames ?? [];
2070     }
2071     private effectiveSkillIds(): number[] {
2072         return this.cardSkinLibrary?.skillSpriteIds ?? [];
2073     }
2074     private effectiveSkillFrames(): SpriteFrame[] {
2075         return this.cardSkinLibrary?.skillSpriteFrames ?? [];
2076     }
2077     generateRandomNumbers(): void {
2078         const numbers: number[] = [];
2079         for (let i = 1; i <= this.cardCount; i++) {
2080             numbers.push(i);
2081         }
2082         const shuffledNumbers = shuffle(numbers);
2083         for (let i = 1; i <= this.cardCount; i++) {
2084             const label = this.node
2085                 .getChildByName(`Card${i}`)
2086                 ?.getChildByName("CardFront")
2087                 ?.getChildByName("Number")
2088                 ?.getComponent(Label);
```

```

2089         if (label) {
2090             label.string = shuffledNumbers[i - 1].toString();
2091         }
2092     }
2093     this.applyEquippedSkinTextures();
2094 }
2095 private resolveSkinTextureIndex(equippedSkinId: number): number {
2096     return Math.floor(equippedSkinId / 100);
2097 }
2098 private applyEquippedSkinTextures(): void {
2099     const equippedSkinId = DataManager.instance.equippedSkinId;
2100     const textureIndex = this.resolveSkinTextureIndex(equippedSkinId);
2101     const fronts = this.effectiveFrontFrames();
2102     const backs = this.effectiveBackFrames();
2103     const frontLen = fronts.length;
2104     const backLen = backs.length;
2105     const maxIdx = Math.max(0, Math.min(frontLen, backLen) - 1);
2106     const clamped = Math.max(0, Math.min(textureIndex, maxIdx));
2107     const frontSf = fronts[clamped] ?? fronts[0];
2108     const backSf = backs[clamped] ?? backs[0];
2109     console.log(
2110         `[CardRandNum] equippedSkinId=${equippedSkinId} → textureIndex=${textureIndex} →
2111         clamped=${clamped} | frontLen=${frontLen} backLen=${backLen} | frontOk=${!!frontSf} backOk=${!!backSf}`
2112     );
2113     if (textureIndex !== clamped && maxIdx >= 0) {
2114         console.warn(
2115             `[CardRandNum] 贴图下标 ${textureIndex} 超出已绑定数量（最大 ${maxIdx}），已回退
2116             为 ${clamped}`
2117         );
2118     }
2119     if (!frontSf || !backSf) {
2120         console.warn(
2121             "CardRandNum: 未绑定皮肤图。请在 CardSkinLibrary 上配置 cardFrontSkinFrames /
2122             cardBackSkinFrames（至少 [0]），且正背面数量一致，并在本组件指定 cardSkinLibrary"
2123         );
2124         return;
2125     }
2126     const skillSf =
2127         equippedSkinId >= 100 ? this.resolveSkillSpriteFrame(equippedSkinId) : null;
2128     if (equippedSkinId >= 100) {
2129         console.log(
2130             `[CardRandNum] Skill equippedSkinId=${equippedSkinId} skillSf=${skillSf ? "ok" : "missing"} |
2131             skillIdsLen=${this.effectiveSkillIds().length} skillFramesLen=${this.effectiveSkillFrames().length}`
2132         );
2133     }
2134     let skillStructureWarned = false;
2135     for (let i = 1; i <= this.cardCount; i++) {
2136         const card = this.node.getChildByName(`Card${i}`);
2137         if (!card) {
2138             continue;
2139         }
2140         const cardFront = card.getChildByName("CardFront");
2141         const cardBack = card.getChildByName("CardBack");
2142         applySpriteFrameToUINode(cardBack, backSf);
2143         const skillNode =
2144             cardBack?.getChildByName("Skill") ?? card.getChildByName("Skill");
2145         if (equippedSkinId >= 100 && skillSf) {
2146             const hasSkillRenderer =

```

```

2147         !!skillNode?.getComponent(RoundBox) || !!skillNode?.getComponent(Sprite);
2148     if (skillNode) {
2149         skillNode.active = true;
2150     }
2151     applySpriteFrameToUINode(skillNode ?? undefined, skillSf);
2152     if (!hasSkillRenderer && !skillStructureWarned) {
2153         skillStructureWarned = true;
2154         if (skillNode) {
2155             console.warn("CardRandNum: Skill 节点上未找到 RoundBox 或 Sprite 组件");
2156         } else {
2157             console.warn(
2158                 "CardRandNum: 未找到 Skill 节点（优先查 CardBack/Skill，否则与
2159 CardFront 同级的 Skill）"
2160             );
2161         }
2162     }
2163 } else {
2164     clearSpriteFrameOnUINode(skillNode ?? undefined);
2165 }
2166 applySpriteFrameToUINode(cardFront, frontSf);
2167 }
2168 }
2169 private resolveSkillSpriteFrame(equippedSkinId: number): SpriteFrame | null {
2170     const ids = this.effectiveSkillIds();
2171     const frames = this.effectiveSkillFrames();
2172     if (ids.length !== frames.length) {
2173         console.warn(
2174             `[CardRandNum] skillSpriteIds 与 skillSpriteFrames 数量不一致 (${ids.length} vs
2175 ${frames.length})`
2176         );
2177     }
2178     const idx = ids.indexOf(equippedSkinId);
2179     if (idx < 0) {
2180         console.warn(
2181             `[CardRandNum] Skill 未配置业务 ID ${equippedSkinId}: 请在 CardSkinLibrary 的
2182 skillSpriteIds 中加入 ${equippedSkinId}，并在 skillSpriteFrames 同下标拖入
2183 assets/Sprites/Skill/${equippedSkinId}.png`
2184         );
2185         return null;
2186     }
2187     const sf = frames[idx];
2188     if (!sf) {
2189         console.warn(`[CardRandNum] skillSpriteFrames[${idx}] 为空`);
2190     }
2191     return sf ?? null;
2192 }
2193 }
2194 ```
2195 ---
2196 ## 文件: `Config/CardSkinLibrary.ts`
2197 ```typescript
2198 import { _decorator, CCInteger, Component, SpriteFrame } from 'cc';
2199 const { ccclass, property } = _decorator;
2200 @ccclass('CardSkinLibrary')
2201 export class CardSkinLibrary extends Component {
2202     @property([SpriteFrame])
2203     cardFrontSkinFrames: SpriteFrame[] = [];
2204     @property([SpriteFrame])

```

```

2205     cardBackSkinFrames: SpriteFrame[] = [];
2206     @property([SpriteFrame])
2207     cardBackButtonSkinFrames: SpriteFrame[] = [];
2208     @property([SpriteFrame])
2209     cardEquipButtonSkinFrames: SpriteFrame[] = [];
2210     @property([CCInteger])
2211     skillSpriteIds: number[] = [];
2212     @property([SpriteFrame])
2213     skillSpriteFrames: SpriteFrame[] = [];
2214 }
2215 ...
2216 ---
2217 ## 文件: `Config/DebugMockSave.ts`
2218 ```typescript
2219 import type { PlayerData } from './GameTypes';
2220 export const USE_DEBUG MOCK_SAVE = false;
2221 export const MOCK_PLAYER_DATA: Partial<PlayerData> = {
2222     silverCoin: 10000,
2223     goldCoin: 5000,
2224     ownedSkins: [0, 101, 103, 106, 107, 202, 203, 204, 205, 207, 301, 302, 304, 401, 402],
2225     equippedSkinId: 0,
2226 };
2227 ...
2228 ---
2229 ## 文件: `Config/GameTypes.ts`
2230 ```typescript
2231 import { CARD_STATE_ENUM, SKIN_RARITY_ENUM, ACTIVE_SKILL_ENUM, PASSIVE_SKILL_ENUM } from
2232 '../Utils/Enum';
2233 export interface LevelConfig {
2234     levelId: number; // 关卡 ID
2235     totalCard: number; // 总卡片数 (包含炸弹卡)
2236     realCard: number; // 真实卡片数 (totalCard > realCard 时, 差值即为炸弹卡数量)
2237     displayTime: number; // 展示时间 (秒)
2238     swapDegree: number;
2239     swapDegree2: number;
2240     skillCost: number; // 技能消耗 (银贝贝)
2241     rewardSilver: number; // 银贝贝奖励
2242     rewardGold: number; // 金贝贝奖励
2243 }
2244 export const SWAP_KIND_NONE = 0;
2245 export const SWAP_KIND_PAIR = 1;
2246 export const SWAP_KIND_HOLE = 2;
2247 export type ChunQiuChanSwapSlotRecord =
2248     | { kind: 'pair'; a: number; b: number }
2249     | { kind: 'hole'; cardId: number; cardX: number; cardY: number; holeX: number; holeY: number };
2250 export interface CardData {
2251     id: number; // 卡片唯一 ID
2252     number: number; // 数字 (数字卡片: 1-N, 炸弹卡片: 0)
2253     x: number; // 网格 X 坐标
2254     y: number; // 网格 Y 坐标
2255     state: CARD_STATE_ENUM; // 卡片状态
2256     isClicked: boolean; // 是否已点击
2257 }
2258 export interface SkinConfig {
2259     skinId: number; // 皮肤 ID
2260     skinName: string; // 皮肤名称
2261     rarity: SKIN_RARITY_ENUM; // 皮肤等级
2262     activeSkills: ACTIVE_SKILL_ENUM[]; // 主动技能列表

```

```

2263     passiveSkills: PASSIVE_SKILL_ENUM[]; // 被动技能列表
2264     iconPath: string; // 图标路径
2265 }
2266 export interface PlayerData {
2267     silverCoin: number; // 银贝贝数量
2268     goldCoin: number; // 金贝贝数量
2269     ownedSkins: number[]; // 已拥有皮肤 ID 列表（默认含 0 号默认皮）
2270     equippedSkinId: number; // 当前装配皮肤 ID（0 为默认皮）
2271     lastdaily: string;
2272 }
2273 export interface SkillUseData {
2274     skillType: ACTIVE_SKILL_ENUM | PASSIVE_SKILL_ENUM;
2275     cardId?: number; // 相关卡片 ID（如验牌技能）
2276     levelId?: number; // 相关关卡 ID
2277 }
2278 export type SkillLevelGrade = 0 | 1 | 2 | 3 | 4;
2279 export interface SkillConfigEntry {
2280     skinId: number | null;
2281     name: string;
2282     skillLevel: SkillLevelGrade;
2283     isActive: boolean;
2284     effect: string;
2285     intro: string;
2286 }
2287 ...
2288 ---
2289 ## 文件: `Config/Level.ts`
2290 ```typescript
2291 import { LevelConfig } from './GameTypes';
2292 export const levels: LevelConfig[] = [
2293     {
2294         levelId: 1,
2295         totalCard: 4,
2296         realCard: 4,
2297         displayTime: 3.0,
2298         swapDegree: 0,
2299         swapDegree2: 0,
2300         skillCost: 100,
2301         rewardSilver: 100,
2302         rewardGold: 2,
2303     },
2304     {
2305         levelId: 2,
2306         totalCard: 5,
2307         realCard: 5,
2308         displayTime: 4.0,
2309         swapDegree: 0,
2310         swapDegree2: 0,
2311         skillCost: 100,
2312         rewardSilver: 100,
2313         rewardGold: 2,
2314     },
2315     {
2316         levelId: 3,
2317         totalCard: 5,
2318         realCard: 4,
2319         displayTime: 4.0,
2320         swapDegree: 0,

```



```
2321         swapDegree2: 0,
2322         skillCost: 100,
2323         rewardSilver: 150,
2324         rewardGold: 4,
2325     },
2326     {
2327         levelId: 4,
2328         totalCard: 6,
2329         realCard: 5,
2330         displayTime: 4.0,
2331         swapDegree: 0,
2332         swapDegree2: 0,
2333         skillCost: 100,
2334         rewardSilver: 150,
2335         rewardGold: 2,
2336     },
2337     {
2338         levelId: 5,
2339         totalCard: 6,
2340         realCard: 5,
2341         displayTime: 4.0,
2342         swapDegree: 0,
2343         swapDegree2: 0.25,
2344         skillCost: 100,
2345         rewardSilver: 200,
2346         rewardGold: 2,
2347     },
2348     {
2349         levelId: 6,
2350         totalCard: 7,
2351         realCard: 5,
2352         displayTime: 4.5,
2353         swapDegree: 0,
2354         swapDegree2: 0.25,
2355         skillCost: 200,
2356         rewardSilver: 200,
2357         rewardGold: 6,
2358     },
2359     {
2360         levelId: 7,
2361         totalCard: 7,
2362         realCard: 6,
2363         displayTime: 5.0,
2364         swapDegree: 0,
2365         swapDegree2: 0.25,
2366         skillCost: 200,
2367         rewardSilver: 250,
2368         rewardGold: 2,
2369     },
2370     {
2371         levelId: 8,
2372         totalCard: 8,
2373         realCard: 6,
2374         displayTime: 5.5,
2375         swapDegree: 0.25,
2376         swapDegree2: 0,
2377         skillCost: 300,
2378         rewardSilver: 250,
```

```
2379         rewardGold: 2,
2380     },
2381     {
2382         levelId: 9,
2383         totalCard: 9,
2384         realCard: 6,
2385         displayTime: 5.5,
2386         swapDegree: 0.25,
2387         swapDegree2: 0,
2388         skillCost: 300,
2389         rewardSilver: 300,
2390         rewardGold: 2,
2391     },
2392     {
2393         levelId: 10,
2394         totalCard: 9,
2395         realCard: 6,
2396         displayTime: 6.0,
2397         swapDegree: 0.4,
2398         swapDegree2: 0,
2399         skillCost: 300,
2400         rewardSilver: 300,
2401         rewardGold: 8,
2402     },
2403     {
2404         levelId: 11,
2405         totalCard: 9,
2406         realCard: 7,
2407         displayTime: 6.0,
2408         swapDegree: 0.3,
2409         swapDegree2: 0,
2410         skillCost: 500,
2411         rewardSilver: 350,
2412         rewardGold: 4,
2413     },
2414     {
2415         levelId: 12,
2416         totalCard: 9,
2417         realCard: 8,
2418         displayTime: 7.0,
2419         swapDegree: 0.4,
2420         swapDegree2: 0,
2421         skillCost: 500,
2422         rewardSilver: 350,
2423         rewardGold: 4,
2424     },
2425     {
2426         levelId: 13,
2427         totalCard: 10,
2428         realCard: 8,
2429         displayTime: 7.0,
2430         swapDegree: 0.25,
2431         swapDegree2: 0.25,
2432         skillCost: 500,
2433         rewardSilver: 400,
2434         rewardGold: 4,
2435     },
2436     {
```

```
2437         levelId: 14,
2438         totalCard: 11,
2439         realCard: 7,
2440         displayTime: 8.0,
2441         swapDegree: 0.25,
2442         swapDegree2: 0.25,
2443         skillCost: 500,
2444         rewardSilver: 400,
2445         rewardGold: 10,
2446     },
2447     {
2448         levelId: 15,
2449         totalCard: 11,
2450         realCard: 8,
2451         displayTime: 8.0,
2452         swapDegree: 0.25,
2453         swapDegree2: 0.25,
2454         skillCost: 500,
2455         rewardSilver: 450,
2456         rewardGold: 6,
2457     },
2458     {
2459         levelId: 16,
2460         totalCard: 12,
2461         realCard: 8,
2462         displayTime: 9.0,
2463         swapDegree: 0.4,
2464         swapDegree2: 0.25,
2465         skillCost: 800,
2466         rewardSilver: 450,
2467         rewardGold: 6,
2468     },
2469     {
2470         levelId: 17,
2471         totalCard: 12,
2472         realCard: 9,
2473         displayTime: 10.0,
2474         swapDegree: 0.4,
2475         swapDegree2: 0.25,
2476         skillCost: 800,
2477         rewardSilver: 500,
2478         rewardGold: 6,
2479     },
2480     {
2481         levelId: 18,
2482         totalCard: 12,
2483         realCard: 8,
2484         displayTime: 10.0,
2485         swapDegree: 0.5,
2486         swapDegree2: 0.25,
2487         skillCost: 800,
2488         rewardSilver: 800,
2489         rewardGold: 12,
2490     },
2491     {
2492         levelId: 19,
2493         totalCard: 12,
2494         realCard: 9,
```

```

2495         displayTime: 11.0,
2496         swapDegree: 0.5,
2497         swapDegree2: 0.25,
2498         skillCost: 800,
2499         rewardSilver: 800,
2500         rewardGold: 10,
2501     },
2502     {
2503         levelId: 20,
2504         totalCard: 14,
2505         realCard: 9,
2506         displayTime: 12.0,
2507         swapDegree: 0.6,
2508         swapDegree2: 0.3,
2509         skillCost: 800,
2510         rewardSilver: 1000,
2511         rewardGold: 14,
2512     },
2513 ];
2514 export function getLevelConfig(levelId: number): LevelConfig | null {
2515     return levels.find(level => level.levelId === levelId) || null;
2516 }
2517 export function getTotalLevelCount(): number {
2518     return levels.length;
2519 }
2520 ```
2521 ---
2522 ## 文件: `Config/Skills.ts`
2523 ```typescript
2524 import { SkillConfigEntry } from './GameTypes';
2525 export const skillConfigs: SkillConfigEntry[] = [
2526     {
2527         skinId: 401,
2528         name: '春秋蝉',
2529         skillLevel: 4,
2530         isActive: true,
2531         effect: '重开本局，卡片顺序不变，所有卡片翻面，重新进入记忆阶段（记忆时间等同本局时间）。',
2532     },
2533     {
2534         intro: '落魄谷中寒风吹，春秋蝉鸣少年归。',
2535     },
2536     {
2537         skinId: 301,
2538         name: '我要验牌',
2539         skillLevel: 3,
2540         isActive: true,
2541         effect: '可选择一张牌进行查验，得知卡牌的具体数字或炸弹。',
2542         intro: '一瞬窥天机，数显难藏。',
2543     },
2544     {
2545         skinId: 302,
2546         name: '序列指引',
2547         skillLevel: 3,
2548         isActive: true,
2549         effect: '翻转下一张正确序列卡片。',
2550         intro: '前路迷离，天启一指，序牌自现。',
2551     },
2552     {
2553         skinId: 202,

```

```
2553     name: '拆弹专家',
2554     skillLevel: 2,
2555     isActive: true,
2556     effect: '剔除当前两张炸弹牌。',
2557     intro: '双雷除去，步履从容，劫数暂消。',
2558 },
2559 {
2560     skinId: 203,
2561     name: '言出法随',
2562     skillLevel: 2,
2563     isActive: true,
2564     effect: '点击的下一张卡片必为正确卡片（如果是错的，当前卡片会和正确卡片互换位置）。',
2565     intro: '言落法随，错者移位，正者自来。',
2566 },
2567 {
2568     skinId: 101,
2569     name: '再来一局',
2570     skillLevel: 1,
2571     isActive: true,
2572     effect: '重开本局，卡片随机重新生成，开始新一轮游戏。',
2573     intro: '万象更新，天地重开。',
2574 },
2575 {
2576     skinId: 102,
2577     name: '拆弹能手',
2578     skillLevel: 1,
2579     isActive: true,
2580     effect: '随机剔除当前一张炸弹牌。',
2581     intro: '一雷随风去，前路少惊心。',
2582 },
2583 {
2584     skinId: 103,
2585     name: '预言家',
2586     skillLevel: 1,
2587     isActive: true,
2588     effect: '可选择一张牌进行查验，得知卡牌是数字或炸弹。',
2589     intro: '勘破阴阳，数雷瞬明。',
2590 },
2591 {
2592     skinId: 402,
2593     name: '金贝贝',
2594     skillLevel: 4,
2595     isActive: false,
2596     effect: '每局结束之后获得金贝贝奖励+50%。',
2597     intro: '金雨落，余财盈。',
2598 },
2599 {
2600     skinId: 303,
2601     name: '万古匆匆',
2602     skillLevel: 3,
2603     isActive: false,
2604     effect: '展示阶段的数字停留时间增加 2 秒。',
2605     intro: '执念深种，万古匆匆。',
2606 },
2607 {
2608     skinId: 304,
2609     name: '凤凰蛊',
2610     skillLevel: 3,
```

```
2611         isActive: false,
2612         effect: '首次点击错误牌，错误牌反转展示后，盖回，游戏继续。',
2613         intro: '凤凰涅槃，起死回生，一生只有一盅。',
2614     },
2615     {
2616         skinId: 201,
2617         name: '银贝贝',
2618         skillLevel: 2,
2619         isActive: false,
2620         effect: '每局结束之后获得银贝贝奖励+50%。',
2621         intro: '银光落，月盈缺。',
2622     },
2623     {
2624         skinId: 204,
2625         name: '龙胤之力',
2626         skillLevel: 2,
2627         isActive: false,
2628         effect: '首次点击错误牌，错误牌反转展示后，所有翻开的牌反转盖上，游戏继续。',
2629         intro: '樱龙赋予凡人的不死之力。',
2630     },
2631     {
2632         skinId: 205,
2633         name: '移花接木',
2634         skillLevel: 2,
2635         isActive: false,
2636         effect: '卡片交换触发时，已翻开的卡片也会参与交换。',
2637         intro: '花开移影，明者替，暗者藏。',
2638     },
2639     {
2640         skinId: 206,
2641         name: '不动明王',
2642         skillLevel: 2,
2643         isActive: false,
2644         effect: '卡片交换概率降为原先的 50%。',
2645         intro: '明王镇岳，余者安然。',
2646     },
2647     {
2648         skinId: 207,
2649         name: '岁月神偷',
2650         skillLevel: 2,
2651         isActive: false,
2652         effect: '展示阶段的数字停留时间增加 1 秒。',
2653         intro: '岁月留情，且记且惜。',
2654     },
2655     {
2656         skinId: 104,
2657         name: '奇变',
2658         skillLevel: 1,
2659         isActive: false,
2660         effect: '奇数卡片背面显示“奇”。',
2661         intro: '奇变偶不变，符号看象限。',
2662     },
2663     {
2664         skinId: 105,
2665         name: '让你三招',
2666         skillLevel: 1,
2667         isActive: false,
2668         effect: '前 3 次点击不触发位置交换。',
```

```

2669         intro: '三招之内，乾坤不换。',
2670     },
2671     {
2672         skinId: 106,
2673         name: '乱金桥',
2674         skillLevel: 1,
2675         isActive: false,
2676         effect: '展示阶段卡片盖牌动画速度降低为原先的 30%。',
2677         intro: '此间方寸，时空皆乱。',
2678     },
2679     {
2680         skinId: 107,
2681         name: '木头人',
2682         skillLevel: 1,
2683         isActive: false,
2684         effect: '卡片交换概率降为原先的 80%。',
2685         intro: '木人不语，略减风霜。',
2686     },
2687     {
2688         skinId: 0,
2689         name: '平凡',
2690         skillLevel: 0,
2691         isActive: false,
2692         effect: '默认卡牌皮肤，没有特殊技能加成。',
2693         intro: '修炼尘世中，平凡不平庸。',
2694     },
2695 ];
2696 export function getSkillConfigBySkinId(skinId: number): SkillConfigEntry | null {
2697     return skillConfigs.find((c) => c.skinId === skinId) ?? null;
2698 }
2699 export function getSkillConfigsWithSkinId(): SkillConfigEntry[] {
2700     return skillConfigs.filter((c): c is SkillConfigEntry & { skinId: number } => c.skinId !== null);
2701 }
2702 ```
2703 ---
2704 ## 文件: `GameObject/Card.ts`
2705 ```typescript
2706 import { _decorator, Component, Node, Label, tween, Tween, Vec3 } from 'cc';
2707 import { ACTIVE_SKILL_CARD_INTERCEPT_MODE, CARD_STATE_ENUM, EVENT_ENUM, GAME_STATE_ENUM }
2708 from '../Utils/Enum';
2709 import { ACTIVE_SKILL_PROPHET_CARD_TAP, ACTIVE_SKILL_VERIFY_CARD_TAP, CARD_CLICK } from
2710 '../Utils/Event';
2711 import { CardData } from '../Config/GameTypes';
2712 import { DataManager } from '../Manager/DataManager';
2713 const { ccclass, property } = _decorator;
2714 @ccclass('Card')
2715 export class Card extends Component {
2716     @property(Node)
2717     frontNode: Node = null;
2718     @property(Node)
2719     backNode: Node = null;
2720     @property(Label)
2721     frontLabel: Label = null;
2722     public cardData: CardData = null;
2723     private isFlipping: boolean = false;
2724     private isShowingFront: boolean = false;
2725     private flipDuration: number = 0.375;
2726     protected onLoad(): void {

```

```

2727         this.node.on(Node.EventType.TOUCH_END, this.onCardClick, this);
2728     }
2729     protected onDestroy(): void {
2730         if (this.node && this.node.isValid) {
2731             this.node.off(Node.EventType.TOUCH_END, this.onCardClick, this);
2732         }
2733     }
2734     setData(data: CardData): void {
2735         this.cardData = data;
2736         this.updateDisplay();
2737     }
2738     updateDisplay(): void {
2739         if (!this.cardData) {
2740             return;
2741         }
2742         if (this.frontLabel) {
2743             if (this.cardData.number === 0) {
2744                 this.frontLabel.string = "💣";
2745             } else {
2746                 this.frontLabel.string = this.cardData.number.toString();
2747             }
2748         }
2749         if (this.cardData.state === CARD_STATE_ENUM.DISPLAY || this.cardData.state ===
2750 CARD_STATE_ENUM.CLICKED) {
2751             this.showCard();
2752         } else {
2753             this.hideCard();
2754         }
2755     }
2756     showCard(): void {
2757         this.isShowingFront = true;
2758         if (this.frontNode) {
2759             this.frontNode.active = true;
2760         }
2761         if (this.backNode) {
2762             this.backNode.active = false;
2763         }
2764         if (this.cardData) {
2765             this.cardData.state = CARD_STATE_ENUM.DISPLAY;
2766         }
2767     }
2768     hideCard(): void {
2769         this.isShowingFront = false;
2770         if (this.frontNode) {
2771             this.frontNode.active = false;
2772         }
2773         if (this.backNode) {
2774             this.backNode.active = true;
2775         }
2776     }
2777     onCardClick(): void {
2778         if (!this.cardData || this.isFlipping || this.cardData.isClicked) {
2779             return;
2780         }
2781         if (this.isShowingFront) {
2782             return;
2783         }
2784         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {

```



```

2785         return;
2786     }
2787     const cardId = this.cardData.id;
2788     if (DataManager.instance.activeSkillCardInterceptMode ===
2789 ACTIVE_SKILL_CARD_INTERCEPT_MODE.VERIFY_CARD_301) {
2790         DataManager.instance.activeSkillCardInterceptMode =
2791 ACTIVE_SKILL_CARD_INTERCEPT_MODE.NONE;
2792         ACTIVE_SKILL_VERIFY_CARD_TAP.emit(EVENT_ENUM.ACTIVE_SKILL_VERIFY_CARD_TAP, cardId);
2793         return;
2794     }
2795     if (DataManager.instance.activeSkillCardInterceptMode ===
2796 ACTIVE_SKILL_CARD_INTERCEPT_MODE.PROPHET_103) {
2797         DataManager.instance.activeSkillCardInterceptMode =
2798 ACTIVE_SKILL_CARD_INTERCEPT_MODE.NONE;
2799         ACTIVE_SKILL_PROPHET_CARD_TAP.emit(EVENT_ENUM.ACTIVE_SKILL_PROPHET_CARD_TAP,
2800 cardId);
2801         return;
2802     }
2803     if (DataManager.instance.isProcessingCard) {
2804         return;
2805     }
2806     const cardClickEvent = CARD_CLICK;
2807     const eventType = EVENT_ENUM.CARD_CLICK;
2808     cardClickEvent.emit(eventType, cardId);
2809     if (DataManager.instance.suppressNextCardFlipAfterClick) {
2810         DataManager.instance.suppressNextCardFlipAfterClick = false;
2811         return;
2812     }
2813     this.flipToFront();
2814 }
2815 setFlipDuration(duration: number): void {
2816     this.flipDuration = duration;
2817 }
2818 getFlipDuration(): number {
2819     return this.flipDuration;
2820 }
2821 interruptFlipTween(): void {
2822     Tween.stopAllByTarget(this.node);
2823     this.isFlipping = false;
2824 }
2825 isCurrentlyShowingFront(): boolean {
2826     return this.isShowingFront;
2827 }
2828 flipToFront(callback?: () => void): void {
2829     if (this.isFlipping) {
2830         this.scheduleOnce(() => this.flipToFront(callback), 0.12);
2831         return;
2832     }
2833     if (this.isShowingFront) {
2834         callback?.();
2835         return;
2836     }
2837     this.isFlipping = true;
2838     const originalScale = this.node.scale.clone();
2839     tween(this.node)
2840         .to(this.flipDuration, {
2841             scale: new Vec3(0.05, originalScale.y, originalScale.z)
2842             }, { easing: 'linear' })

```

```

2843         .call(() => {
2844             this.showCard();
2845         })
2846         .to(this.flipDuration, {
2847             scale: originalScale
2848         }, { easing: 'linear', onComplete: () => {
2849             this.isFlipping = false;
2850             if (callback) {
2851                 callback();
2852             }
2853         }}})
2854         .start();
2855     }
2856     flipToBack(callback?: () => void, halfDurationOverride?: number): void {
2857         if (this.isFlipping) {
2858             return;
2859         }
2860         const half =
2861             typeof halfDurationOverride === 'number' && Number.isFinite(halfDurationOverride) &&
2862             halfDurationOverride > 0
2863             ? halfDurationOverride
2864             : this.flipDuration;
2865         this.isFlipping = true;
2866         const originalScale = this.node.scale.clone();
2867         tween(this.node)
2868             .to(half, {
2869                 scale: new Vec3(0.05, originalScale.y, originalScale.z)
2870             }, { easing: 'linear' })
2871             .call(() => {
2872                 this.hideCard();
2873             })
2874             .to(half, {
2875                 scale: originalScale
2876             }, { easing: 'linear', onComplete: () => {
2877                 this.isFlipping = false;
2878                 if (callback) {
2879                     callback();
2880                 }
2881             }}})
2882             .start();
2883     }
2884 }
2885 ---
2886 ---
2887 ## 文件: `Manager/DataManager.ts`
2888 ```typescript
2889 import { sys } from "cc";
2890 import { LevelConfig, PlayerData, CardData, ChunQiuChanSwapSlotRecord } from "../Config/GameTypes";
2891 import { getLevelConfig } from "../Config/Level";
2892 import { MOCK_PLAYER_DATA, USE_DEBUG_MOCK_SAVE } from "../Config/DebugMockSave";
2893 import { GAME_STATE_ENUM } from "../Utils/Enum";
2894 const PLAYER_DATA_STORAGE_KEY = "MemoMaster_PlayerData";
2895 export const AD_RETRY_MAX_CHARGES = 3;
2896 function getStorageItem(key: string): string | null {
2897     let fromSys: string | null = null;
2898     let fromGlobal: string | null = null;
2899     try {
2900         const a = sys.localStorage.getItem(key);

```

```

2901         if (a != null && a !== "") {
2902             fromSys = a;
2903         }
2904     } catch {
2905     }
2906     try {
2907         const gl = (globalThis as { localStorage?: Storage }).localStorage;
2908         if (gl && typeof gl.getItem === "function") {
2909             const b = gl.getItem(key);
2910             if (b != null && b !== "") {
2911                 fromGlobal = b;
2912             }
2913         }
2914     } catch {
2915     }
2916     if (fromSys && fromGlobal && fromSys !== fromGlobal) {
2917         console.warn("[MemoMaster] sys 与 window.localStorage 中同一 key 内容不一致，本次使用
2918 window 中的值");
2919         return fromGlobal;
2920     }
2921     return fromGlobal ?? fromSys;
2922 }
2923 function setStorageItem(key: string, value: string): void {
2924     try {
2925         sys.localStorage.setItem(key, value);
2926     } catch {
2927     }
2928     try {
2929         const gl = (globalThis as { localStorage?: Storage }).localStorage;
2930         if (gl && typeof gl.setItem === "function") {
2931             gl.setItem(key, value);
2932         }
2933     } catch {
2934     }
2935 }
2936 const DEFAULT_PLAYER_SAVE: PlayerData = {
2937     silverCoin: 0,
2938     goldCoin: 0,
2939     ownedSkins: [],
2940     equippedSkinId: 0,
2941     lastdaily: "",
2942 };
2943 function cloneDefaultPlayerSave(): PlayerData {
2944     return {
2945         ...DEFAULT_PLAYER_SAVE,
2946         ownedSkins: [...DEFAULT_PLAYER_SAVE.ownedSkins],
2947     };
2948 }
2949 function mergePlayerSaveFromStore(raw: Record<string, unknown>): PlayerData {
2950     const d = DEFAULT_PLAYER_SAVE;
2951     const silverCoin =
2952         typeof raw.silverCoin === 'number' && Number.isFinite(raw.silverCoin) ? raw.silverCoin : d.silverCoin;
2953     const goldCoin =
2954         typeof raw.goldCoin === 'number' && Number.isFinite(raw.goldCoin) ? raw.goldCoin : d.goldCoin;
2955     const ownedSkins =
2956         Array.isArray(raw.ownedSkins) && raw.ownedSkins.length > 0
2957             ? [...(raw.ownedSkins as number[])]
2958             : [...d.ownedSkins];

```

```
2959     const equippedSkinId =
2960         typeof raw.equippedSkinId === 'number' && Number.isFinite(raw.equippedSkinId)
2961             ? raw.equippedSkinId
2962             : d.equippedSkinId;
2963     const lastdaily = typeof raw.lastdaily === 'string' ? raw.lastdaily : d.lastdaily;
2964     return { silverCoin, goldCoin, ownedSkins, equippedSkinId, lastdaily };
2965 }
2966 export class DataManager {
2967     private static _instance: DataManager = null;
2968     private static _debugMockSeededThisRun = false;
2969     static get instance(): DataManager {
2970         return this.getInstance<DataManager>();
2971     }
2972     static getInstance<T>(): T {
2973         if (this._instance === null) {
2974             this._instance = new this();
2975         }
2976         return this._instance as T;
2977     }
2978     currentLevel: LevelConfig = null; // 当前关卡配置
2979     gameStatus: GAME_STATE_ENUM = GAME_STATE_ENUM.INIT; // 游戏状态
2980     cards: CardData[] = []; // 当前关卡的所有卡片
2981     clickedCardIds: number[] = []; // 已点击的卡片 ID 列表（按顺序）
2982     isProcessingCard: boolean = false; // 是否在处理卡牌（处理中不能点击，处理完可以点击）
2983     swapKindAfterClick: number[] = [];
2984     isGameContinuing: boolean = true; // 当前游戏是否继续（点击错误后设为 false，技能可以设
2985     为 true）
2986     activeSkillConsumedThisRun: boolean = false;
2987     skill101ConsumedThisLevelCycle: boolean = false;
2988     activeSkillCardInterceptMode: number = 0;
2989     skill203NextClickEnabled: boolean = false;
2990     suppressNextCardFlipAfterClick: boolean = false;
2991     chunQiuChanSwapRecordBySlot: (ChunQiuChanSwapSlotRecord | null)[] = [];
2992     chunQiuChanReplayingAfterSkill: boolean = false;
2993     skill304FengHuangGuConsumedThisRun: boolean = false;
2994     skill204YingShiErDuConsumedThisRun: boolean = false;
2995     private _playerData: PlayerData = cloneDefaultPlayerSave();
2996     private _level: number = 1;
2997     private _adRetryRemaining: number = AD_RETRY_MAX_CHARGES;
2998     private _adRetryChargesLevelId: number | null = null;
2999     private _pendingKeepAdRetryCharges = false;
3000     private _pendingFailAdRetryRestart = false;
3001     private _pendingPreserveSkill101Consumption = false;
3002     private _pendingSkill101RefreshFromFailSettlement = false;
3003     private _currentRunFromFailAdRetry = false;
3004     public get level(): number {
3005         return this._level;
3006     }
3007     public set level(data: number) {
3008         this._level = data;
3009     }
3010     getResolvedLevelConfig(): LevelConfig | null {
3011         return this.currentLevel ?? getLevelConfig(this.level);
3012     }
3013     hasLevelConfig(levelId: number): boolean {
3014         return getLevelConfig(levelId) !== null;
3015     }
3016     private static readonly SKIN_PASSIVE_304_FENG_HUANG = 304;
```

```
3017     private static readonly SKIN_PASSIVE_204_YING_SHI = 204;
3018     private static readonly SKIN_PASSIVE_303_WAN_GU = 303;
3019     private static readonly WAN_GU_DISPLAY_EXTRA_SEC = 2;
3020     private static readonly SKIN_PASSIVE_207_SUI_YUE_SHEN_TOU = 207;
3021     private static readonly SUI_YUE_SHEN_TOU_DISPLAY_EXTRA_SEC = 1;
3022     getEffectiveDisplayTimeSec(): number {
3023         const base = this.currentLevel?.displayTime ?? 0;
3024         if (this.equippedSkinId === DataManager.SKIN_PASSIVE_303_WAN_GU) {
3025             return base + DataManager.WAN_GU_DISPLAY_EXTRA_SEC;
3026         }
3027         if (this.equippedSkinId === DataManager.SKIN_PASSIVE_207_SUI_YUE_SHEN_TOU) {
3028             return base + DataManager.SUI_YUE_SHEN_TOU_DISPLAY_EXTRA_SEC;
3029         }
3030         return base;
3031     }
3032     isWanGuCongCongEquipped(): boolean {
3033         return this.equippedSkinId === DataManager.SKIN_PASSIVE_303_WAN_GU;
3034     }
3035     isSuiYueShenTou207Equipped(): boolean {
3036         return this.equippedSkinId === DataManager.SKIN_PASSIVE_207_SUI_YUE_SHEN_TOU;
3037     }
3038     isFengHuangGu304Available(): boolean {
3039         return (
3040             this.equippedSkinId === DataManager.SKIN_PASSIVE_304_FENG_HUANG &&
3041             !this.skill304FengHuangGuConsumedThisRun
3042         );
3043     }
3044     isYingShiErDu204Available(): boolean {
3045         return (
3046             this.equippedSkinId === DataManager.SKIN_PASSIVE_204_YING_SHI &&
3047             !this.skill204YingShiErDuConsumedThisRun
3048         );
3049     }
3050     public get silverCoin(): number {
3051         return this._playerData.silverCoin;
3052     }
3053     public set silverCoin(value: number) {
3054         this._playerData.silverCoin = value;
3055         this.save();
3056     }
3057     public get goldCoin(): number {
3058         return this._playerData.goldCoin;
3059     }
3060     public set goldCoin(value: number) {
3061         this._playerData.goldCoin = value;
3062         this.save();
3063     }
3064     public get equippedSkinId(): number {
3065         return this._playerData.equippedSkinId;
3066     }
3067     public set equippedSkinId(value: number) {
3068         this._playerData.equippedSkinId = value;
3069         this.save();
3070     }
3071     public get lastdaily(): string {
3072         return this._playerData.lastdaily ?? "";
3073     }
3074     public set lastdaily(value: string) {
```

```
3075         this._playerData.lastdaily = value;
3076         this.save();
3077     }
3078     public claimDailyMenuReward(todayStr: string): void {
3079         this._playerData.goldCoin += 10;
3080         this._playerData.silverCoin += 200;
3081         this._playerData.lastdaily = todayStr;
3082         this.save();
3083     }
3084     public get ownedSkins(): number[] {
3085         return this._playerData.ownedSkins;
3086     }
3087     public get adRetryRemaining(): number {
3088         return this._adRetryRemaining;
3089     }
3090     public requestKeepAdRetryChargesOnNextStartLevel(): void {
3091         this._pendingKeepAdRetryCharges = true;
3092     }
3093     public applyAdRetryChargesBeforeStartLevel(levelId: number): void {
3094         const keep = this._pendingKeepAdRetryCharges;
3095         this._pendingKeepAdRetryCharges = false;
3096         if (!keep) {
3097             this._adRetryChargesLevelId = levelId;
3098             this._adRetryRemaining = AD_RETRY_MAX_CHARGES;
3099             return;
3100         }
3101         if (this._adRetryChargesLevelId !== levelId) {
3102             this._adRetryChargesLevelId = levelId;
3103             this._adRetryRemaining = AD_RETRY_MAX_CHARGES;
3104         }
3105     }
3106     public consumeAdRetryCharge(): void {
3107         if (this._adRetryRemaining > 0) {
3108             this._adRetryRemaining -= 1;
3109         }
3110     }
3111     public get currentRunFromFailAdRetry(): boolean {
3112         return this._currentRunFromFailAdRetry;
3113     }
3114     public requestNextStartFromFailAdRetry(): void {
3115         this._pendingFailAdRetryRestart = true;
3116     }
3117     public requestPreserveSkill101ConsumptionOnNextStartLevel(): void {
3118         this._pendingPreserveSkill101Consumption = true;
3119     }
3120     public requestSkill101RefreshFromFailSettlementOnNextStartLevel(): void {
3121         this._pendingSkill101RefreshFromFailSettlement = true;
3122     }
3123     public applyFailAdRetryRunFlagAfterReset(): void {
3124         this._currentRunFromFailAdRetry = this._pendingFailAdRetryRestart;
3125         this._pendingFailAdRetryRestart = false;
3126     }
3127     public get playerData(): PlayerData {
3128         return this._playerData;
3129     }
3130     init(): void {
3131         if (USE_DEBUG MOCK_SAVE && !DataManager._debugMockSeededThisRun) {
3132             try {
```

```
3133         setStorageItem(PLAYER_DATA_STORAGE_KEY, JSON.stringify(MOCK_PLAYER_DATA));
3134         DataManager._debugMockSeededThisRun = true;
3135     } catch (e) {
3136         console.error("调试写入本地存档失败:", e);
3137     }
3138 }
3139 this.restore();
3140 const raw = getStorageItem(PLAYER_DATA_STORAGE_KEY);
3141 console.log(
3142     "[MemoMaster] 存档读取后    goldCoin=%s    silverCoin=%s    equippedSkinId=%s    |
3143     USE_DEBUG MOCK_SAVE=%s hasRawSave=%s",
3144     this.goldCoin,
3145     this.silverCoin,
3146     this.equippedSkinId,
3147     USE_DEBUG MOCK_SAVE,
3148     !!raw
3149 );
3150 }
3151 resetGameData(): void {
3152     if (this._pendingSkill101RefreshFromFailSettlement) {
3153         this.skill101ConsumedThisLevelCycle = false;
3154         this._pendingSkill101RefreshFromFailSettlement = false;
3155         this._pendingPreserveSkill101Consumption = false;
3156     } else if (this._pendingPreserveSkill101Consumption) {
3157         this._pendingPreserveSkill101Consumption = false;
3158     } else {
3159         this.skill101ConsumedThisLevelCycle = false;
3160     }
3161     this.cards = [];
3162     this.clickedCardIds = [];
3163     this.currentLevel = null;
3164     this.gameStatus = GAME_STATE_ENUM.INIT;
3165     this._currentRunFromFailAdRetry = false;
3166     this.isProcessingCard = false;
3167     this.swapKindAfterClick = [];
3168     this.isGameContinuing = true;
3169     this.activeSkillConsumedThisRun = false;
3170     this.activeSkillCardInterceptMode = 0;
3171     this.skill203NextClickEnabled = false;
3172     this.suppressNextCardFlipAfterClick = false;
3173     this.chunQiuChanSwapRecordBySlot = [];
3174     this.chunQiuChanReplayingAfterSkill = false;
3175     this.skill304FengHuangGuConsumedThisRun = false;
3176     this.skill204YingShiErDuConsumedThisRun = false;
3177 }
3178 initChunQiuChanSwapSlots(swapSlotCount: number): void {
3179     const n = Math.max(0, Math.floor(swapSlotCount));
3180     this.chunQiuChanSwapRecordBySlot = new Array(n).fill(null);
3181     this.chunQiuChanReplayingAfterSkill = false;
3182 }
3183 reset(): void {
3184     this._playerData = cloneDefaultPlayerSave();
3185     this._level = 1;
3186     this._pendingFailAdRetryRestart = false;
3187     this._pendingPreserveSkill101Consumption = false;
3188     this._pendingSkill101RefreshFromFailSettlement = false;
3189     this._pendingKeepAdRetryCharges = false;
3190     this._adRetryChargesLevelId = null;
```

```
3191         this._adRetryRemaining = AD_RETRY_MAX_CHARGES;
3192         this.resetGameData();
3193         this.save();
3194     }
3195     save(): void {
3196         try {
3197             const payload: PlayerData = {
3198                 silverCoin: this._playerData.silverCoin,
3199                 goldCoin: this._playerData.goldCoin,
3200                 ownedSkins: [...this._playerData.ownedSkins],
3201                 equippedSkinId: this._playerData.equippedSkinId,
3202                 lastdaily: this._playerData.lastdaily ?? "",
3203             };
3204             setStorageItem(PLAYER_DATA_STORAGE_KEY, JSON.stringify(payload));
3205         } catch (error) {
3206             console.error('保存数据失败:', error);
3207         }
3208     }
3209     restore(): void {
3210         try {
3211             const dataStr = getStorageItem(PLAYER_DATA_STORAGE_KEY);
3212             if (!dataStr) {
3213                 this._playerData = cloneDefaultPlayerSave();
3214                 this._level = 1;
3215                 this.save();
3216                 return;
3217             }
3218             console.log("[MemoMaster] 存档原始 JSON:", dataStr.length > 800 ? dataStr.slice(0, 800) + "..." :
3219 dataStr);
3220             const parsed = JSON.parse(dataStr) as Record<string, unknown>;
3221             this._playerData = mergePlayerSaveFromStore(parsed);
3222         } catch (error) {
3223             console.error("读取数据失败:", error);
3224             this.reset();
3225         }
3226     }
3227     addCard(card: CardData): void {
3228         this.cards.push(card);
3229     }
3230     getCard(cardId: number): CardData | null {
3231         return this.cards.find(card => card.id === cardId) || null;
3232     }
3233     clearCards(): void {
3234         this.cards = [];
3235         this.clickedCardIds = [];
3236     }
3237     recordCardClick(cardId: number): void {
3238         this.clickedCardIds.push(cardId);
3239     }
3240     addSkin(skinId: number): void {
3241         if (!this._playerData.ownedSkins.includes(skinId)) {
3242             this._playerData.ownedSkins.push(skinId);
3243             this.save();
3244         }
3245     }
3246     hasSkin(skinId: number): boolean {
3247         return this._playerData.ownedSkins.includes(skinId);
3248     }
```



```
3249     equipSkin(skinId: number): boolean {
3250         if (this.hasSkin(skinId)) {
3251             this._playerData.equippedSkinId = skinId;
3252             this.save();
3253             return true;
3254         }
3255         return false;
3256     }
3257     addSilverCoin(amount: number): void {
3258         this.silverCoin = this.silverCoin + amount;
3259     }
3260     consumeSilverCoin(amount: number): boolean {
3261         if (this.silverCoin >= amount) {
3262             this.silverCoin = this.silverCoin - amount;
3263             return true;
3264         }
3265         return false;
3266     }
3267     addGoldCoin(amount: number): void {
3268         this.goldCoin = this.goldCoin + amount;
3269     }
3270     consumeGoldCoin(amount: number): boolean {
3271         if (this.goldCoin >= amount) {
3272             this.goldCoin = this.goldCoin - amount;
3273             return true;
3274         }
3275         return false;
3276     }
3277 }
3278 ---
3279 ---
3280 ## 文件: `Manager/GameManager.ts`
3281 ```typescript
3282 import { _decorator, CCInteger, Color, Component, Node, Prefab, instantiate, UITransform, Vec3, tween, Tween,
3283 SpriteFrame } from 'cc';
3284 import { DataManager } from '../DataManager';
3285 import { getLevelConfig } from '../Config/Level';
3286 import { GAME_STATE_ENUM, EVENT_ENUM, CARD_STATE_ENUM, AUDIO_EFFECT_ENUM } from '../Utils/Enum';
3287 import {
3288     GAME_STATE_CHANGE,
3289     GAME_START,
3290     CARD_CLICK,
3291     LEVEL_START,
3292     LEVEL_COMPLETE,
3293     LEVEL_FAILED,
3294     PLAY_AUDIO,
3295     UI_UPDATE,
3296     PASSIVE_304_FENG_HUANG_GU_TRIGGER,
3297     PASSIVE_204_YING_SHI_ER_DU_TRIGGER,
3298 } from '../Utils/Event';
3299 import { CardSkinLibrary } from '../Config/CardSkinLibrary';
3300 import {
3301     CardData,
3302     LevelConfig,
3303     SWAP_KIND_NONE,
3304     SWAP_KIND_PAIR,
3305     SWAP_KIND_HOLE,
3306     ChunQiuChanSwapSlotRecord,
```

```
3307 } from '../Config/GameTypes';
3308 import {
3309     generateUniqueRandomNumbers,
3310     shuffle,
3311     calculateGridSize,
3312     delay,
3313     applySpriteFrameToUINode,
3314     clearSpriteFrameOnUINode,
3315     randomInt,
3316 } from '../Utils/Utils';
3317 import { Card } from '../GameObject/Card';
3318 const { ccclass, property } = _decorator;
3319 @ccclass('GameManager')
3320 export class GameManager extends Component {
3321     @property(Node)
3322     gameContainerNode: Node = null;
3323     @property(Prefab)
3324     cardPrefab: Prefab = null;
3325     @property(CardSkinLibrary)
3326     cardSkinLibrary: CardSkinLibrary = null;
3327     @property([SpriteFrame])
3328     cardFrontSkinFrames: SpriteFrame[] = [];
3329     @property([SpriteFrame])
3330     cardBackSkinFrames: SpriteFrame[] = [];
3331     @property([CCInteger])
3332     skillSpriteIds: number[] = [];
3333     @property([SpriteFrame])
3334     skillSpriteFrames: SpriteFrame[] = [];
3335     private cardNodes: Node[] = [];
3336     private safeCardNodes(): Node[] {
3337         if (this.cardNodes == null || !Array.isArray(this.cardNodes)) {
3338             this.cardNodes = [];
3339         }
3340         return this.cardNodes;
3341     }
3342     private chunQiuChanSlots(): (ChunQiuChanSwapSlotRecord | null)[] {
3343         const dm = DataManager.instance;
3344         if (dm.chunQiuChanSwapRecordBySlot == null || !Array.isArray(dm.chunQiuChanSwapRecordBySlot)) {
3345             dm.chunQiuChanSwapRecordBySlot = [];
3346         }
3347         return dm.chunQiuChanSwapRecordBySlot;
3348     }
3349     private currentDisplayTimer: number = 0;
3350     private startLevelRunId = 0;
3351     private chunQiuChanSkillToken = 0;
3352     private displayPhaseUseFlipToFrontTransition = false;
3353     private _pendingChunQiuChanBackLogoAfterDisplay = false;
3354     private currentClickIndex: number = 0; // 当前应该点击的卡片索引（从小到大）
3355     private expectedClickOrder: number[] = []; // 期望的点击顺序数组（从小到大排序）
3356     private _skill304PendingCardId: number | null = null;
3357     private static readonly SKILL_304_PAUSE_BEFORE_FLIP_BACK_SEC = 0.5;
3358     private static readonly SKILL_304_AFTER_TRIGGER_BACK_LOGO_ID = 3041;
3359     private readonly deferredSkill304AfterReveal = (): void => {
3360         const id = this._skill304PendingCardId;
3361         if (id == null) {
3362             return;
3363         }
3364         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
```

```
3365         this._skill304PendingCardId = null;
3366         DataManager.instance.isProcessingCard = false;
3367         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3368         return;
3369     }
3370
3371     PASSIVE_304_FENG_HUANG_GU_TRIGGER.emit(EVENT_ENUM.PASSIVE_304_FENG_HUANG_GU_TRIGGER);
3372     this.replaceAllCardsBackSkillLogo(GameManager.SKILL_304_AFTER_TRIGGER_BACK_LOGO_ID);
3373     this.unschedule(this.deferredSkill304FlipBack);
3374     this.scheduleOnce(this.deferredSkill304FlipBack,
3375     GameManager.SKILL_304_PAUSE_BEFORE_FLIP_BACK_SEC);
3376     };
3377     private readonly deferredSkill304FlipBack = (): void => {
3378         const id = this._skill304PendingCardId;
3379         this._skill304PendingCardId = null;
3380         if (id == null) {
3381             return;
3382         }
3383         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
3384             DataManager.instance.isProcessingCard = false;
3385             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3386             return;
3387         }
3388         const comp = this.getCardComponentByCardId(id);
3389         if (comp?.isValid) {
3390             comp.interruptFlipTween();
3391             comp.flipToBack(() => {
3392                 if (DataManager.instance.gameStatus === GAME_STATE_ENUM.CLICKING) {
3393                     DataManager.instance.isProcessingCard = false;
3394                     UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3395                 }
3396             });
3397         } else {
3398             DataManager.instance.isProcessingCard = false;
3399             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3400         }
3401     };
3402     private static readonly SKILL_204_PAUSE_BEFORE_MASS_FLIP_BACK_SEC = 0.5;
3403     private static readonly SKILL_204_AFTER_TRIGGER_BACK_LOGO_ID = 2041;
3404     private readonly deferredSkill204AfterReveal = (): void => {
3405         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
3406             DataManager.instance.isProcessingCard = false;
3407             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3408             return;
3409         }
3410
3411         PASSIVE_204_YING_SHI_ER_DU_TRIGGER.emit(EVENT_ENUM.PASSIVE_204_YING_SHI_ER_DU_TRIGGER);
3412         this.replaceAllCardsBackSkillLogo(GameManager.SKILL_204_AFTER_TRIGGER_BACK_LOGO_ID);
3413         this.unschedule(this.deferredSkill204MassFlipBack);
3414         this.scheduleOnce(this.deferredSkill204MassFlipBack,
3415         GameManager.SKILL_204_PAUSE_BEFORE_MASS_FLIP_BACK_SEC);
3416         };
3417         private readonly deferredSkill204MassFlipBack = (): void => {
3418             if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
3419                 DataManager.instance.isProcessingCard = false;
3420                 UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3421                 return;
3422             }
```

```

3423         const promises: Promise<void>[] = [];
3424         for (const n of this.safeCardNodes()) {
3425             const comp = n.getComponent(Card);
3426             if (!comp?.cardData || !comp.isCurrentlyShowingFront()) {
3427                 continue;
3428             }
3429             comp.interruptFlipTween();
3430             promises.push(
3431                 new Promise<void>((resolve) => {
3432                     comp.flipToBack(() => resolve());
3433                 })
3434             );
3435         }
3436         if (promises.length === 0) {
3437             this.finishSkill204StateReset();
3438             return;
3439         }
3440         void Promise.all(promises).then(() => this.finishSkill204StateReset());
3441     };
3442     private _skill203Pending: { correctId: number; suppressRandomSwap: boolean } | null = null;
3443     private readonly deferredSkill203AfterSwap = (): void => {
3444         const p = this._skill203Pending;
3445         this._skill203Pending = null;
3446         if (!p) {
3447             return;
3448         }
3449         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING
3450         || !DataManager.instance.isGameContinuing) {
3451             DataManager.instance.isProcessingCard = false;
3452             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3453             return;
3454         }
3455         this.applySkill203CorrectReveal(p.correctId, p.suppressRandomSwap);
3456     };
3457     private static readonly FAIL_UNCLICKED_NUMBER_COLOR = new Color(255, 0, 0, 255);
3458     private static readonly CARD_FLIP_TOTAL_SEC = 0.75;
3459     private static readonly CARD_SWAP_DURATION_SEC = 1.0;
3460     private static readonly CARD_FLIP_HALF_DURATION_SEC = 0.375;
3461     private static readonly SKIN_PASSIVE_205_YI_HUA_JIE_MU = 205;
3462     private static readonly SKIN_PASSIVE_206_BU_DONG_MING_WANG = 206;
3463     private static readonly PASSIVE_206_SWAP_COUNT_MULT = 0.5;
3464     private static readonly SKIN_PASSIVE_107_MU_TOU_REN = 107;
3465     private static readonly PASSIVE_107_SWAP_COUNT_MULT = 0.8;
3466     private static readonly SKIN_PASSIVE_105_RANG_NI_SAN_ZHAO = 105;
3467     private static readonly SKIN_PASSIVE_104_QI_BIAN = 104;
3468     private static readonly SKILL_104_ODD_MEMORY_LOGO_ID = 1041;
3469     private static readonly SKIN_PASSIVE_106_LUAN_JIN_GUO = 106;
3470     private static readonly POST_DISPLAY_MASS_FLIP_BACK_TOTAL_SEC_106 = 2.5;
3471     private static readonly WIN_DELAY_BEFORE_SETTLEMENT_SEC = 1.5;
3472     private static readonly FAIL_PAUSE_AFTER_MASS_REVEAL_SEC = 1.5;
3473     protected onLoad(): void {
3474         CARD_CLICK.on(EVENT_ENUM.CARD_CLICK, this.onCardClick, this);
3475         LEVEL_START.on(EVENT_ENUM.LEVEL_START, this.onLevelStart, this);
3476         this.init();
3477     }
3478     protected onDestroy(): void {
3479         this.cancelEndGameSequences();
3480         CARD_CLICK.off(EVENT_ENUM.CARD_CLICK, this.onCardClick, this);

```

```
3481         LEVEL_START.off(EVENT_ENUM.LEVEL_START, this.onLevelStart, this);
3482     }
3483     init(): void {
3484         DataManager.instance.init();
3485     }
3486     private effectiveFrontSkinFrames(): SpriteFrame[] {
3487         return this.cardSkinLibrary ? this.cardSkinLibrary.cardFrontSkinFrames : this.cardFrontSkinFrames;
3488     }
3489     private effectiveBackSkinFrames(): SpriteFrame[] {
3490         return this.cardSkinLibrary ? this.cardSkinLibrary.cardBackSkinFrames : this.cardBackSkinFrames;
3491     }
3492     private effectiveSkillSpriteIds(): number[] {
3493         return this.cardSkinLibrary ? this.cardSkinLibrary.skillSpriteIds : this.skillSpriteIds;
3494     }
3495     private effectiveSkillSpriteFrames(): SpriteFrame[] {
3496         return this.cardSkinLibrary ? this.cardSkinLibrary.skillSpriteFrames : this.skillSpriteFrames;
3497     }
3498     onLevelStart(levelId: number): void {
3499         this.startLevel(levelId);
3500     }
3501     onCardClick(cardId: number): void {
3502         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
3503             return;
3504         }
3505         DataManager.instance.isProcessingCard = true;
3506         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3507         this.handleCardClick(cardId);
3508     }
3509     async startLevel(levelId: number): Promise<void> {
3510         const runId = ++this.startLevelRunId;
3511         this.chunQiuChanSkillToken++;
3512         this.displayPhaseUseFlipToFrontTransition = false;
3513         this.cancelEndGameSequences();
3514         if (!this.gameContainerNode || !this.cardPrefab) {
3515             console.error('GameManager: gameContainerNode or cardPrefab is not set');
3516             return;
3517         }
3518         const levelConfig = getLevelConfig(levelId);
3519         if (!levelConfig) {
3520             console.error('GameManager: levelConfig is not found');
3521             return;
3522         }
3523         DataManager.instance.applyAdRetryChargesBeforeStartLevel(levelId);
3524         DataManager.instance.resetGameData();
3525         DataManager.instance.applyFailAdRetryRunFlagAfterReset();
3526         DataManager.instance.currentLevel = levelConfig;
3527         DataManager.instance.level = levelId;
3528         DataManager.instance.swapKindAfterClick = this.buildSwapKindAfterClickArray(levelConfig);
3529         DataManager.instance.initChunQiuChanSwapSlots(DataManager.instance.swapKindAfterClick.length);
3530         this.currentClickIndex = 0;
3531         this.expectedClickOrder = [];
3532         this.generateCards(levelConfig);
3533         this.changeGameState(GAME_STATE_ENUM.PREPARE);
3534         await delay(500);
3535         if (runId !== this.startLevelRunId) {
3536             return;
3537         }
3538         await this.displayPhase();
```

```
3539         if (runId !== this.startLevelRunId) {
3540             return;
3541         }
3542         await this.flipPhase();
3543         if (runId !== this.startLevelRunId) {
3544             return;
3545         }
3546         DataManager.instance.isProcessingCard = false;
3547         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3548         this.changeGameState(GAME_STATE_ENUM.CLICKING);
3549     }
3550     generateCards(levelConfig: any): void {
3551         this.clearCards();
3552         const numbers = generateUniqueRandomNumbers(levelConfig.realCard, 1, levelConfig.realCard);
3553         const shuffledNumbers = shuffle(numbers);
3554         this.expectedClickOrder = [...numbers].sort((a, b) => a - b);
3555         const gridSize = calculateGridSize(levelConfig.totalCard);
3556         const cardSpacingX = 174; // 横向间距
3557         const cardSpacingY = 240; // 纵向间距
3558         const startX = -(gridSize.cols - 1) * cardSpacingX / 2;
3559         const startY = (gridSize.rows - 1) * cardSpacingY / 2;
3560         const totalPositions = gridSize.rows * gridSize.cols;
3561         const positionIndices = shuffle(Array.from({ length: totalPositions }, (_, i) => i));
3562         let numberIndex = 0;
3563         for (let i = 0; i < levelConfig.realCard; i++) {
3564             const posIndex = positionIndices[i];
3565             const row = Math.floor(posIndex / gridSize.cols);
3566             const col = posIndex % gridSize.cols;
3567             const cardData: CardData = {
3568                 id: i + 1,
3569                 number: shuffledNumbers[numberIndex++],
3570                 x: col,
3571                 y: row,
3572                 state: CARD_STATE_ENUM.DISPLAY,
3573                 isClicked: false,
3574             };
3575             DataManager.instance.addCard(cardData);
3576             this.createCardNode(cardData, startX + col * cardSpacingX, startY - row * cardSpacingY);
3577         }
3578         const bombCount = levelConfig.totalCard - levelConfig.realCard;
3579         if (bombCount > 0) {
3580             for (let i = 0; i < bombCount; i++) {
3581                 const posIndex = positionIndices[levelConfig.realCard + i];
3582                 const row = Math.floor(posIndex / gridSize.cols);
3583                 const col = posIndex % gridSize.cols;
3584                 const cardData: CardData = {
3585                     id: levelConfig.realCard + i + 1,
3586                     number: 0, // 炸弹卡片用 0 表示
3587                     x: col,
3588                     y: row,
3589                     state: CARD_STATE_ENUM.DISPLAY,
3590                     isClicked: false,
3591                 };
3592                 DataManager.instance.addCard(cardData);
3593                 this.createCardNode(cardData, startX + col * cardSpacingX, startY - row * cardSpacingY);
3594             }
3595         }
3596     }
```

```

3597     createCardNode(cardData: CardData, x: number, y: number): void {
3598         if (!this.cardPrefab || !this.gameContainerNode || !this.gameContainerNode.isValid) {
3599             return;
3600         }
3601         const cardNode = instantiate(this.cardPrefab);
3602         if (!cardNode || !cardNode.isValid) {
3603             return;
3604         }
3605         cardNode.setPosition(x, y, 0);
3606         if (this.gameContainerNode && this.gameContainerNode.isValid) {
3607             cardNode.setParent(this.gameContainerNode);
3608         } else {
3609             cardNode.destroy();
3610             return;
3611         }
3612         const cardComponent = cardNode.getComponent(Card);
3613         if (cardComponent) {
3614             cardComponent.setData(cardData);
3615         }
3616         this.applyEquippedSkinToCardNode(cardNode);
3617         this.safeCardNodes().push(cardNode);
3618     }
3619     private resolveSkinTextureIndex(equippedSkinId: number): number {
3620         return Math.floor(equippedSkinId / 100);
3621     }
3622     private applyEquippedSkinToCardNode(cardNode: Node): void {
3623         const equippedSkinId = DataManager.instance.equippedSkinId;
3624         const textureIndex = this.resolveSkinTextureIndex(equippedSkinId);
3625         const fronts = this.effectiveFrontSkinFrames();
3626         const backs = this.effectiveBackSkinFrames();
3627         const frontLen = fronts.length;
3628         const backLen = backs.length;
3629         const maxIdx = Math.max(0, Math.min(frontLen, backLen) - 1);
3630         const clamped = Math.max(0, Math.min(textureIndex, maxIdx));
3631         const frontSf = fronts[clamped] ?? fronts[0];
3632         const backSf = backs[clamped] ?? backs[0];
3633         if (!frontSf || !backSf) {
3634             console.warn(
3635                 `[GameManager] 未绑定 cardFrontSkinFrames/cardBackSkinFrames (或 CardSkinLibrary 为
3636                 空数组), 跳过卡牌皮肤替换 (至少需要配置索引 0)`
3637             );
3638             return;
3639         }
3640         const cardFront = cardNode.getChildByName('CardFront');
3641         const cardBack = cardNode.getChildByName('CardBack');
3642         applySpriteFrameToUINode(cardBack ?? undefined, backSf);
3643         const skillNode = cardBack?.getChildByName('Skill') ?? cardNode.getChildByName('Skill');
3644         if (equippedSkinId >= 100) {
3645             const skillIds = this.effectiveSkillSpriteIds();
3646             const skillFrames = this.effectiveSkillSpriteFrames();
3647             const skillIdx = skillIds.indexOf(equippedSkinId);
3648             const skillSf = skillIdx >= 0 ? skillFrames[skillIdx] : null;
3649             if (!skillSf) {
3650                 console.warn(
3651                     `[GameManager] 未找到 Skill 贴图映射 equippedSkinId=${equippedSkinId}, 请配置
3652                     skillSpriteIds/skillSpriteFrames`
3653                 );
3654                 clearSpriteFrameOnUINode(skillNode ?? undefined);

```

```

3655         } else {
3656             if (skillNode) {
3657                 skillNode.active = true;
3658             }
3659             applySpriteFrameToUINode(skillNode ?? undefined, skillSf);
3660         }
3661     } else {
3662         clearSpriteFrameOnUINode(skillNode ?? undefined);
3663     }
3664     applySpriteFrameToUINode(cardFront ?? undefined, frontSf);
3665 }
3666 private resolveSkillSpriteFrameByLogold(logold: number): SpriteFrame | null {
3667     if (typeof logold !== 'number' || !Number.isFinite(logold)) {
3668         return null;
3669     }
3670     const skillIds = this.effectiveSkillSpriteIds();
3671     const skillFrames = this.effectiveSkillSpriteFrames();
3672     const skillIdx = skillIds.indexOf(logold);
3673     return skillIdx >= 0 ? skillFrames[skillIdx] ?? null : null;
3674 }
3675 private resolveGameCardBackAndFrontSkillNodes(cardNode: Node): {
3676     backSkill: Node | null;
3677     frontSkill: Node | null;
3678 } {
3679     const cardBack = cardNode.getChildByName('CardBack');
3680     const cardFront = cardNode.getChildByName('CardFront');
3681     const backSkill = (cardBack?.getChildByName('Skill') ?? cardNode.getChildByName('Skill')) ?? null;
3682     const frontSkill = cardFront?.getChildByName('Skill') ?? null;
3683     return { backSkill, frontSkill };
3684 }
3685 private applyPassive104QiBianMemoryPhaseOddCardSkillLogos(): void {
3686     if (DataManager.instance.equippedSkinId !== GameManager.SKIN_PASSIVE_104_QI_BIAN) {
3687         return;
3688     }
3689     const sfOdd =
3690     this.resolveSkillSpriteFrameByLogold(GameManager.SKILL_104_ODD_MEMORY_LOGO_ID);
3691     const sfEven = this.resolveSkillSpriteFrameByLogold(GameManager.SKIN_PASSIVE_104_QI_BIAN);
3692     if (!sfOdd) {
3693         console.warn(
3694             `[GameManager] 104 奇 变 : 未 找 到 Skill 业 务 ID
3695             ${GameManager.SKILL_104_ODD_MEMORY_LOGO_ID} 的贴图, 请在 skillSpriteIds / skillSpriteFrames 中配置`
3696         );
3697         return;
3698     }
3699     for (const cardNode of this.safeCardNodes()) {
3700         if (!cardNode?.isValid) {
3701             continue;
3702         }
3703         const cardComp = cardNode.getComponent(Card);
3704         const num = cardComp?.cardData?.number;
3705         const isOddDigit = typeof num === 'number' && num > 0 && (num & 1) === 1;
3706         const target = isOddDigit ? sfOdd : sfEven;
3707         const { backSkill, frontSkill } = this.resolveGameCardBackAndFrontSkillNodes(cardNode);
3708         if (!target) {
3709             this.applyEquippedSkinToCardNode(cardNode);
3710             continue;
3711         }
3712         if (backSkill) {

```



```

3713         backSkill.active = true;
3714     }
3715     applySpriteFrameToUINode(backSkill ?? undefined, target);
3716     if (frontSkill) {
3717         if (isOddDigit) {
3718             frontSkill.active = true;
3719             applySpriteFrameToUINode(frontSkill, sfOdd);
3720         } else if (sfEven) {
3721             frontSkill.active = true;
3722             applySpriteFrameToUINode(frontSkill, sfEven);
3723         } else {
3724             clearSpriteFrameOnUINode(frontSkill);
3725             frontSkill.active = false;
3726         }
3727     }
3728 }
3729 }
3730 replaceAllCardsBackSkillLogo(logold: number): void {
3731     if (typeof logold !== 'number' || !Number.isFinite(logold)) {
3732         return;
3733     }
3734     const skillSf = this.resolveSkillSpriteFrameByLogold(logold);
3735     if (!skillSf) {
3736         console.warn(
3737             `[GameManager] replaceAllCardsBackSkillLogo: 未找到 logold=${logold} 的贴图，请检查
3738 skillSpriteIds / skillSpriteFrames`
3739         );
3740         return;
3741     }
3742     for (const cardNode of this.safeCardNodes()) {
3743         if (!cardNode?.isValid) {
3744             continue;
3745         }
3746         const cardBack = cardNode.getChildByName('CardBack');
3747         const skillNode = cardBack?.getChildByName('Skill') ?? cardNode.getChildByName('Skill');
3748         if (skillNode) {
3749             skillNode.active = true;
3750         }
3751         applySpriteFrameToUINode(skillNode ?? undefined, skillSf);
3752     }
3753 }
3754 clearCards(): void {
3755     this.safeCardNodes().forEach(node => {
3756         if (node && node.isValid) {
3757             node.destroy();
3758         }
3759     });
3760     this.cardNodes = [];
3761     DataManager.instance.clearCards();
3762     this.currentClickIndex = 0;
3763     this.expectedClickOrder = [];
3764 }
3765 async displayPhase(): Promise<void> {
3766     const useFlip = this.displayPhaseUseFlipToFrontTransition;
3767     this.displayPhaseUseFlipToFrontTransition = false;
3768     if (useFlip) {
3769         const flipPromises: Promise<void>[] = [];
3770         this.safeCardNodes().forEach(node => {

```

```
3771         const comp = node.getComponent(Card);
3772         if (!comp || comp.isCurrentlyShowingFront()) {
3773             return;
3774         }
3775         flipPromises.push(
3776             new Promise<void>(resolve => {
3777                 comp.flipToFront(() => resolve());
3778             })
3779         );
3780     });
3781     await Promise.all(flipPromises);
3782     this.changeGameState(GAME_STATE_ENUM.DISPLAY);
3783 } else {
3784     this.changeGameState(GAME_STATE_ENUM.DISPLAY);
3785     this.safeCardNodes().forEach(node => {
3786         const cardComponent = node.getComponent(Card);
3787         if (cardComponent) {
3788             cardComponent.showCard();
3789         }
3790     });
3791 }
3792 if (this._pendingChunQiuChanBackLogoAfterDisplay) {
3793     this._pendingChunQiuChanBackLogoAfterDisplay = false;
3794 }
3795 this.replaceAllCardsBackSkillLogo(GameManager.CHUN_QIU_CHAN_AFTER_SKILL_BACK_LOGO_ID);
3796 }
3797 this.applyPassive104QiBianMemoryPhaseOddCardSkillLogos();
3798 const displayTime = DataManager.instance.getEffectiveDisplayTimeSec();
3799 await delay(displayTime * 1000);
3800 }
3801 private getPostDisplayMassFlipBackHalfDurationSec(): number {
3802     if (DataManager.instance.equippedSkinId === GameManager.SKIN_PASSIVE_106_LUAN_JIN_GUO) {
3803         return GameManager.POST_DISPLAY_MASS_FLIP_BACK_TOTAL_SEC_106 * 0.5;
3804     }
3805     return GameManager.CARD_FLIP_HALF_DURATION_SEC;
3806 }
3807 async flipPhase(): Promise<void> {
3808     const halfSec = this.getPostDisplayMassFlipBackHalfDurationSec();
3809     const flipPromises: Promise<void>[] = [];
3810     this.safeCardNodes().forEach(node => {
3811         const cardComponent = node.getComponent(Card);
3812         if (cardComponent) {
3813             const promise = new Promise<void>((resolve) => {
3814                 cardComponent.flipToBack(() => {
3815                     resolve();
3816                 }, halfSec);
3817             });
3818             flipPromises.push(promise);
3819         }
3820     });
3821     await Promise.all(flipPromises);
3822 }
3823 private getSwapOpportunityCount(totalCard: number, realCard: number): number {
3824     if (totalCard > realCard) {
3825         return Math.max(0, realCard - 1);
3826     }
3827     return Math.max(0, realCard - 2);
3828 }
```

```

3829     private getPassiveSwapCountMultiplier(): number {
3830         const skin = DataManager.instance.equippedSkinId;
3831         if (skin === GameManager.SKIN_PASSIVE_206_BU_DONG_MING_WANG) {
3832             return GameManager.PASSIVE_206_SWAP_COUNT_MULT;
3833         }
3834         if (skin === GameManager.SKIN_PASSIVE_107_MU_TOU_REN) {
3835             return GameManager.PASSIVE_107_SWAP_COUNT_MULT;
3836         }
3837         return 1;
3838     }
3839     private levelConfigHasHole(config: LevelConfig): boolean {
3840         const g = calculateGridSize(config.totalCard);
3841         return config.totalCard < g.rows * g.cols;
3842     }
3843     private buildSwapKindAfterClickArray(config: LevelConfig): number[] {
3844         const n = this.getSwapOpportunityCount(config.totalCard, config.realCard);
3845         const mult = this.getPassiveSwapCountMultiplier();
3846         const deg1 = typeof config.swapDegree === 'number' && !Number.isNaN(config.swapDegree) ?
3847 config.swapDegree : 0;
3848         const rawDeg2 =
3849             typeof config.swapDegree2 === 'number' && !Number.isNaN(config.swapDegree2) ?
3850 config.swapDegree2 : 0;
3851         const deg2 = this.levelConfigHasHole(config) ? rawDeg2 : 0;
3852         let k1 = Math.round(n * deg1 * mult);
3853         let k2 = Math.round(n * deg2 * mult);
3854         if (n > 0) {
3855             if (deg1 > 0 && k1 === 0) {
3856                 k1 = 1;
3857             }
3858             if (deg2 > 0 && k2 === 0) {
3859                 k2 = 1;
3860             }
3861         }
3862         k1 = Math.min(Math.max(0, k1), n);
3863         k2 = Math.min(Math.max(0, k2), n);
3864         if (k1 + k2 > n) {
3865             k2 = Math.max(0, n - k1);
3866         }
3867         const arr: number[] = new Array(n).fill(SWAP_KIND_NONE);
3868         if (n === 0 || (k1 === 0 && k2 === 0)) {
3869             return this.applyPassive105SwapKindGuard(arr);
3870         }
3871         const types: number[] = [
3872             ...Array.from({ length: k1 }, () => SWAP_KIND_PAIR),
3873             ...Array.from({ length: k2 }, () => SWAP_KIND_HOLE),
3874         ];
3875         shuffle(types);
3876         const slotOrder = shuffle(Array.from({ length: n }, (_, i) => i)).slice(0, k1 + k2);
3877         for (let i = 0; i < slotOrder.length; i++) {
3878             arr[slotOrder[i]] = types[i];
3879         }
3880         return this.applyPassive105SwapKindGuard(arr);
3881     }
3882     private applyPassive105SwapKindGuard(arr: number[]): number[] {
3883         if (DataManager.instance.equippedSkinId === GameManager.SKIN_PASSIVE_105_RANG_NI_SAN_ZHAO)
3884         {
3885             for (let i = 0; i < Math.min(3, arr.length); i++) {
3886                 arr[i] = SWAP_KIND_NONE;

```

```
3887     }
3888   }
3889   return arr;
3890 }
3891 restoreClickState(forceSuppressRandomSwap = false): void {
3892   if (!DataManager.instance.isGameContinuing) {
3893     DataManager.instance.isProcessingCard = false;
3894     UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3895     return;
3896   }
3897   const clickIndex = DataManager.instance.clickedCardIds.length;
3898   const kindArr = DataManager.instance.swapKindAfterClick ?? [];
3899   const slotIdx = clickIndex - 1;
3900   const kind =
3901     clickIndex > 0 && clickIndex <= kindArr.length ? kindArr[slotIdx] ?? SWAP_KIND_NONE :
3902   SWAP_KIND_NONE;
3903   const needShuffle = kind !== SWAP_KIND_NONE;
3904   if (needShuffle && !forceSuppressRandomSwap) {
3905     const flipTotal = GameManager.CARD_FLIP_TOTAL_SEC;
3906     const swapDuration = GameManager.CARD_SWAP_DURATION_SEC;
3907     this.scheduleOnce(() => {
3908       if (kind === SWAP_KIND_PAIR) {
3909         this.swapRandomCards();
3910       } else if (kind === SWAP_KIND_HOLE) {
3911         this.slideRandomCardToHole();
3912       }
3913     }, flipTotal * 0.5);
3914     this.scheduleOnce(() => {
3915       DataManager.instance.isProcessingCard = false;
3916       UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3917     }, flipTotal*0.5 + swapDuration);
3918   } else {
3919     this.scheduleOnce(() => {
3920       DataManager.instance.isProcessingCard = false;
3921       UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3922     }, GameManager.CARD_FLIP_TOTAL_SEC * 0.1);
3923   }
3924 }
3925 handleCardClick(cardId: number): void {
3926   if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
3927     return;
3928   }
3929   const card = DataManager.instance.getCard(cardId);
3930   if (!card || card.isClicked) {
3931     DataManager.instance.isProcessingCard = false;
3932     UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
3933     return;
3934   }
3935   if (card.number === 0) {
3936     if (this.startSkill203MisPickResolution(card)) {
3937       return;
3938     }
3939     if (this.tryStartSkill304PhoenixGu(card.id)) {
3940       return;
3941     }
3942     if (this.tryStartSkill204YingShiErDu(card.id)) {
3943       return;
3944     }
3945   }
```

```
3945         DataManager.instance.skill203NextClickEnabled = false;
3946         this.scheduleOnce(() => {
3947             PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.BOMB_EXPLODE);
3948         }, 0.2);
3949         DataManager.instance.isGameContinuing = false;
3950         this.onGameFailed();
3951         return;
3952     }
3953     if (!this.isClickSequenceCorrect(cardId)) {
3954         if (this.startSkill203MisPickResolution(card)) {
3955             return;
3956         }
3957     }
3958     DataManager.instance.skill203NextClickEnabled = false;
3959     const isCorrect = this.checkClickCorrect(cardId);
3960     if (isCorrect) {
3961         console.log('正确点击, number:', card.number);
3962         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CARD_CLICK_RIGHT);
3963         card.isClicked = true;
3964         card.state = CARD_STATE_ENUM.CLICKED;
3965         DataManager.instance.recordCardClick(cardId);
3966         const cardNode = this.safeCardNodes().find(node => {
3967             const cardComponent = node.getComponent(Card);
3968             return cardComponent && cardComponent.cardData && cardComponent.cardData.id ===
3969             cardId;
3970         });
3971         if (cardNode) {
3972             const cardComponent = cardNode.getComponent(Card);
3973             if (cardComponent) {
3974             }
3975         }
3976         const allClicked = DataManager.instance.cards
3977             .filter(c => c.number > 0)
3978             .every(c => c.isClicked);
3979         if (allClicked) {
3980             this.onGameWin();
3981             return;
3982         }
3983         this.restoreClickState();
3984     } else {
3985         console.log('错误点击, number:', card.number);
3986         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CARD_CLICK_WRONG);
3987         if (this.tryStartSkill304PhoenixGu(cardId)) {
3988             return;
3989         }
3990         if (this.tryStartSkill204YingShiErDu(cardId)) {
3991             return;
3992         }
3993         DataManager.instance.isGameContinuing = false;
3994         this.onGameFailed();
3995     }
3996 }
3997 private tryStartSkill204YingShiErDu(_cardId: number): boolean {
3998     if (!DataManager.instance.isYingShiErDu204Available()) {
3999         return false;
4000     }
4001     DataManager.instance.skill204YingShiErDuConsumedThisRun = true;
4002     this.unschedule(this.deferredSkill204AfterReveal);
```

```
4003         this.unschedule(this.deferredSkill204MassFlipBack);
4004         this.scheduleOnce(this.deferredSkill204AfterReveal, GameManager.CARD_FLIP_TOTAL_SEC);
4005         return true;
4006     }
4007     private finishSkill204StateReset(): void {
4008         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
4009             DataManager.instance.isProcessingCard = false;
4010             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
4011             return;
4012         }
4013         DataManager.instance.clickedCardIds = [];
4014         this.currentClickIndex = 0;
4015         for (const n of this.safeCardNodes()) {
4016             const comp = n.getComponent(Card);
4017             if (!comp?.cardData) {
4018                 continue;
4019             }
4020             comp.cardData.isClicked = false;
4021             comp.cardData.state = CARD_STATE_ENUM.HIDDEN;
4022             comp.updateDisplay();
4023         }
4024         DataManager.instance.skill203NextClickEnabled = false;
4025         DataManager.instance.suppressNextCardFlipAfterClick = false;
4026         DataManager.instance.isProcessingCard = false;
4027         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
4028     }
4029     private tryStartSkill304PhoenixGu(cardId: number): boolean {
4030         if (!DataManager.instance.isFengHuangGu304Available()) {
4031             return false;
4032         }
4033         DataManager.instance.skill304FengHuangGuConsumedThisRun = true;
4034         this._skill304PendingCardId = cardId;
4035         this.unschedule(this.deferredSkill304AfterReveal);
4036         this.unschedule(this.deferredSkill304FlipBack);
4037         this.scheduleOnce(this.deferredSkill304AfterReveal, GameManager.CARD_FLIP_TOTAL_SEC);
4038         return true;
4039     }
4040     private startSkill203MisPickResolution(misPickedCard: CardData): boolean {
4041         if (!DataManager.instance.skill203NextClickEnabled) {
4042             return false;
4043         }
4044         const correctCard = this.findNextCorrectUnclickedCard();
4045         if (!correctCard) {
4046             return false;
4047         }
4048         DataManager.instance.skill203NextClickEnabled = false;
4049         DataManager.instance.suppressNextCardFlipAfterClick = true;
4050         const L = DataManager.instance.clickedCardIds.length;
4051         const arr = DataManager.instance.swapKindAfterClick ?? [];
4052         const suppressRandomSwap = L < arr.length && arr[L] !== SWAP_KIND_NONE;
4053         this.unschedule(this.deferredSkill203AfterSwap);
4054         this._skill203Pending = { correctId: correctCard.id, suppressRandomSwap };
4055         this.swapTwoCardsByData(misPickedCard, correctCard);
4056         this.scheduleOnce(this.deferredSkill203AfterSwap, GameManager.CARD_SWAP_DURATION_SEC);
4057         return true;
4058     }
4059     private applySkill203CorrectReveal(correctCardId: number, forceSuppressRandomSwap: boolean): void {
4060         const c = DataManager.instance.getCard(correctCardId);
```

```

4061         if (!c || c.isClicked || c.number === 0) {
4062             DataManager.instance.isProcessingCard = false;
4063             UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
4064             return;
4065         }
4066         console.log('203 纠正后正确, number:', c.number);
4067         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CARD_CLICK_RIGHT);
4068         this.currentClickIndex++;
4069         c.isClicked = true;
4070         c.state = CARD_STATE_ENUM.CLICKED;
4071         DataManager.instance.recordCardClick(correctCardId);
4072         this.getCardComponentByCardId(correctCardId)?.flipToFront();
4073         const allClicked = DataManager.instance.cards.filter(x => x.number > 0).every(x => x.isClicked);
4074         if (allClicked) {
4075             this.onGameWin();
4076             return;
4077         }
4078         this.restoreClickState(forceSuppressRandomSwap);
4079     }
4080     private isCardEligibleForPositionShuffle(c: CardData): boolean {
4081         const passive205 =
4082             DataManager.instance.equippedSkinId === GameManager.SKIN_PASSIVE_205_YI_HUA_JIE_MU;
4083         if (!passive205 && c.isClicked) {
4084             return false;
4085         }
4086         if (c.number === 0) {
4087             const comp = this.getCardComponentByCardId(c.id);
4088             if (!comp) {
4089                 return false;
4090             }
4091             if (comp.isCurrentlyShowingFront()) {
4092                 return passive205;
4093             }
4094         }
4095         return true;
4096     }
4097     private findHoleGridCell(): { x: number; y: number } | null {
4098         const holes = this.collectHoleGridCells();
4099         if (holes.length === 0) {
4100             return null;
4101         }
4102         return holes[randomInt(0, holes.length - 1)];
4103     }
4104     private collectHoleGridCells(): { x: number; y: number }[] {
4105         const cfg = DataManager.instance.currentLevel;
4106         if (!cfg) {
4107             return [];
4108         }
4109         const g = calculateGridSize(cfg.totalCard);
4110         const occupied = new Set<string>();
4111         for (const c of DataManager.instance.cards ?? []) {
4112             occupied.add(`${c.x},${c.y}`);
4113         }
4114         const out: { x: number; y: number }[] = [];
4115         for (let row = 0; row < g.rows; row++) {
4116             for (let col = 0; col < g.cols; col++) {
4117                 if (!occupied.has(`${col},${row}`)) {
4118                     out.push({ x: col, y: row });
4119                 }
4120             }
4121         }
4122         return out;
4123     }

```

```

4119         }
4120     }
4121 }
4122     return out;
4123 }
4124     private getCardGridWorldPosition(col: number, row: number): Vec3 {
4125         const cfg = DataManager.instance.currentLevel;
4126         if (!cfg) {
4127             return new Vec3(0, 0, 0);
4128         }
4129         const g = calculateGridSize(cfg.totalCard);
4130         const cardSpacingX = 174;
4131         const cardSpacingY = 240;
4132         const startX = -(g.cols - 1) * cardSpacingX / 2;
4133         const startY = (g.rows - 1) * cardSpacingY / 2;
4134         return new Vec3(startX + col * cardSpacingX, startY - row * cardSpacingY, 0);
4135     }
4136     private slideCardToGridCell(card: CardData, targetCol: number, targetRow: number, durationSec: number):
4137 void {
4138         const node = this.safeCardNodes().find(n => {
4139             const comp = n.getComponent(Card);
4140             return comp?.cardData?.id === card.id;
4141         });
4142         if (!node) {
4143             return;
4144         }
4145         const targetPos = this.getCardGridWorldPosition(targetCol, targetRow);
4146         card.x = targetCol;
4147         card.y = targetRow;
4148         tween(node).to(durationSec, { position: targetPos }, { easing: 'smooth' }).start();
4149     }
4150     private slideCardToGridCellPromise(
4151         card: CardData,
4152         targetCol: number,
4153         targetRow: number,
4154         durationSec: number
4155     ): Promise<void> {
4156         const node = this.safeCardNodes().find(n => {
4157             const comp = n.getComponent(Card);
4158             return comp?.cardData?.id === card.id;
4159         });
4160         if (!node) {
4161             return Promise.resolve();
4162         }
4163         const targetPos = this.getCardGridWorldPosition(targetCol, targetRow);
4164         card.x = targetCol;
4165         card.y = targetRow;
4166         return new Promise<void>(resolve => {
4167             tween(node)
4168                 .to(durationSec, { position: targetPos }, { easing: 'smooth' })
4169                 .call(() => resolve())
4170                 .start();
4171         });
4172     }
4173     swapRandomCards(): void {
4174         const eligibleCards = (DataManager.instance.cards ?? []).filter((card) =>
4175             this.isCardEligibleForPositionShuffle(card)
4176         );

```



```
4177         if (eligibleCards.length < 2) {
4178             return;
4179         }
4180         const slotIdx = DataManager.instance.clickedCardIds.length - 1;
4181         const slots = this.chunQiuChanSlots();
4182         const existing =
4183             slotIdx >= 0 && slotIdx < slots.length && slots[slotIdx] !== null ? slots[slotIdx] : null;
4184         if (existing?.kind === 'hole') {
4185             return;
4186         }
4187         let card1: CardData;
4188         let card2: CardData;
4189         if (existing?.kind === 'pair') {
4190             const c1 = DataManager.instance.getCard(existing.a);
4191             const c2 = DataManager.instance.getCard(existing.b);
4192             if (
4193                 !c1 ||
4194                 !c2 ||
4195                 !this.isCardEligibleForPositionShuffle(c1) ||
4196                 !this.isCardEligibleForPositionShuffle(c2)
4197             ) {
4198                 return;
4199             }
4200             card1 = c1;
4201             card2 = c2;
4202         } else {
4203             const shuffled = shuffle(eligibleCards);
4204             card1 = shuffled[0];
4205             card2 = shuffled[1];
4206             if (slotIdx >= 0 && slotIdx < slots.length) {
4207                 slots[slotIdx] = { kind: 'pair', a: card1.id, b: card2.id };
4208             }
4209         }
4210         this.swapTwoCardsByData(card1, card2, GameManager.CARD_SWAP_DURATION_SEC);
4211     }
4212     slideRandomCardToHole(): void {
4213         const hole = this.findHoleGridCell();
4214         if (!hole) {
4215             return;
4216         }
4217         const eligibleCards = (DataManager.instance.cards ?? []).filter((card) =>
4218             this.isCardEligibleForPositionShuffle(card)
4219         );
4220         if (eligibleCards.length === 0) {
4221             return;
4222         }
4223         const slotIdx = DataManager.instance.clickedCardIds.length - 1;
4224         const slots = this.chunQiuChanSlots();
4225         const existing =
4226             slotIdx >= 0 && slotIdx < slots.length && slots[slotIdx] !== null ? slots[slotIdx] : null;
4227         if (existing?.kind === 'pair') {
4228             return;
4229         }
4230         if (existing?.kind === 'hole') {
4231             const c = DataManager.instance.getCard(existing.cardId);
4232             if (
4233                 !c ||
4234                 !this.isCardEligibleForPositionShuffle(c) ||
```

```

4235         c.x !== existing.cardX ||
4236         c.y !== existing.cardY
4237     ) {
4238         return;
4239     }
4240     const holeStillEmpty = !(DataManager.instance.cards ?? []).some(
4241         (card) => card.x === existing.holeX && card.y === existing.holeY
4242     );
4243     if (!holeStillEmpty) {
4244         return;
4245     }
4246     this.slideCardToGridCell(c, existing.holeX, existing.holeY,
4247     GameManager.CARD_SWAP_DURATION_SEC);
4248     return;
4249 }
4250 const shuffled = shuffle(eligibleCards);
4251 const picked = shuffled[0];
4252 if (slotIdx >= 0 && slotIdx < slots.length) {
4253     slots[slotIdx] = {
4254         kind: 'hole',
4255         cardId: picked.id,
4256         cardX: picked.x,
4257         cardY: picked.y,
4258         holeX: hole.x,
4259         holeY: hole.y,
4260     };
4261 }
4262 this.slideCardToGridCell(picked, hole.x, hole.y, GameManager.CARD_SWAP_DURATION_SEC);
4263 }
4264 private swapTwoCardsByData(
4265     cardA: CardData,
4266     cardB: CardData,
4267     durationSec: number = GameManager.CARD_SWAP_DURATION_SEC
4268 ): void {
4269     const node1 = this.safeCardNodes().find(node => {
4270         const cardComponent = node.getComponent(Card);
4271         return cardComponent?.cardData?.id === cardA.id;
4272     });
4273     const node2 = this.safeCardNodes().find(node => {
4274         const cardComponent = node.getComponent(Card);
4275         return cardComponent?.cardData?.id === cardB.id;
4276     });
4277     if (!node1 || !node2) {
4278         return;
4279     }
4280     const pos1 = node1.getPosition();
4281     const pos2 = node2.getPosition();
4282     const tempX = cardA.x;
4283     const tempY = cardA.y;
4284     cardA.x = cardB.x;
4285     cardA.y = cardB.y;
4286     cardB.x = tempX;
4287     cardB.y = tempY;
4288     tween(node1)
4289         .to(durationSec, { position: pos2 }, { easing: 'smooth' })
4290         .start();
4291     tween(node2)
4292         .to(durationSec, { position: pos1 }, { easing: 'smooth' })

```

```
4293         .start();
4294     }
4295     private swapTwoCardsByDataPromise(cardA: CardData, cardB: CardData, durationSec: number):
4296     Promise<void> {
4297         const node1 = this.safeCardNodes().find(node => {
4298             const cardComponent = node.getComponent(Card);
4299             return cardComponent?.cardData?.id === cardA.id;
4300         });
4301         const node2 = this.safeCardNodes().find(node => {
4302             const cardComponent = node.getComponent(Card);
4303             return cardComponent?.cardData?.id === cardB.id;
4304         });
4305         if (!node1 || !node2) {
4306             return Promise.resolve();
4307         }
4308         const pos1 = node1.getPosition();
4309         const pos2 = node2.getPosition();
4310         const tempX = cardA.x;
4311         const tempY = cardA.y;
4312         cardA.x = cardB.x;
4313         cardA.y = cardB.y;
4314         cardB.x = tempX;
4315         cardB.y = tempY;
4316         return new Promise<void>(resolve => {
4317             let done = 0;
4318             const check = (): void => {
4319                 if (++done >= 2) {
4320                     resolve();
4321                 }
4322             };
4323             tween(node1)
4324                 .to(durationSec, { position: pos2 }, { easing: 'smooth' })
4325                 .call(check)
4326                 .start();
4327             tween(node2)
4328                 .to(durationSec, { position: pos1 }, { easing: 'smooth' })
4329                 .call(check)
4330                 .start();
4331         });
4332     }
4333     private getCardComponentByCardId(cardId: number): Card | null {
4334         const node = this.safeCardNodes().find(n => {
4335             const comp = n.getComponent(Card);
4336             return comp?.cardData?.id === cardId;
4337         });
4338         return node?.getComponent(Card) ?? null;
4339     }
4340     private findFaceDownUnclickedBombCards(): CardData[] {
4341         const out: CardData[] = [];
4342         for (const c of DataManager.instance.cards) {
4343             if (c.number !== 0 || c.isClicked) {
4344                 continue;
4345             }
4346             const comp = this.getCardComponentByCardId(c.id);
4347             if (comp && !comp.isCurrentlyShowingFront()) {
4348                 out.push(c);
4349             }
4350         }
4351     }
```

```

4351         return out;
4352     }
4353     public canRevealBombsForBombRevealSkill(): boolean {
4354         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
4355             return false;
4356         }
4357         if (DataManager.instance.isProcessingCard) {
4358             return false;
4359         }
4360         return this.findFaceDownUnclickedBombCards().length >= 1;
4361     }
4362     private revealRandomFaceDownBombs(maxCount: number): void {
4363         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
4364             return;
4365         }
4366         const pool = this.findFaceDownUnclickedBombCards();
4367         if (pool.length === 0) {
4368             return;
4369         }
4370         const k = Math.max(0, Math.floor(maxCount));
4371         const picks = shuffle([...pool]).slice(0, k);
4372         for (const card of picks) {
4373             this.getCardComponentByCardId(card.id)?.flipToFront();
4374         }
4375     }
4376     public revealRandomBombsForSkill102(): void {
4377         this.revealRandomFaceDownBombs(1);
4378     }
4379     public revealRandomBombsForSkill202(): void {
4380         this.revealRandomFaceDownBombs(2);
4381     }
4382     private getNextExpectedNumber(): number | null {
4383         if (this.currentClickIndex >= this.expectedClickOrder.length) {
4384             return null; // 所有卡片都已点击
4385         }
4386         return this.expectedClickOrder[this.currentClickIndex];
4387     }
4388     private findNextCorrectUnclickedCard(): CardData | null {
4389         const expected = this.getNextExpectedNumber();
4390         if (expected === null) {
4391             return null;
4392         }
4393         return (
4394             DataManager.instance.cards.find(c => !c.isClicked && c.number === expected && c.number > 0) ??
4395             null
4396         );
4397     }
4398     private resolveSimulateNextCorrectTarget(): CardData | null {
4399         if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.CLICKING) {
4400             return null;
4401         }
4402         if (DataManager.instance.isProcessingCard) {
4403             return null;
4404         }
4405         return this.findNextCorrectUnclickedCard();
4406     }
4407     public canSimulateNextCorrectCardClick(): boolean {
4408         return this.resolveSimulateNextCorrectTarget() !== null;

```

```

4409     }
4410     public simulateNextCorrectCardClick(): boolean {
4411         const card = this.resolveSimulateNextCorrectTarget();
4412         if (!card) {
4413             return false;
4414         }
4415         DataManager.instance.isProcessingCard = true;
4416         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
4417         this.handleCardClick(card.id);
4418         const cardNode = this.safeCardNodes().find(node => {
4419             const cardComponent = node.getComponent(Card);
4420             return cardComponent?.cardData?.id === card.id;
4421         });
4422         cardNode?.getComponent(Card)?.flipToFront();
4423         return true;
4424     }
4425     private isClickSequenceCorrect(cardId: number): boolean {
4426         const c = DataManager.instance.getCard(cardId);
4427         if (!c || c.number === 0) {
4428             return false;
4429         }
4430         const expectedNumber = this.getNextExpectedNumber();
4431         if (expectedNumber === null) {
4432             return false;
4433         }
4434         return c.number === expectedNumber;
4435     }
4436     private checkClickCorrect(cardId: number): boolean {
4437         if (!this.isClickSequenceCorrect(cardId)) {
4438             return false;
4439         }
4440         this.currentClickIndex++;
4441         return true;
4442     }
4443     private static readonly CHUN_QIU_CHAN_AFTER_SKILL_BACK_LOGO_ID = 4011;
4444     private buildChunQiuChanRewindSteps(): { flipId: number; swapRecord: ChunQiuChanSwapSlotRecord |
4445     null }[] {
4446         const ids = [...(DataManager.instance.clickedCardIds ?? [])];
4447         const kinds = DataManager.instance.swapKindAfterClick ?? [];
4448         const records = this.chunQiuChanSlots();
4449         const out: { flipId: number; swapRecord: ChunQiuChanSwapSlotRecord | null }[] = [];
4450         for (let i = 0; i < ids.length; i++) {
4451             let rec: ChunQiuChanSwapSlotRecord | null = null;
4452             if (i < kinds.length && kinds[i] !== SWAP_KIND_NONE && i < records.length && records[i] !== null)
4453             {
4454                 rec = records[i]!;
4455             }
4456             out.push({ flipId: ids[i], swapRecord: rec });
4457         }
4458         return out;
4459     }
4460     private flipCardToBackForRewindPromise(flipId: number): Promise<void> {
4461         const comp = this.getCardComponentByCardId(flipId);
4462         if (!comp) {
4463             return Promise.resolve();
4464         }
4465         return new Promise<void>(resolve => {
4466             comp.flipToBack(() => resolve());

```

```
4467     });
4468   }
4469   public executeChunQiuChanSkill(): void {
4470     void this.runChunQiuChanSkillAsync();
4471   }
4472   public canActivateChunQiuChanSkill(): boolean {
4473     return (
4474       DataManager.instance.gameStatus === GAME_STATE_ENUM.CLICKING &&
4475       DataManager.instance.isGameContinuing &&
4476       !DataManager.instance.isProcessingCard
4477     );
4478   }
4479   private async runChunQiuChanSkillAsync(): Promise<void> {
4480     const token = ++this.chunQiuChanSkillToken;
4481     this.unscheduleAllCallbacks();
4482     this.cancelEndGameSequences();
4483     for (const n of this.safeCardNodes()) {
4484       Tween.stopAllByTarget(n);
4485       n.getComponent(Card)?.interruptFlipTween();
4486     }
4487     const hadProgress =
4488       DataManager.instance.clickedCardIds.length > 0 ||
4489       this.chunQiuChanSlots().some(p => p !== null);
4490     DataManager.instance.isProcessingCard = true;
4491     this.changeGameState(GAME_STATE_ENUM.PAUSED);
4492     const half = 0.5;
4493     const swapDur = GameManager.CARD_SWAP_DURATION_SEC * half;
4494     const flipHalf = GameManager.CARD_FLIP_HALF_DURATION_SEC * half;
4495     for (const n of this.safeCardNodes()) {
4496       n.getComponent(Card)?.setFlipDuration(flipHalf);
4497     }
4498     const steps = this.buildChunQiuChanRewindSteps();
4499     for (let s = steps.length - 1; s >= 0; s--) {
4500       if (token !== this.chunQiuChanSkillToken) {
4501         return;
4502       }
4503       const st = steps[s];
4504       if (st.swapRecord?.kind === 'pair') {
4505         const ca = DataManager.instance.getCard(st.swapRecord.a);
4506         const cb = DataManager.instance.getCard(st.swapRecord.b);
4507         if (ca && cb) {
4508           await this.swapTwoCardsByDataPromise(ca, cb, swapDur);
4509         }
4510       } else if (st.swapRecord?.kind === 'hole') {
4511         const c = DataManager.instance.getCard(st.swapRecord.cardId);
4512         if (c) {
4513           await this.slideCardToGridCellPromise(
4514             c,
4515             st.swapRecord.cardX,
4516             st.swapRecord.cardY,
4517             swapDur
4518           );
4519         }
4520       }
4521       if (token !== this.chunQiuChanSkillToken) {
4522         return;
4523       }
4524       await this.flipCardToBackForRewindPromise(st.flipId);
```

```
4525     }
4526     if (token !== this.chunQiuChanSkillToken) {
4527         return;
4528     }
4529     for (const n of this.safeCardNodes()) {
4530         const comp = n.getComponent(Card);
4531         if (!comp?.cardData) {
4532             continue;
4533         }
4534         if (comp.isCurrentlyShowingFront()) {
4535             await new Promise<void>(res => comp.flipToBack(() => res()));
4536         }
4537     }
4538     if (token !== this.chunQiuChanSkillToken) {
4539         return;
4540     }
4541     DataManager.instance.clickedCardIds = [];
4542     this.currentClickIndex = 0;
4543     for (const n of this.safeCardNodes()) {
4544         const comp = n.getComponent(Card);
4545         if (!comp?.cardData) {
4546             continue;
4547         }
4548         comp.cardData.isClicked = false;
4549         comp.cardData.state = CARD_STATE_ENUM.HIDDEN;
4550         comp.updateDisplay();
4551     }
4552     for (const n of this.safeCardNodes()) {
4553         n.getComponent(Card)?.setFlipDuration(GameManager.CARD_FLIP_HALF_DURATION_SEC);
4554     }
4555     DataManager.instance.chunQiuChanReplayingAfterSkill = hadProgress;
4556     DataManager.instance.isGameContinuing = true;
4557     DataManager.instance.skill203NextClickEnabled = false;
4558     DataManager.instance.suppressNextCardFlipAfterClick = false;
4559     this.changeGameState(GAME_STATE_ENUM.PREPARE);
4560     await delay(500);
4561     if (token !== this.chunQiuChanSkillToken) {
4562         return;
4563     }
4564     this.displayPhaseUseFlipToFrontTransition = true;
4565     this._pendingChunQiuChanBackLogoAfterDisplay = true;
4566     await this.displayPhase();
4567     if (token !== this.chunQiuChanSkillToken) {
4568         return;
4569     }
4570     await this.flipPhase();
4571     if (token !== this.chunQiuChanSkillToken) {
4572         return;
4573     }
4574     DataManager.instance.isProcessingCard = false;
4575     UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
4576     this.changeGameState(GAME_STATE_ENUM.CLICKING);
4577 }
4578 onGameWin(): void {
4579     DataManager.instance.isProcessingCard = true;
4580     this.changeGameState(GAME_STATE_ENUM.PAUSED);
4581     this.scheduleOnce(this.deferredWinShow, GameManager.WIN_DELAY_BEFORE_SETTLEMENT_SEC);
4582 }
```

```
4583     private deferredWinShow = (): void => {
4584         this.changeGameState(GAME_STATE_ENUM.WIN);
4585         LEVEL_COMPLETE.emit(EVENT_ENUM.LEVEL_COMPLETE, DataManager.instance.level);
4586     };
4587     onGameFailed(): void {
4588         DataManager.instance.isProcessingCard = true;
4589         this.changeGameState(GAME_STATE_ENUM.PAUSED);
4590         this.scheduleOnce(this.deferredFailAfterTriggerFlip, GameManager.CARD_FLIP_TOTAL_SEC);
4591     }
4592     private deferredFailAfterTriggerFlip = (): void => {
4593         this.deferredFailRevealPhase();
4594     };
4595     private deferredFailRevealPhase = (): void => {
4596         const promises: Promise<void>[] = [];
4597         const red = GameManager.FAIL_UNCLICKED_NUMBER_COLOR;
4598         for (const node of this.safeCardNodes()) {
4599             const comp = node.getComponent(Card);
4600             if (!comp?.cardData) {
4601                 continue;
4602             }
4603             const d = comp.cardData;
4604             if (d.number > 0 && !d.isClicked && comp.frontLabel) {
4605                 comp.frontLabel.color = red;
4606             }
4607             promises.push(
4608                 new Promise<void>(resolve => {
4609                     comp.flipToFront(() => resolve());
4610                 })
4611             );
4612         }
4613         if (promises.length === 0) {
4614             this.scheduleOnce(this.deferredFailEmitFailed,
4615                 GameManager.FAIL_PAUSE_AFTER_MASS_REVEAL_SEC);
4616             return;
4617         }
4618         void Promise.all(promises).then(() => {
4619             this.scheduleOnce(this.deferredFailEmitFailed,
4620                 GameManager.FAIL_PAUSE_AFTER_MASS_REVEAL_SEC);
4621         });
4622     };
4623     private deferredFailEmitFailed = (): void => {
4624         this.emitFailedStateAndEvent();
4625     };
4626     private emitFailedStateAndEvent(): void {
4627         this.changeGameState(GAME_STATE_ENUM.FAILED);
4628         LEVEL_FAILED.emit(EVENT_ENUM.LEVEL_FAILED, DataManager.instance.level);
4629     }
4630     private cancelEndGameSequences(): void {
4631         this.unschedule(this.deferredWinShow);
4632         this.unschedule(this.deferredFailAfterTriggerFlip);
4633         this.unschedule(this.deferredFailRevealPhase);
4634         this.unschedule(this.deferredFailEmitFailed);
4635         this.unschedule(this.deferredSkill203AfterSwap);
4636         this._skill203Pending = null;
4637         this.unschedule(this.deferredSkill304AfterReveal);
4638         this.unschedule(this.deferredSkill304FlipBack);
4639         this._skill304PendingCardId = null;
4640         this.unschedule(this.deferredSkill204AfterReveal);
```



```
4641         this.unschedule(this.deferredSkill204MassFlipBack);
4642         this._pendingChunQiuChanBackLogoAfterDisplay = false;
4643     }
4644     changeGameState(state: GAME_STATE_ENUM): void {
4645         DataManager.instance.gameStatus = state;
4646         GAME_STATE_CHANGE.emit(EVENT_ENUM.GAME_STATE_CHANGE, state);
4647     }
4648 }
4649 ...
4650 ---
4651 ## 文件: `Manager/GameUIManager.ts`
4652 ```typescript
4653 import { _decorator, Component, Node, Label, director, macro, Widget, Button, UIOpacity } from 'cc';
4654 import { ACTIVE_SKILL_CARD_INTERCEPT_MODE, EVENT_ENUM, GAME_STATE_ENUM, SCENE_ENUM,
4655 AUDIO_EFFECT_ENUM } from '../Utils/Enum';
4656 import {
4657     ACTIVE_SKILL_PROPHET_CARD_TAP,
4658     ACTIVE_SKILL_VERIFY_CARD_TAP,
4659     UI_UPDATE,
4660     COIN_UPDATE,
4661     GAME_STATE_CHANGE,
4662     LEVEL_START,
4663     LEVEL_COMPLETE,
4664     LEVEL_FAILED,
4665     SCENE_CHANGE,
4666     PLAY_AUDIO,
4667     PASSIVE_304_FENG_HUANG_GU_TRIGGER,
4668     PASSIVE_204_YING_SHI_ER_DU_TRIGGER,
4669 } from '../Utils/Event';
4670 import { AD_RETRY_MAX_CHARGES, DataManager } from './DataManager';
4671 import { GameManager } from './GameManager';
4672 import { getSkillConfigBySkinId } from './Config/Skills';
4673 import { LevelConfig } from './Config/GameTypes';
4674 import { ToastManager } from './ToastManager';
4675 const { ccclass, property } = _decorator;
4676 @ccclass('GameUIManager')
4677 export class GameUIManager extends Component {
4678     @property(Label)
4679     levelLabel: Label = null;
4680     @property(Label)
4681     skillCostLabel: Label = null;
4682     @property(Node)
4683     retryButtonNode: Node = null;
4684     @property(Node)
4685     skillButtonNode: Node = null;
4686     @property(Label)
4687     retryAdCountLabel: Label = null;
4688     @property(Label)
4689     silverCoinLabel: Label = null;
4690     @property(Label)
4691     goldCoinLabel: Label = null;
4692     @property(Node)
4693     backToMenuButtonNode: Node = null;
4694     @property(Label)
4695     memoryHintLabel: Label = null;
4696     @property(Node)
4697     gameWinResultNode: Node = null;
4698     @property(Label)
```

```
4699     winRewardGoldLabel: Label = null;
4700     @property(Label)
4701     winRewardSilverLabel: Label = null;
4702     @property(Node)
4703     winRewardGoldNode: Node = null;
4704     @property(Node)
4705     winRewardSilverNode: Node = null;
4706     @property(Node)
4707     winNextLevelButtonNode: Node = null;
4708     @property(Label)
4709     winNextLevelButtonLabel: Label = null;
4710     @property(Node)
4711     winDoubleRewardAdButtonNode: Node = null;
4712     @property(Node)
4713     gameLoseResultNode: Node = null;
4714     @property(Node)
4715     failReturnToMenuButtonNode: Node = null;
4716     @property(Node)
4717     failRetryChallengeButtonNode: Node = null;
4718     @property	ToastManager)
4719     toastManager: ToastManager = null;
4720     private static readonly FAIL_RETURN_LOCAL_X_WITH_RETRY = 85;
4721     private static readonly WIN_REWARD_SILVER_LOCAL_X_WITH_GOLD = -85;
4722     private static readonly SKIN_PASSIVE_WIN_GOLD_BONUS = 402;
4723     private static readonly SKIN_PASSIVE_WIN_SILVER_BONUS = 201;
4724     private static readonly PASSIVE_WIN_COIN_MULT = 1.5;
4725     private static readonly WIN_COIN_PASSIVE_TOAST_BG_WIDTH = 400;
4726     private static readonly WIN_COIN_PASSIVE_TOAST_LOCAL_Y = 250;
4727     private static readonly RETRY_OPACITY_NORMAL = 255;
4728     private static readonly RETRY_OUTER_NO_CHARGES = 180;
4729     private static readonly RETRY_ICON_NO_CHARGES = 120;
4730     private static readonly SKILL_OPACITY_NORMAL = 255;
4731     private static readonly SKILL_OUTER_USED = 180;
4732     private static readonly SKILL_ICON_USED = 150;
4733     private static readonly SKILL_POST_TOAST_DELAY_SEC = 0.3;
4734     private _memoryEndAtMs = 0;
4735     private _winSettlementConsumed = false;
4736     private _failSettlementConsumed = false;
4737     private _winResultPanelVisible = false;
4738     private _loseResultPanelVisible = false;
4739     private readonly skill302FlipCallback = (): void => {
4740         this.findGameManager()?.simulateNextCorrectCardClick();
4741     };
4742     private readonly skill102RevealCallback = (): void => {
4743         this.findGameManager()?.revealRandomBombsForSkill102();
4744     };
4745     private readonly skill202RevealCallback = (): void => {
4746         this.findGameManager()?.revealRandomBombsForSkill202();
4747     };
4748     private readonly skill101RestartCallback = (): void => {
4749         LEVEL_START.emit(EVENT_ENUM.LEVEL_START, DataManager.instance.level);
4750     };
4751     private readonly skill401ChunQiuChanCallback = (): void => {
4752         this.findGameManager()?.executeChunQiuChanSkill();
4753     };
4754     protected onLoad(): void {
4755         UI_UPDATE.on(EVENT_ENUM.UI_UPDATE, this.onUIUpdate, this);
4756         COIN_UPDATE.on(EVENT_ENUM.COIN_UPDATE, this.onCoinUpdate, this);
```

```

4757     GAME_STATE_CHANGE.on(EVENT_ENUM.GAME_STATE_CHANGE, this.onGameStateChange, this);
4758     LEVEL_START.on(EVENT_ENUM.LEVEL_START, this.onLevelStart, this);
4759     LEVEL_COMPLETE.on(EVENT_ENUM.LEVEL_COMPLETE, this.onLevelComplete, this);
4760     LEVEL_FAILED.on(EVENT_ENUM.LEVEL_FAILED, this.onLevelFailed, this);
4761     ACTIVE_SKILL_VERIFY_CARD_TAP.on(
4762         EVENT_ENUM.ACTIVE_SKILL_VERIFY_CARD_TAP,
4763         this.onActiveSkillVerify301CardTap,
4764         this
4765     );
4766     ACTIVE_SKILL_PROPHET_CARD_TAP.on(
4767         EVENT_ENUM.ACTIVE_SKILL_PROPHET_CARD_TAP,
4768         this.onActiveSkillProphet103CardTap,
4769         this
4770     );
4771     PASSIVE_304_FENG_HUANG_GU_TRIGGER.on(
4772         EVENT_ENUM.PASSIVE_304_FENG_HUANG_GU_TRIGGER,
4773         this.onPassive304FengHuangGuToast,
4774         this
4775     );
4776     PASSIVE_204_YING_SHI_ER_DU_TRIGGER.on(
4777         EVENT_ENUM.PASSIVE_204_YING_SHI_ER_DU_TRIGGER,
4778         this.onPassive204YingShiErDuToast,
4779         this
4780     );
4781     director.preloadScene(SCENE_ENUM.MENU);
4782     this.backToMenuButtonNode.on(Node.EventType.TOUCH_END, this.onBackToMenuClick, this);
4783     if (this.retryButtonNode) {
4784         this.retryButtonNode.on(Node.EventType.TOUCH_END, this.onRetryButtonClick, this);
4785     }
4786     if (this.skillButtonNode) {
4787         this.skillButtonNode.on(Node.EventType.TOUCH_END, this.onSkillButtonClick, this);
4788     }
4789     if (this.winNextLevelButtonNode) {
4790         this.winNextLevelButtonNode.on(Node.EventType.TOUCH_END, this.onWinNextLevelClick, this);
4791     }
4792     if (this.winDoubleRewardAdButtonNode) {
4793         this.winDoubleRewardAdButtonNode.on(Node.EventType.TOUCH_END,
4794 this.onWinDoubleRewardAdClick, this);
4795     }
4796     if (this.failReturnToMenuButtonNode) {
4797         this.failReturnToMenuButtonNode.on(Node.EventType.TOUCH_END, this.onFailReturnToMenuClick,
4798 this);
4799     }
4800     if (this.failRetryChallengeButtonNode) {
4801         this.failRetryChallengeButtonNode.on(Node.EventType.TOUCH_END,
4802 this.onFailRetryChallengeClick, this);
4803     }
4804     this.init();
4805 }
4806 protected onDestroy(): void {
4807     UI_UPDATE.off(EVENT_ENUM.UI_UPDATE, this.onUIUpdate, this);
4808     COIN_UPDATE.off(EVENT_ENUM.COIN_UPDATE, this.onCoinUpdate, this);
4809     GAME_STATE_CHANGE.off(EVENT_ENUM.GAME_STATE_CHANGE, this.onGameStateChange, this);
4810     LEVEL_START.off(EVENT_ENUM.LEVEL_START, this.onLevelStart, this);
4811     LEVEL_COMPLETE.off(EVENT_ENUM.LEVEL_COMPLETE, this.onLevelComplete, this);
4812     LEVEL_FAILED.off(EVENT_ENUM.LEVEL_FAILED, this.onLevelFailed, this);
4813     ACTIVE_SKILL_VERIFY_CARD_TAP.off(
4814         EVENT_ENUM.ACTIVE_SKILL_VERIFY_CARD_TAP,

```

```
4815         this.onActiveSkillVerify301CardTap,
4816         this
4817     );
4818     ACTIVE_SKILL_PROPHET_CARD_TAP.off(
4819         EVENT_ENUM.ACTIVE_SKILL_PROPHET_CARD_TAP,
4820         this.onActiveSkillProphet103CardTap,
4821         this
4822     );
4823     PASSIVE_304_FENG_HUANG_GU_TRIGGER.off(
4824         EVENT_ENUM.PASSIVE_304_FENG_HUANG_GU_TRIGGER,
4825         this.onPassive304FengHuangGuToast,
4826         this
4827     );
4828     PASSIVE_204_YING_SHI_ER_DU_TRIGGER.off(
4829         EVENT_ENUM.PASSIVE_204_YING_SHI_ER_DU_TRIGGER,
4830         this.onPassive204YingShiErDuToast,
4831         this
4832     );
4833     if (this.backToMenuButtonNode?.isValid) {
4834         this.backToMenuButtonNode.off(Node.EventType.TOUCH_END, this.onBackToMenuClick, this);
4835     }
4836     if (this.retryButtonNode?.isValid) {
4837         this.retryButtonNode.off(Node.EventType.TOUCH_END, this.onRetryButtonClick, this);
4838     }
4839     if (this.skillButtonNode?.isValid) {
4840         this.skillButtonNode.off(Node.EventType.TOUCH_END, this.onSkillButtonClick, this);
4841     }
4842     if (this.winNextLevelButtonNode?.isValid) {
4843         this.winNextLevelButtonNode.off(Node.EventType.TOUCH_END, this.onWinNextLevelClick, this);
4844     }
4845     if (this.winDoubleRewardAdButtonNode?.isValid) {
4846         this.winDoubleRewardAdButtonNode.off(Node.EventType.TOUCH_END,
4847         this.onWinDoubleRewardAdClick, this);
4848     }
4849     if (this.failReturnToMenuButtonNode?.isValid) {
4850         this.failReturnToMenuButtonNode.off(Node.EventType.TOUCH_END,
4851         this.onFailReturnToMenuClick, this);
4852     }
4853     if (this.failRetryChallengeButtonNode?.isValid) {
4854         this.failRetryChallengeButtonNode.off(Node.EventType.TOUCH_END,
4855         this.onFailRetryChallengeClick, this);
4856     }
4857     this.unschedule(this.skill302FlipCallback);
4858     this.unschedule(this.skill102RevealCallback);
4859     this.unschedule(this.skill202RevealCallback);
4860     this.unschedule(this.skill101RestartCallback);
4861     this.unschedule(this.skill401ChunQiuChanCallback);
4862     this.clearMemoryHintSchedulers();
4863 }
4864 init(): void {
4865     DataManager.instance.init();
4866     this.updateAllUI();
4867     this.hideGameResultPanels();
4868 }
4869 onUIUpdate(): void {
4870     this.updateAllUI();
4871 }
4872 onCoinUpdate(): void {
```

```
4873         this.updateCoinDisplay();
4874     }
4875     onGameStateChange(state: GAME_STATE_ENUM): void {
4876         this.updateAllUI();
4877         if (state === GAME_STATE_ENUM.PREPARE) {
4878             this.hideGameResultPanels();
4879             this.clearMemoryHintSchedulers();
4880             this.setMemoryWaitTextFromLevel();
4881         } else if (state === GAME_STATE_ENUM.DISPLAY) {
4882             this.startMemoryCountdown();
4883         } else if (state === GAME_STATE_ENUM.WIN) {
4884             this.clearMemoryHintSchedulers();
4885             this.setMemoryHintText("");
4886             this.showGameResultPanels(true, false);
4887         } else if (state === GAME_STATE_ENUM.FAILED) {
4888             this.clearMemoryHintSchedulers();
4889             this.setMemoryHintText("");
4890             this.showGameResultPanels(false, true);
4891         }
4892     }
4893     onLevelStart(levelId: number): void {
4894         this.clearMemoryHintSchedulers();
4895         this.hideGameResultPanels();
4896         this._winSettlementConsumed = false;
4897         this._failSettlementConsumed = false;
4898         this.updateLevelDisplay();
4899     }
4900     onLevelComplete(levelId: number): void {
4901         this.updateCoinDisplay();
4902         this.showGameResultPanels(true, false);
4903     }
4904     onLevelFailed(levelId: number): void {
4905         this.showGameResultPanels(false, true);
4906     }
4907     updateAllUI(): void {
4908         this.updateLevelDisplay();
4909         this.updateSkillCostDisplay();
4910         this.updateCoinDisplay();
4911         this.updateSkillButtonsLayout();
4912         this.updateRetryAdCountLabel();
4913         this.updateRetryButtonInteractable();
4914         this.updateSkillButtonInteractable();
4915     }
4916     showToast(message: string, bgWidth?: number, localY: number = 0): void {
4917         if (this.toastManager?.isValid) {
4918             this.toastManager.show(message, bgWidth, localY);
4919         }
4920     }
4921     private onPassive304FengHuangGuToast(): void {
4922         this.showToast('「凤凰蛊」技能发动', 400);
4923     }
4924     private onPassive204YingShiErDuToast(): void {
4925         this.showToast('「龙胤之力」技能发动', 430);
4926     }
4927     updateLevelDisplay(): void {
4928         if (this.levelLabel) {
4929             const level = DataManager.instance.level;
4930             this.levelLabel.string = `第${level}关`;
```

```

4931     }
4932 }
4933 private updateSkillCostDisplay(): void {
4934     if (!this.skillCostLabel) {
4935         return;
4936     }
4937     const cfg = DataManager.instance.getResolvedLevelConfig();
4938     const cost = cfg?.skillCost ?? 0;
4939     this.skillCostLabel.string = `×${cost}`;
4940 }
4941 private updateSkillButtonsLayout(): void {
4942     const equippedSkinId = DataManager.instance.equippedSkinId;
4943     const cfg = getSkillConfigBySkinId(equippedSkinId);
4944     const isActive = !!cfg?.isActive;
4945     const retryWidget = this.retryButtonNode?.getComponent(Widget);
4946     if (retryWidget?.enabled) {
4947         retryWidget.enabled = false;
4948     }
4949     const skillWidget = this.skillButtonNode?.getComponent(Widget);
4950     if (skillWidget?.enabled) {
4951         skillWidget.enabled = false;
4952     }
4953     const retryPosX = isActive ? -170 : 0;
4954     if (this.retryButtonNode?.isValid) {
4955         this.retryButtonNode.setPosition(retryPosX, 0, 0);
4956         this.retryButtonNode.active = true;
4957     }
4958     if (this.skillButtonNode?.isValid) {
4959         this.skillButtonNode.active = isActive;
4960     }
4961 }
4962 private canCardsBeClickedNow(): boolean {
4963     return (
4964         DataManager.instance.gameStatus === GAME_STATE_ENUM.CLICKING &&
4965         !DataManager.instance.isProcessingCard
4966     );
4967 }
4968 private canRetryInteract(): boolean {
4969     return this.canCardsBeClickedNow() && DataManager.instance.adRetryRemaining > 0;
4970 }
4971 private getResolvedSkillCost(): number {
4972     const cfg = DataManager.instance.getResolvedLevelConfig();
4973     const c = cfg?.skillCost;
4974     return typeof c === 'number' && Number.isFinite(c) ? Math.max(0, c) : 0;
4975 }
4976 private isEquippedSkill101ConsumedThisCycle(): boolean {
4977     return (
4978         DataManager.instance.equippedSkinId === 101 &&
4979         DataManager.instance.skill101ConsumedThisLevelCycle
4980     );
4981 }
4982 private canSkillInteract(): boolean {
4983     if (
4984         !this.canCardsBeClickedNow() ||
4985         DataManager.instance.activeSkillConsumedThisRun ||
4986         this.isEquippedSkill101ConsumedThisCycle()
4987     ) {
4988         return false;

```

```

4989     }
4990     return DataManager.instance.silverCoin >= this.getResolvedSkillCost();
4991 }
4992 private updateRetryAdCountLabel(): void {
4993     if (!this.retryAdCountLabel) {
4994         return;
4995     }
4996     const x = DataManager.instance.adRetryRemaining;
4997     this.retryAdCountLabel.string = `(${x})/${AD_RETRY_MAX_CHARGES}`;
4998 }
4999 private updateRetryButtonInteractable(): void {
5000     const btn = this.retryButtonNode?.getComponent(Button);
5001     if (btn) {
5002         btn.interactable = this.canRetryInteract();
5003     }
5004     this.syncRetryButtonOpacityByAdCharges();
5005 }
5006 private updateSkillButtonInteractable(): void {
5007     const btn = this.skillButtonNode?.getComponent(Button);
5008     if (btn) {
5009         btn.interactable = this.canSkillInteract();
5010     }
5011     this.syncSkillButtonOpacityByConsumedState();
5012 }
5013 private applySkillButtonOuterAndIconOpacity(outerOpacity: number, iconOpacity: number): void {
5014     this.setNodeUIOpacity(this.skillButtonNode, outerOpacity);
5015     const cost = this.skillButtonNode?.getChildByName('cost');
5016     const icon = cost?.getChildByName('icon');
5017     this.setNodeUIOpacity(icon ?? null, iconOpacity);
5018 }
5019 private syncSkillButtonOpacityByConsumedState(): void {
5020     if (!this.skillButtonNode?.isValid) {
5021         return;
5022     }
5023     if (DataManager.instance.activeSkillConsumedThisRun || this.isEquippedSkill101ConsumedThisCycle())
5024 {
5025         this.applySkillButtonOuterAndIconOpacity(
5026             GameUIManager.SKILL_OUTER_USED,
5027             GameUIManager.SKILL_ICON_USED
5028         );
5029     } else {
5030         this.applySkillButtonOuterAndIconOpacity(
5031             GameUIManager.SKILL_OPACITY_NORMAL,
5032             GameUIManager.SKILL_OPACITY_NORMAL
5033         );
5034     }
5035 }
5036 private applyRetryButtonOuterAndIconOpacity(outerOpacity: number, iconOpacity: number): void {
5037     this.setNodeUIOpacity(this.retryButtonNode, outerOpacity);
5038     const cost = this.retryButtonNode?.getChildByName('cost');
5039     const icon = cost?.getChildByName('icon');
5040     this.setNodeUIOpacity(icon ?? null, iconOpacity);
5041 }
5042 private syncRetryButtonOpacityByAdCharges(): void {
5043     if (DataManager.instance.adRetryRemaining > 0) {
5044         this.applyRetryButtonOuterAndIconOpacity(
5045             GameUIManager.RETRY_OPACITY_NORMAL,
5046             GameUIManager.RETRY_OPACITY_NORMAL

```

```
5047         );
5048     } else {
5049         this.applyRetryButtonOuterAndIconOpacity(
5050             GameUIManager.RETRY_OUTER_NO_CHARGES,
5051             GameUIManager.RETRY_ICON_NO_CHARGES
5052         );
5053     }
5054 }
5055 private setNodeUIOpacity(target: Node | null, opacity: number): void {
5056     if (!target?.isValid) {
5057         return;
5058     }
5059     let op = target.getComponent(UIOpacity);
5060     if (!op) {
5061         op = target.addComponent(UIOpacity);
5062     }
5063     op.opacity = opacity;
5064 }
5065 updateCoinDisplay(): void {
5066     if (this.silverCoinLabel) {
5067         this.silverCoinLabel.string = DataManager.instance.silverCoin.toString();
5068     }
5069     if (this.goldCoinLabel) {
5070         this.goldCoinLabel.string = DataManager.instance.goldCoin.toString();
5071     }
5072 }
5073 private activateNodeWithAncestors(node: Node | null): void {
5074     if (!node?.isValid) {
5075         return;
5076     }
5077     const chain: Node[] = [];
5078     let c: Node | null = node;
5079     while (c) {
5080         chain.unshift(c);
5081         c = c.parent;
5082     }
5083     for (const n of chain) {
5084         if (!n.active) {
5085             n.active = true;
5086         }
5087     }
5088 }
5089 private setResultNodeActive(node: Node | null, active: boolean): void {
5090     if (!node?.isValid) {
5091         return;
5092     }
5093     if (active) {
5094         this.activateNodeWithAncestors(node);
5095     } else {
5096         node.active = false;
5097     }
5098 }
5099 private hideGameResultPanels(): void {
5100     this._winResultPanelVisible = false;
5101     this._loseResultPanelVisible = false;
5102     this.setResultNodeActive(this.gameWinResultNode, false);
5103     this.setResultNodeActive(this.gameLoseResultNode, false);
5104 }
```



```

5105     private showGameResultPanels(showWin: boolean, showLose: boolean): void {
5106         const winJustOpened = showWin && !this._winResultPanelVisible;
5107         const loseJustOpened = showLose && !this._loseResultPanelVisible;
5108         this._winResultPanelVisible = showWin;
5109         this._loseResultPanelVisible = showLose;
5110         this.setResultNodeActive(this.gameWinResultNode, showWin);
5111         this.setResultNodeActive(this.gameLoseResultNode, showLose);
5112         if (showWin) {
5113             this.updateWinRewardLabels();
5114             if (winJustOpened) {
5115                 const cfg = DataManager.instance.getResolvedLevelConfig();
5116                 if (cfg) {
5117                     this.showWinPassiveCoinBonusToasts(cfg);
5118                 }
5119             }
5120         }
5121         if (showLose) {
5122             this.updateFailPanelLayout();
5123         }
5124         if (winJustOpened) {
5125             PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.LEVEL_COMPLETE);
5126         }
5127         if (loseJustOpened) {
5128             PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.LEVEL_FAILED);
5129         }
5130     }
5131     private updateWinRewardLabels(): void {
5132         if (this.winNextLevelButtonLabel) {
5133             const lid = DataManager.instance.level;
5134             this.winNextLevelButtonLabel.string = DataManager.instance.hasLevelConfig(lid + 1)
5135                 ? '下一关'
5136                 : '返回';
5137         }
5138         const cfg = DataManager.instance.getResolvedLevelConfig();
5139         if (!cfg) {
5140             return;
5141         }
5142         const eff = GameUIManager.resolveEffectiveWinCoinRewards(cfg,
5143             DataManager.instance.equippedSkinId);
5144         this.applyWinRewardGoldSilverNodesLayout(eff.rewardGold);
5145         if (this.winRewardGoldLabel) {
5146             this.winRewardGoldLabel.string = `×${eff.rewardGold}`;
5147         }
5148         if (this.winRewardSilverLabel) {
5149             this.winRewardSilverLabel.string = `×${eff.rewardSilver}`;
5150         }
5151     }
5152     private static resolveEffectiveWinCoinRewards(cfg: LevelConfig, equippedSkinId: number): {
5153         rewardGold: number;
5154         rewardSilver: number;
5155     } {
5156         let rewardGold = cfg.rewardGold;
5157         let rewardSilver = cfg.rewardSilver;
5158         if (equippedSkinId === GameUIManager.SKIN_PASSIVE_WIN_GOLD_BONUS && rewardGold > 0) {
5159             rewardGold = Math.round(rewardGold * GameUIManager.PASSIVE_WIN_COIN_MULT);
5160         }
5161         if (equippedSkinId === GameUIManager.SKIN_PASSIVE_WIN_SILVER_BONUS && rewardSilver > 0) {
5162             rewardSilver = Math.round(rewardSilver * GameUIManager.PASSIVE_WIN_COIN_MULT);

```

```
5163     }
5164     return { rewardGold, rewardSilver };
5165 }
5166 private showWinPassiveCoinBonusToasts(cfg: LevelConfig): void {
5167     const skin = DataManager.instance.equippedSkinId;
5168     const w = GameUIManager.WIN_COIN_PASSIVE_TOAST_BG_WIDTH;
5169     const y = GameUIManager.WIN_COIN_PASSIVE_TOAST_LOCAL_Y;
5170     if (skin === GameUIManager.SKIN_PASSIVE_WIN_GOLD_BONUS && cfg.rewardGold > 0) {
5171         this.showToast('「金贝贝」增益+50%', w, y);
5172     } else if (skin === GameUIManager.SKIN_PASSIVE_WIN_SILVER_BONUS && cfg.rewardSilver > 0) {
5173         this.showToast('「银贝贝」增益+50%', w, y);
5174     }
5175 }
5176 private applyWinRewardGoldSilverNodesLayout(rewardGold: number): void {
5177     const hasGold = typeof rewardGold === 'number' && rewardGold > 0;
5178     if (this.winRewardGoldNode) {
5179         this.winRewardGoldNode.active = hasGold;
5180     }
5181     if (this.winRewardSilverNode) {
5182         const silverWidget = this.winRewardSilverNode.getComponent(Widget);
5183         if (silverWidget?.enabled) {
5184             silverWidget.enabled = false;
5185         }
5186         const x = hasGold ? GameUIManager.WIN_REWARD_SILVER_LOCAL_X_WITH_GOLD : 0;
5187         this.winRewardSilverNode.setPosition(x, 0, 0);
5188     }
5189 }
5190 private updateFailPanelLayout(): void {
5191     const solo = DataManager.instance.currentRunFromFailAdRetry;
5192     if (this.failRetryChallengeButtonNode) {
5193         this.failRetryChallengeButtonNode.active = !solo;
5194     }
5195     if (this.failReturnToMenuButtonNode) {
5196         const x = solo ? 0 : GameUIManager.FAIL_RETURN_LOCAL_X_WITH_RETRY;
5197         this.failReturnToMenuButtonNode.setPosition(x, 0, 0);
5198     }
5199 }
5200 tryWatchFailRetryAd(): boolean {
5201     return true;
5202 }
5203 private onFailReturnToMenuClick = (): void => {
5204     if (this._failSettlementConsumed) {
5205         return;
5206     }
5207     if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.FAILED) {
5208         return;
5209     }
5210     this._failSettlementConsumed = true;
5211     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5212     this.goToMainMenuScene();
5213 };
5214 private onFailRetryChallengeClick = (): void => {
5215     if (this._failSettlementConsumed) {
5216         return;
5217     }
5218     if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.FAILED) {
5219         return;
5220     }
```

```
5221     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5222     if (!this.tryWatchFailRetryAd()) {
5223         return;
5224     }
5225     this._failSettlementConsumed = true;
5226     DataManager.instance.requestKeepAdRetryChargesOnNextStartLevel();
5227     DataManager.instance.requestNextStartFromFailAdRetry();
5228     DataManager.instance.requestSkill101RefreshFromFailSettlementOnNextStartLevel();
5229     LEVEL_START.emit(EVENT_ENUM.LEVEL_START, DataManager.instance.level);
5230 };
5231 tryWatchRewardAdForDoubleWin(): boolean {
5232     return false;
5233 }
5234 private settleWinRewardsAndStartNext(multiplier: number): void {
5235     if (this._winSettlementConsumed) {
5236         return;
5237     }
5238     if (DataManager.instance.gameStatus !== GAME_STATE_ENUM.WIN) {
5239         return;
5240     }
5241     const cfg = DataManager.instance.getResolvedLevelConfig();
5242     if (!cfg) {
5243         return;
5244     }
5245     this._winSettlementConsumed = true;
5246     const      eff      =      GameUIManager.resolveEffectiveWinCoinRewards(cfg,
5247 DataManager.instance.equippedSkinId);
5248     DataManager.instance.addSilverCoin(eff.rewardSilver * multiplier);
5249     DataManager.instance.addGoldCoin(eff.rewardGold * multiplier);
5250     COIN_UPDATE.emit(EVENT_ENUM.COIN_UPDATE);
5251     const completedId = DataManager.instance.level;
5252     const nextId = completedId + 1;
5253     if (DataManager.instance.hasLevelConfig(nextId)) {
5254         DataManager.instance.level = nextId;
5255         LEVEL_START.emit(EVENT_ENUM.LEVEL_START, nextId);
5256     } else {
5257         this.goToMainMenuScene();
5258     }
5259 }
5260 private onWinNextLevelClick = (): void => {
5261     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5262     this.settleWinRewardsAndStartNext(1);
5263 };
5264 private onWinDoubleRewardAdClick = (): void => {
5265     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5266     if (!this.tryWatchRewardAdForDoubleWin()) {
5267         return;
5268     }
5269     this.settleWinRewardsAndStartNext(2);
5270 };
5271 private setMemoryHintText(text: string): void {
5272     if (this.memoryHintLabel?.isValid) {
5273         this.memoryHintLabel.string = text;
5274     }
5275 }
5276 private setMemoryWaitTextFromLevel(): void {
5277     if (DataManager.instance.isWanGuCongCongEquipped()) {
5278         this.showToast('「万古匆匆」增益+2s', 400, 450);
```

```
5279         } else if (DataManager.instance.isSuiYueShenTou207Equipped()) {
5280             this.showToast('「岁月神偷」增益+1s', 400, 450);
5281         }
5282         const dt = DataManager.instance.getEffectiveDisplayTimeSec();
5283         this.setMemoryHintText(`你还有 ${dt.toFixed(1)} 秒记忆时间`);
5284     }
5285     private clearMemoryHintSchedulers(): void {
5286         this.unschedule(this.onMemoryCountdownTick);
5287         this.unschedule(this.onShowClickHintAfterMemory);
5288     }
5289     private startMemoryCountdown(): void {
5290         if (!this.memoryHintLabel) {
5291             return;
5292         }
5293         this.clearMemoryHintSchedulers();
5294         const displayTime = DataManager.instance.getEffectiveDisplayTimeSec();
5295         this._memoryEndAtMs = performance.now() + displayTime * 1000;
5296         this.refreshMemoryCountdownString();
5297         this.schedule(this.onMemoryCountdownTick, 0.05, macro.REPEAT_FOREVER, 0);
5298     }
5299     private refreshMemoryCountdownString(): boolean {
5300         if (!this.memoryHintLabel?.isValid) {
5301             return true;
5302         }
5303         const remainSec = Math.max(0, (this._memoryEndAtMs - performance.now()) / 1000);
5304         this.memoryHintLabel.string = `你还有 ${remainSec.toFixed(1)} 秒记忆时间`;
5305         return remainSec <= 0;
5306     }
5307     private onMemoryCountdownTick = (): void => {
5308         if (!this.memoryHintLabel?.isValid) {
5309             this.unschedule(this.onMemoryCountdownTick);
5310             return;
5311         }
5312         const ended = this.refreshMemoryCountdownString();
5313         if (ended) {
5314             this.unschedule(this.onMemoryCountdownTick);
5315             this.memoryHintLabel.string = '你还有 0.0 秒记忆时间';
5316             this.scheduleOnce(this.onShowClickHintAfterMemory, 1);
5317         }
5318     };
5319     private onShowClickHintAfterMemory = (): void => {
5320         this.setMemoryHintText('请从小到大点击卡牌');
5321     };
5322     private onBackToMenuClick(): void {
5323         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5324         this.goToMainMenuScene();
5325     }
5326     private tryWatchRetryAd(): boolean {
5327         return true;
5328     }
5329     private onRetryButtonClick = (): void => {
5330         if (!this.canRetryInteract()) {
5331             return;
5332         }
5333         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5334         if (!this.tryWatchRetryAd()) {
5335             return;
5336         }
5337     }
```

```

5337     DataManager.instance.consumeAdRetryCharge();
5338     this.updateRetryAdCountLabel();
5339     this.updateRetryButtonInteractable();
5340     DataManager.instance.requestKeepAdRetryChargesOnNextStartLevel();
5341     DataManager.instance.requestPreserveSkill101ConsumptionOnNextStartLevel();
5342     if (DataManager.instance.currentRunFromFailAdRetry) {
5343         DataManager.instance.requestNextStartFromFailAdRetry();
5344     }
5345     LEVEL_START.emit(EVENT_ENUM.LEVEL_START, DataManager.instance.level);
5346 };
5347 private findGameManager(): GameManager | null {
5348     const scene = director.getScene();
5349     return scene?.getComponentInChildren(GameManager) ?? null;
5350 }
5351 private tryExecuteEquippedActiveSkill(): boolean {
5352     const skinId = DataManager.instance.equippedSkinId;
5353     if (skinId === 101) {
5354         if (DataManager.instance.skill101ConsumedThisLevelCycle) {
5355             return false;
5356         }
5357         DataManager.instance.skill101ConsumedThisLevelCycle = true;
5358         this.showToast('「再来一局」技能发动', 400);
5359         DataManager.instance.requestKeepAdRetryChargesOnNextStartLevel();
5360         DataManager.instance.requestPreserveSkill101ConsumptionOnNextStartLevel();
5361         this.unschedule(this.skill101RestartCallback);
5362         this.scheduleOnce(this.skill101RestartCallback, GameUIManager.SKILL_POST_TOAST_DELAY_SEC);
5363         return true;
5364     }
5365     if (skinId === 401) {
5366         const gm = this.findGameManager();
5367         if (!gm?.canActivateChunQiuChanSkill()) {
5368             return false;
5369         }
5370         this.showToast('「春秋蝉」技能发动', 360);
5371         this.unschedule(this.skill401ChunQiuChanCallback);
5372         this.scheduleOnce(this.skill401ChunQiuChanCallback,
5373 GameUIManager.SKILL_POST_TOAST_DELAY_SEC);
5374         return true;
5375     }
5376     if (skinId === 301) {
5377         DataManager.instance.activeSkillCardInterceptMode =
5378 ACTIVE_SKILL_CARD_INTERCEPT_MODE.VERIFY_CARD_301;
5379         this.showToast('请点击查验卡牌', 350);
5380         return true;
5381     }
5382     if (skinId === 103) {
5383         DataManager.instance.activeSkillCardInterceptMode =
5384 ACTIVE_SKILL_CARD_INTERCEPT_MODE.PROPHET_103;
5385         this.showToast('请点击查验卡牌', 350);
5386         return true;
5387     }
5388     if (skinId === 302) {
5389         const gm = this.findGameManager();
5390         if (!gm?.canSimulateNextCorrectCardClick()) {
5391             return false;
5392         }
5393         this.showToast('「序列指引」技能发动', 400);
5394         this.unschedule(this.skill302FlipCallback);

```

```
5395         this.scheduleOnce(this.skill302FlipCallback, GameUIManager.SKILL_POST_TOAST_DELAY_SEC);
5396         return true;
5397     }
5398     if (skinId === 102) {
5399         if (!DataManager.instance.cards.some(c => c.number === 0)) {
5400             this.showToast('当前无「」牌', 340);
5401             return false;
5402         }
5403         const gm = this.findGameManager();
5404         if (!gm?.canRevealBombsForBombRevealSkill()) {
5405             return false;
5406         }
5407         this.showToast('「拆弹能手」技能发动', 400);
5408         this.unschedule(this.skill102RevealCallback);
5409         this.scheduleOnce(this.skill102RevealCallback, GameUIManager.SKILL_POST_TOAST_DELAY_SEC);
5410         return true;
5411     }
5412     if (skinId === 202) {
5413         if (!DataManager.instance.cards.some(c => c.number === 0)) {
5414             this.showToast('当前无「」牌', 340);
5415             return false;
5416         }
5417         const gm = this.findGameManager();
5418         if (!gm?.canRevealBombsForBombRevealSkill()) {
5419             return false;
5420         }
5421         this.showToast('「拆弹专家」技能发动', 400);
5422         this.unschedule(this.skill202RevealCallback);
5423         this.scheduleOnce(this.skill202RevealCallback, GameUIManager.SKILL_POST_TOAST_DELAY_SEC);
5424         return true;
5425     }
5426     if (skinId === 203) {
5427         const gm = this.findGameManager();
5428         if (!gm?.canSimulateNextCorrectCardClick()) {
5429             return false;
5430         }
5431         this.showToast('「言出法随」技能发动', 400);
5432         DataManager.instance.skill203NextClickEnabled = true;
5433         return true;
5434     }
5435     return false;
5436 }
5437 private onActiveSkillVerify301CardTap(cardId: number): void {
5438     const card = DataManager.instance.getCard(cardId);
5439     if (!card) {
5440         return;
5441     }
5442     if (card.number === 0) {
5443         this.showToast('该卡牌为「」牌', 360);
5444     } else {
5445         this.showToast(`该卡牌为数字牌「${card.number}」`, 400);
5446     }
5447 }
5448 private onActiveSkillProphet103CardTap(cardId: number): void {
5449     const card = DataManager.instance.getCard(cardId);
5450     if (!card) {
5451         return;
5452     }
```

```

5453         if (card.number === 0) {
5454             this.showToast('该卡牌为「」牌', 360);
5455         } else {
5456             this.showToast('该卡牌为「数字」牌', 360);
5457         }
5458     }
5459     private onSkillButtonClick = (): void => {
5460         if (!this.canSkillInteract()) {
5461             return;
5462         }
5463         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5464         const cost = this.getResolvedSkillCost();
5465         if (!this.tryExecuteEquippedActiveSkill()) {
5466             return;
5467         }
5468         DataManager.instance.activeSkillConsumedThisRun = true;
5469         if (cost > 0) {
5470             DataManager.instance.consumeSilverCoin(cost);
5471         }
5472         COIN_UPDATE.emit(EVENT_ENUM.COIN_UPDATE);
5473         UI_UPDATE.emit(EVENT_ENUM.UI_UPDATE);
5474     };
5475     private goToMainMenuScene(): void {
5476         director.loadScene(SCENE_ENUM.MENU, () => {
5477             SCENE_CHANGE.emit(EVENT_ENUM.SCENE_CHANGE, SCENE_ENUM.MENU);
5478         });
5479     }
5480 }
5481 ...
5482 ---
5483 ## 文件: `Manager/MenuManager.ts`
5484 ```typescript
5485 import { _decorator, Component, Node, director, Label } from 'cc';
5486 import { SCENE_ENUM, EVENT_ENUM, AUDIO_EFFECT_ENUM } from '../Utils/Enum';
5487 import { COIN_UPDATE, SCENE_CHANGE, LEVEL_START, PLAY_AUDIO } from '../Utils/Event';
5488 import { AlbumWindow } from '../AlbumWindow';
5489 import { BannerDrawWindow } from '../BannerDrawWindow';
5490 import { DataManager } from '../DataManager';
5491 import { ToastManager } from '../ToastManager';
5492 const { ccclass, property } = _decorator;
5493 export interface MenuBarData {
5494     goldCoin: number;
5495     silverCoin: number;
5496     equippedSkinId: number;
5497 }
5498 @ccclass('MenuManager')
5499 export class MenuManager extends Component {
5500     @property(Node)
5501     startButtonNode: Node = null;
5502     @property(Label)
5503     goldCoinLabel: Label = null;
5504     @property(Label)
5505     silverCoinLabel: Label = null;
5506     @property(Node)
5507     albumButtonNode: Node = null;
5508     @property(Node)
5509     albumWindowNode: Node = null;
5510     @property(Node)

```

```
5511     bannerButtonNode: Node = null;
5512     @property(Node)
5513     bannerWindowNode: Node = null;
5514     @property(Node)
5515     dailyButtonNode: Node = null;
5516     @property(Node)
5517     dailyWindowNode: Node = null;
5518     @property(Node)
5519     aboutButtonNode: Node = null;
5520     @property(Node)
5521     aboutWindowNode: Node = null;
5522     @property(Node)
5523     toastManagerNode: Node = null;
5524     protected onLoad(): void {
5525         director.preloadScene(SCENE_ENUM.GAME);
5526         DataManager.instance.init();
5527         this.refreshMenuBarFromData();
5528         this.hoistMenuToastToCanvas();
5529         if (this.albumWindowNode) {
5530             this.albumWindowNode.active = false;
5531         }
5532         if (this.bannerWindowNode) {
5533             this.bannerWindowNode.active = false;
5534         }
5535         if (this.dailyWindowNode) {
5536             this.dailyWindowNode.active = false;
5537         }
5538         if (this.aboutWindowNode) {
5539             this.aboutWindowNode.active = false;
5540         }
5541         COIN_UPDATE.on(EVENT_ENUM.COIN_UPDATE, this.refreshMenuBarFromData, this);
5542         if (this.startButtonNode) {
5543             this.startButtonNode.on(Node.EventType.TOUCH_END, this.onStartButtonClick, this);
5544         }
5545         if (this.albumButtonNode) {
5546             this.albumButtonNode.on(Node.EventType.TOUCH_END, this.onAlbumButtonClick, this);
5547         }
5548         if (this.bannerButtonNode) {
5549             this.bannerButtonNode.on(Node.EventType.TOUCH_END, this.onBannerButtonClick, this);
5550         }
5551         if (this.dailyButtonNode) {
5552             this.dailyButtonNode.on(Node.EventType.TOUCH_END, this.onDailyButtonClick, this);
5553         }
5554         if (this.aboutButtonNode) {
5555             this.aboutButtonNode.on(Node.EventType.TOUCH_END, this.onAboutButtonClick, this);
5556         }
5557     }
5558     refreshMenuBarFromData(): void {
5559         const data = this.readMenuBarData();
5560         if (this.goldCoinLabel) {
5561             this.goldCoinLabel.string = String(data.goldCoin);
5562         }
5563         if (this.silverCoinLabel) {
5564             this.silverCoinLabel.string = String(data.silverCoin);
5565         }
5566     }
5567     readMenuBarData(): MenuBarData {
5568         return {
```



```
5569         goldCoin: DataManager.instance.goldCoin,
5570         silverCoin: DataManager.instance.silverCoin,
5571         equippedSkinId: DataManager.instance.equippedSkinId,
5572     };
5573 }
5574 protected onDestroy(): void {
5575     COIN_UPDATE.off(EVENT_ENUM.COIN_UPDATE, this.refreshMenuBarFromData, this);
5576     if (this.startButtonNode && this.startButtonNode.isValid) {
5577         this.startButtonNode.off(Node.EventType.TOUCH_END, this.onStartButtonClick, this);
5578     }
5579     if (this.albumButtonNode && this.albumButtonNode.isValid) {
5580         this.albumButtonNode.off(Node.EventType.TOUCH_END, this.onAlbumButtonClick, this);
5581     }
5582     if (this.bannerButtonNode && this.bannerButtonNode.isValid) {
5583         this.bannerButtonNode.off(Node.EventType.TOUCH_END, this.onBannerButtonClick, this);
5584     }
5585     if (this.dailyButtonNode && this.dailyButtonNode.isValid) {
5586         this.dailyButtonNode.off(Node.EventType.TOUCH_END, this.onDailyButtonClick, this);
5587     }
5588     if (this.aboutButtonNode && this.aboutButtonNode.isValid) {
5589         this.aboutButtonNode.off(Node.EventType.TOUCH_END, this.onAboutButtonClick, this);
5590     }
5591 }
5592 onStartButtonClick(): void {
5593     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5594     this.startGame();
5595 }
5596 onAlbumButtonClick(): void {
5597     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5598     if (this.albumWindowNode) {
5599         this.albumWindowNode.active = true;
5600         this.albumWindowNode.getComponent(AlbumWindow)?.resetToFirstPage();
5601     }
5602 }
5603 onBannerButtonClick(): void {
5604     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5605     this.hideBannerDailyAboutWindows();
5606     if (this.bannerWindowNode) {
5607         this.bannerWindowNode.active = true;
5608         this.bannerWindowNode.getComponent(BannerDrawWindow)?.onBannerShown();
5609     }
5610 }
5611 onDailyButtonClick(): void {
5612     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5613     this.hideBannerDailyAboutWindows();
5614     const today = MenuManager.todayLocalYmd();
5615     const last = DataManager.instance.lastdaily;
5616     const toast = this.toastManagerNode?.getComponent(ToastManager) ?? null;
5617     if (!last || last !== today) {
5618         DataManager.instance.claimDailyMenuReward(today);
5619         COIN_UPDATE.emit(EVENT_ENUM.COIN_UPDATE);
5620         toast?.show('恭喜获得「金贝贝」×10', 450, 510);
5621         this.scheduleOnce(() => {
5622             if (!this.isValid) {
5623                 return;
5624             }
5625             this.toastManagerNode?.getComponent(ToastManager)?.show('恭喜获得「银贝贝」×200',
5626                 450, 510);
```

```
5627         }, 1.5);
5628     } else {
5629         toast?.show('今日已领取，明天再来吧~', 480, 510);
5630     }
5631     if (this.dailyWindowNode) {
5632         this.dailyWindowNode.active = true;
5633     }
5634 }
5635 onAboutButtonClick(): void {
5636     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5637     this.hideBannerDailyAboutWindows();
5638     if (this.aboutWindowNode) {
5639         this.aboutWindowNode.active = true;
5640     }
5641 }
5642 private hoistMenuToastToCanvas(): void {
5643     const root = this.toastManagerNode;
5644     if (!root?.isValid) {
5645         return;
5646     }
5647     const canvas = this.node;
5648     if (root.parent !== canvas) {
5649         root.setParent(canvas, false);
5650         root.setPosition(0, 0, 0);
5651     }
5652     root.setSiblingIndex(Math.max(0, canvas.children.length - 1));
5653 }
5654 private static todayLocalYmd(): string {
5655     const d = new Date();
5656     const y = d.getFullYear();
5657     const m = String(d.getMonth() + 1).padStart(2, '0');
5658     const day = String(d.getDate()).padStart(2, '0');
5659     return `${y}-${m}-${day}`;
5660 }
5661 private hideBannerDailyAboutWindows(): void {
5662     if (this.bannerWindowNode) {
5663         this.bannerWindowNode.active = false;
5664     }
5665     if (this.dailyWindowNode) {
5666         this.dailyWindowNode.active = false;
5667     }
5668     if (this.aboutWindowNode) {
5669         this.aboutWindowNode.active = false;
5670     }
5671 }
5672 startGame(): void {
5673     director.loadScene(SCENE_ENUM.GAME, () => {
5674         SCENE_CHANGE.emit(EVENT_ENUM.SCENE_CHANGE, SCENE_ENUM.GAME);
5675         DataManager.instance.level = 1;
5676         LEVEL_START.emit(EVENT_ENUM.LEVEL_START, 1);
5677     });
5678 }
5679 }
5680 ```
5681 ---
5682 ## 文件: `Manager/MusicManager.ts`
5683 ```typescript
5684 import { _decorator, Component, AudioClip, AudioSource, director, Director } from 'cc';
```

```
5685 import { EVENT_ENUM, AUDIO_EFFECT_ENUM, SCENE_ENUM } from '../Utils/Enum';
5686 import { PLAY_AUDIO, PLAY_BGM } from '../Utils/Event';
5687 const { ccclass, property } = _decorator;
5688 @ccclass('MusicManager')
5689 export class MusicManager extends Component {
5690     @property(AudioClip)
5691     clickButton: AudioClip = null;
5692     @property(AudioClip)
5693     cardClickRight: AudioClip = null;
5694     @property(AudioClip)
5695     cardClickWrong: AudioClip = null;
5696     @property(AudioClip)
5697     levelComplete: AudioClip = null;
5698     @property(AudioClip)
5699     levelFailed: AudioClip = null;
5700     @property(AudioClip)
5701     skillUse: AudioClip = null;
5702     @property(AudioClip)
5703     bombExplode: AudioClip = null;
5704     @property(AudioClip)
5705     bgmMenu: AudioClip = null;
5706     @property(AudioClip)
5707     bgmGame: AudioClip = null;
5708     @property(AudioSource)
5709     audioSource: AudioSource = null;
5710     protected onLoad(): void {
5711         director.addPersistRootNode(this.node);
5712         director.on(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
5713         PLAY_AUDIO.on(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
5714         PLAY_BGM.on(EVENT_ENUM.PLAY_BGM, this.onBGMPay, this);
5715     }
5716     protected onDestroy(): void {
5717         director.off(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
5718         PLAY_AUDIO.off(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
5719         PLAY_BGM.off(EVENT_ENUM.PLAY_BGM, this.onBGMPay, this);
5720     }
5721     onAfterSceneLaunch(): void {
5722         const sceneName = director.getScene()?.name;
5723         this.updateBackGroundMusic(sceneName);
5724     }
5725     onAudioPlay(type: AUDIO_EFFECT_ENUM): void {
5726         let clip: AudioClip = null;
5727         switch (type) {
5728             case AUDIO_EFFECT_ENUM.CLICK_BUTTON:
5729                 clip = this.clickButton;
5730                 break;
5731             case AUDIO_EFFECT_ENUM.CARD_CLICK_RIGHT:
5732                 clip = this.cardClickRight;
5733                 break;
5734             case AUDIO_EFFECT_ENUM.CARD_CLICK_WRONG:
5735                 clip = this.cardClickWrong;
5736                 break;
5737             case AUDIO_EFFECT_ENUM.LEVEL_COMPLETE:
5738                 clip = this.levelComplete;
5739                 break;
5740             case AUDIO_EFFECT_ENUM.LEVEL_FAILED:
5741                 clip = this.levelFailed;
5742                 break;
```

```

5743         case AUDIO_EFFECT_ENUM.SKILL_USE:
5744             clip = this.skillUse;
5745             break;
5746         case AUDIO_EFFECT_ENUM.BOMB_EXPLODE:
5747             clip = this.bombExplode;
5748             break;
5749     }
5750     if (clip && this.audioSource) {
5751         this.audioSource.playOneShot(clip, 1);
5752     }
5753 }
5754 onBGMPay(sceneName: string): void {
5755     this.updateBackGroundMusic(sceneName);
5756 }
5757 updateBackGroundMusic(sceneName: string): void {
5758     if (!this.audioSource) {
5759         return;
5760     }
5761     this.audioSource.stop();
5762     let clip: AudioClip = null;
5763     switch (sceneName) {
5764         case SCENE_ENUM.MENU:
5765             clip = this.bgmMenu;
5766             break;
5767         case SCENE_ENUM.GAME:
5768             clip = this.bgmGame;
5769             break;
5770     }
5771     if (clip) {
5772         this.audioSource.clip = clip;
5773         this.audioSource.loop = true;
5774         this.audioSource.play();
5775     }
5776 }
5777 }
5778 ```
5779 ---
5780 ## 文件: `Manager/SkillManager.ts`
5781 ```typescript
5782 import { _decorator, Component } from 'cc';
5783 import { ACTIVE_SKILL_ENUM, PASSIVE_SKILL_ENUM, EVENT_ENUM } from '../Utils/Enum';
5784 import { SKILL_USE, SKILL_ACTIVE_USE } from '../Utils/Event';
5785 import { SkillUseData } from '../Config/GameTypes';
5786 const { ccclass } = _decorator;
5787 @ccclass('SkillManager')
5788 export class SkillManager extends Component {
5789     private usedActiveSkills: ACTIVE_SKILL_ENUM[] = [];
5790     protected onLoad(): void {
5791         SKILL_USE.on(EVENT_ENUM.SKILL_USE, this.onSkillUse, this);
5792         SKILL_ACTIVE_USE.on(EVENT_ENUM.SKILL_ACTIVE_USE, this.onActiveSkillUse, this);
5793     }
5794     protected onDestroy(): void {
5795         SKILL_USE.off(EVENT_ENUM.SKILL_USE, this.onSkillUse, this);
5796         SKILL_ACTIVE_USE.off(EVENT_ENUM.SKILL_ACTIVE_USE, this.onActiveSkillUse, this);
5797     }
5798     onSkillUse(data: SkillUseData): void {
5799     }
5800     onActiveSkillUse(skillType: ACTIVE_SKILL_ENUM, data?: any): void {

```

```
5801         if (this.isActiveSkillUsed(skillType)) {
5802             console.warn('主动技能已使用:', skillType);
5803             return;
5804         }
5805         this.useActiveSkill(skillType, data);
5806     }
5807     isActiveSkillUsed(skillType: ACTIVE_SKILL_ENUM): boolean {
5808         return this.usedActiveSkills.includes(skillType);
5809     }
5810     useActiveSkill(skillType: ACTIVE_SKILL_ENUM, data?: any): void {
5811         if (this.isActiveSkillUsed(skillType)) {
5812             return;
5813         }
5814         this.usedActiveSkills.push(skillType);
5815         switch (skillType) {
5816             case ACTIVE_SKILL_ENUM.CHECK_CARD:
5817                 this.skillCheckCard(data?.cardId);
5818                 break;
5819             case ACTIVE_SKILL_ENUM.TIME_MACHINE:
5820                 this.skillTimeMachine();
5821                 break;
5822             case ACTIVE_SKILL_ENUM.RESTART_LEVEL:
5823                 this.skillRestartLevel();
5824                 break;
5825         }
5826     }
5827     skillCheckCard(cardId?: number): void {
5828         console.log('使用技能：我要验牌', cardId);
5829     }
5830     skillTimeMachine(): void {
5831         console.log('使用技能：时光机器');
5832     }
5833     skillRestartLevel(): void {
5834         console.log('使用技能：再来一局');
5835     }
5836     getPassiveSkillEffect(skillType: PASSIVE_SKILL_ENUM): number {
5837         switch (skillType) {
5838             case PASSIVE_SKILL_ENUM.TIME_THIEF:
5839                 return 1.0;
5840             case PASSIVE_SKILL_ENUM.SILVER_BONUS:
5841                 return 0.2;
5842             case PASSIVE_SKILL_ENUM.GOLD_BONUS:
5843                 return 0.2;
5844             default:
5845                 return 0;
5846         }
5847     }
5848     applyTimeThief(baseTime: number): number {
5849         const bonus = this.getPassiveSkillEffect(PASSIVE_SKILL_ENUM.TIME_THIEF);
5850         return baseTime + bonus;
5851     }
5852     applySilverBonus(baseReward: number): number {
5853         const bonus = this.getPassiveSkillEffect(PASSIVE_SKILL_ENUM.SILVER_BONUS);
5854         return Math.floor(baseReward * (1 + bonus));
5855     }
5856     applyGoldBonus(baseReward: number): number {
5857         const bonus = this.getPassiveSkillEffect(PASSIVE_SKILL_ENUM.GOLD_BONUS);
5858         return Math.floor(baseReward * (1 + bonus));
```

```
5859     }
5860     resetSkillState(): void {
5861         this.usedActiveSkills = [];
5862     }
5863 }
5864 ...
5865 ---
5866 ## 文件: `Manager/ToastManager.ts`
5867 ```typescript
5868 import { _decorator, Component, Prefab, instantiate, Node, Label, UITransform, UIOpacity, tween, Tween } from
5869 'cc';
5870 import { RoundBox } from '../RoundBox';
5871 const { ccclass, property } = _decorator;
5872 @ccclass('ToastManager')
5873 export class ToastManager extends Component {
5874     @property(Prefab)
5875     toastPrefab: Prefab = null;
5876     private static readonly DISPLAY_SEC = 1;
5877     private static readonly FADE_SEC = 1;
5878     private static readonly DEFAULT_BG_WIDTH = 320;
5879     private _toastRoot: Node | null = null;
5880     private static resolveBgWidth(bgWidth: number | undefined): number {
5881         if (typeof bgWidth !== 'number' || !Number.isFinite(bgWidth)) {
5882             return ToastManager.DEFAULT_BG_WIDTH;
5883         }
5884         return Math.max(0, bgWidth);
5885     }
5886     private static resolveLocalY(localY: number | undefined): number {
5887         if (typeof localY !== 'number' || !Number.isFinite(localY)) {
5888             return 0;
5889         }
5890         return localY;
5891     }
5892     show(message: string, bgWidth?: number, localY: number = 0): void {
5893         if (!this.toastPrefab) {
5894             console.warn('[ToastManager] 未绑定 toastPrefab');
5895             return;
5896         }
5897         if (!this._toastRoot || !this._toastRoot.isValid) {
5898             this._toastRoot = instantiate(this.toastPrefab);
5899             this._toastRoot.setParent(this.node);
5900         }
5901         const root = this._toastRoot;
5902         const py = ToastManager.resolveLocalY(localY);
5903         root.setPosition(0, py, 0);
5904         const uiOp = root.getComponent(UIOpacity) ?? root.addComponent(UIOpacity);
5905         Tween.stopAllByTarget(uiOp);
5906         const bgNode = root.getChildByName('bg');
5907         const bgUt = bgNode?.getComponent(UITransform);
5908         const wResolved = ToastManager.resolveBgWidth(bgWidth);
5909         if (bgUt) {
5910             bgUt.width = wResolved;
5911             const rb = bgNode?.getComponent(RoundBox);
5912             if (rb?.isValid) {
5913                 rb.forceRefreshRender();
5914                 this.scheduleOnce(() => {
5915                     if (rb.isValid && bgUt.isValid) {
5916                         rb.forceRefreshRender();
```

```

5917         }
5918         }, 0);
5919     }
5920 } else {
5921     console.warn('[ToastManager] 预制体根下未找到子节点 bg 或未挂 UITransform');
5922 }
5923 const textNode = root.getChildByName('text');
5924 const label = textNode?.getComponent(Label);
5925 if (label) {
5926     label.string = message;
5927 } else {
5928     console.warn('[ToastManager] 预制体根下未找到子节点 text 或未挂 Label');
5929 }
5930 root.active = true;
5931 uiOp.opacity = 255;
5932 tween(uiOp)
5933     .delay(ToastManager.DISPLAY_SEC)
5934     .to(ToastManager.FADE_SEC, { opacity: 0 })
5935     .call(() => {
5936         this.scheduleOnce(() => {
5937             if (root?.isValid) {
5938                 root.active = false;
5939             }
5940         }, 0);
5941     })
5942     .start();
5943 }
5944 }
5945 ```
5946 ---
5947 ## 文件: `MenuOverlayWindow.ts`
5948 ```typescript
5949 import { _decorator, Component, Node } from 'cc';
5950 import { RoundBox } from '../RoundBox';
5951 import { EVENT_ENUM, AUDIO_EFFECT_ENUM } from './Utils/Enum';
5952 import { PLAY_AUDIO } from './Utils/Event';
5953 const { ccclass, property } = _decorator;
5954 @ccclass('MenuOverlayWindow')
5955 export class MenuOverlayWindow extends Component {
5956     @property(Node)
5957     closeButtonNode: Node = null;
5958     private _backdropBgNode: Node | null = null;
5959     protected onLoad(): void {
5960         this._backdropBgNode = this.node.getChildByName('bg') ?? null;
5961         if (this._backdropBgNode) {
5962             this._backdropBgNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
5963         }
5964         if (this.closeButtonNode) {
5965             this.closeButtonNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
5966         }
5967     }
5968     protected onEnable(): void {
5969         this.scheduleOnce(this.refreshChildRoundBoxes, 0);
5970     }
5971     private refreshChildRoundBoxes(): void {
5972         if (!this.node?.isValid || !this.node.activeInHierarchy) {
5973             return;
5974         }

```

```
5975         const list = this.node.getComponentsInChildren(RoundBox);
5976         for (let i = 0; i < list.length; i++) {
5977             list[i].forceRefreshRender();
5978         }
5979     }
5980     protected onDestroy(): void {
5981         if (this._backdropBgNode?.isValid) {
5982             this._backdropBgNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
5983         }
5984         this._backdropBgNode = null;
5985         if (this.closeButtonNode?.isValid) {
5986             this.closeButtonNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
5987         }
5988     }
5989     private onCloseClick(): void {
5990         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
5991         this.node.active = false;
5992     }
5993 }
5994 ```
5995 ---
5996 ## 文件: `Utils/Enum.ts`
5997 ```typescript
5998 export enum GAME_STATE_ENUM {
5999     INIT,          // 初始化
6000     PREPARE,       // 准备阶段
6001     DISPLAY,       // 展示阶段
6002     HIDDEN,        // 隐藏阶段
6003     CLICKING,      // 点击阶段
6004     PAUSED,        // 暂停
6005     WIN,           // 胜利
6006     FAILED,        // 失败
6007 }
6008 export enum SCENE_ENUM {
6009     MENU = 'Menu',
6010     GAME = 'Game',
6011 }
6012 export enum EVENT_ENUM {
6013     GAME_STATE_CHANGE = 'GAME_STATE_CHANGE',
6014     GAME_START = 'GAME_START',
6015     GAME_PAUSE = 'GAME_PAUSE',
6016     GAME_RESUME = 'GAME_RESUME',
6017     GAME_WIN = 'GAME_WIN',
6018     GAME_FAILED = 'GAME_FAILED',
6019     GAME_RESTART = 'GAME_RESTART',
6020     CARD_CLICK = 'CARD_CLICK',
6021     CARD_DISPLAY = 'CARD_DISPLAY',
6022     CARD_HIDE = 'CARD_HIDE',
6023     CARD_SWAP = 'CARD_SWAP',
6024     LEVEL_START = 'LEVEL_START',
6025     LEVEL_COMPLETE = 'LEVEL_COMPLETE',
6026     LEVEL_FAILED = 'LEVEL_FAILED',
6027     LEVEL_CHANGE = 'LEVEL_CHANGE',
6028     SKILL_USE = 'SKILL_USE',
6029     SKILL_ACTIVE_USE = 'SKILL_ACTIVE_USE',
6030     ACTIVE_SKILL_VERIFY_CARD_TAP = 'ACTIVE_SKILL_VERIFY_CARD_TAP',
6031     ACTIVE_SKILL_PROPHET_CARD_TAP = 'ACTIVE_SKILL_PROPHET_CARD_TAP',
6032     PASSIVE_304_FENG_HUANG_GU_TRIGGER = 'PASSIVE_304_FENG_HUANG_GU_TRIGGER',
```



```

6033     PASSIVE_204_YING_SHI_ER_DU_TRIGGER = 'PASSIVE_204_YING_SHI_ER_DU_TRIGGER',
6034     PLAY_AUDIO = 'PLAY_AUDIO',
6035     PLAY_BGM = 'PLAY_BGM',
6036     UI_UPDATE = 'UI_UPDATE',
6037     COIN_UPDATE = 'COIN_UPDATE',
6038     EQUIPPED_SKIN_CHANGE = 'EQUIPPED_SKIN_CHANGE',
6039     SCENE_CHANGE = 'SCENE_CHANGE',
6040 }
6041 export enum AUDIO_EFFECT_ENUM {
6042     CLICK_BUTTON = 'CLICK_BUTTON',
6043     CARD_CLICK_RIGHT = 'CARD_CLICK_RIGHT',
6044     CARD_CLICK_WRONG = 'CARD_CLICK_WRONG',
6045     LEVEL_COMPLETE = 'LEVEL_COMPLETE',
6046     LEVEL_FAILED = 'LEVEL_FAILED',
6047     SKILL_USE = 'SKILL_USE',
6048     BOMB_EXPLODE = 'BOMB_EXPLODE',
6049 }
6050 export enum CARD_TYPE_ENUM {
6051     NUMBER = 'NUMBER',    // 数字卡片
6052     BOMB = 'BOMB',        // 炸弹卡片
6053 }
6054 export enum CARD_STATE_ENUM {
6055     DISPLAY = 'DISPLAY',  // 展示状态
6056     HIDDEN = 'HIDDEN',   // 隐藏状态
6057     CLICKED = 'CLICKED', // 已点击状态
6058 }
6059 export enum SKIN_RARITY_ENUM {
6060     BLUE = 'BLUE',        // 蓝色（70%）
6061     PURPLE = 'PURPLE',    // 紫色（25%）
6062     RED = 'RED',          // 红色（5%）
6063 }
6064 export enum ACTIVE_SKILL_CARD_INTERCEPT_MODE {
6065     NONE = 0,
6066     VERIFY_CARD_301 = 1,
6067     PROPHET_103 = 2,
6068 }
6069 export enum ACTIVE_SKILL_ENUM {
6070     CHECK_CARD = 'CHECK_CARD',    // 我要验牌
6071     TIME_MACHINE = 'TIME_MACHINE', // 时光机器
6072     RESTART_LEVEL = 'RESTART_LEVEL', // 再来一局
6073 }
6074 export enum PASSIVE_SKILL_ENUM {
6075     TIME_THIEF = 'TIME_THIEF',    // 岁月神偷
6076     SILVER_BONUS = 'SILVER_BONUS', // 银贝贝
6077     GOLD_BONUS = 'GOLD_BONUS',     // 金贝贝
6078 }
6079 ```
6080 ---
6081 ``` ## 文件: `Utils/Event.ts`
6082 ```typescript
6083 import { EventTarget } from "cc";
6084 import { EVENT_ENUM } from "../Enum";
6085 export const GAME_STATE_CHANGE = new EventTarget();
6086 export const GAME_START = new EventTarget();
6087 export const GAME_PAUSE = new EventTarget();
6088 export const GAME_RESUME = new EventTarget();
6089 export const GAME_WIN = new EventTarget();
6090 export const GAME_FAILED = new EventTarget();

```

```
6091 export const GAME_RESTART = new EventTarget();
6092 export const CARD_CLICK = new EventTarget();
6093 export const CARD_DISPLAY = new EventTarget();
6094 export const CARD_HIDE = new EventTarget();
6095 export const CARD_SWAP = new EventTarget();
6096 export const LEVEL_START = new EventTarget();
6097 export const LEVEL_COMPLETE = new EventTarget();
6098 export const LEVEL_FAILED = new EventTarget();
6099 export const LEVEL_CHANGE = new EventTarget();
6100 export const SKILL_USE = new EventTarget();
6101 export const SKILL_ACTIVE_USE = new EventTarget();
6102 export const ACTIVE_SKILL_VERIFY_CARD_TAP = new EventTarget();
6103 export const ACTIVE_SKILL_PROPHET_CARD_TAP = new EventTarget();
6104 export const PASSIVE_304_FENG_HUANG_GU_TRIGGER = new EventTarget();
6105 export const PASSIVE_204_YING_SHI_ER_DU_TRIGGER = new EventTarget();
6106 export const PLAY_AUDIO = new EventTarget();
6107 export const PLAY_BGM = new EventTarget();
6108 export const UI_UPDATE = new EventTarget();
6109 export const COIN_UPDATE = new EventTarget();
6110 export const EQUIPPED_SKIN_CHANGE = new EventTarget();
6111 export const SCENE_CHANGE = new EventTarget();
6112 ```
6113 ---
6114 ## 文件: `Utils/Utils.ts`
6115 ```typescript
6116 import { Node, Sprite, SpriteFrame, UIOpacity } from 'cc';
6117 import { RoundBox } from '../RoundBox';
6118 export function setNodeUIOpacity(target: Node | null | undefined, opacity: number): void {
6119     if (!target?.isValid) {
6120         return;
6121     }
6122     const o = Math.max(0, Math.min(255, Math.round(opacity)));
6123     let op = target.getComponent(UIOpacity);
6124     if (!op) {
6125         op = target.addComponent(UIOpacity);
6126     }
6127     op.opacity = o;
6128 }
6129 export function shuffle<T>(arr: T[]): T[] {
6130     const result = [...arr];
6131     for (let i = result.length - 1; i > 0; i--) {
6132         const j = Math.floor(Math.random() * (i + 1));
6133         [result[i], result[j]] = [result[j], result[i]];
6134     }
6135     return result;
6136 }
6137 export function randomInt(min: number, max: number): number {
6138     return Math.floor(Math.random() * (max - min + 1)) + min;
6139 }
6140 export function generateUniqueRandomNumbers(count: number, min: number, max: number): number[] {
6141     if (count > (max - min + 1)) {
6142         console.error('数量超过范围，无法生成不重复的随机数');
6143         return [];
6144     }
6145     const allNumbers: number[] = [];
6146     for (let i = min; i <= max; i++) {
6147         allNumbers.push(i);
6148     }
```

```
6149     return shuffle(allNumbers).slice(0, count);
6150 }
6151 export function calculateGridSize(totalCount: number): { rows: number; cols: number } {
6152     if (totalCount <= 9) {
6153         return { rows: 3, cols: 3 };
6154     }
6155     if (totalCount <= 16) {
6156         return { rows: 4, cols: 4 };
6157     }
6158     const cols = Math.ceil(Math.sqrt(totalCount));
6159     const rows = Math.ceil(totalCount / cols);
6160     return { rows, cols };
6161 }
6162 export function delay(ms: number): Promise<void> {
6163     return new Promise(resolve => setTimeout(resolve, ms));
6164 }
6165 export function formatTime(seconds: number): string {
6166     const mins = Math.floor(seconds / 60);
6167     const secs = Math.floor(seconds % 60);
6168     return `${mins.toString().padStart(2, '0')}:${secs.toString().padStart(2, '0')}`;
6169 }
6170 export function applySpriteFrameToUINode(target: Node | null | undefined, sf: SpriteFrame | null | undefined):
6171 void {
6172     if (!target || !sf) {
6173         return;
6174     }
6175     const rb = target.getComponent(RoundBox);
6176     if (rb) {
6177         rb.setDisplaySpriteFrame(sf);
6178         return;
6179     }
6180     const spr = target.getComponent(Sprite);
6181     if (spr) {
6182         spr.spriteFrame = sf;
6183     }
6184 }
6185 export function clearSpriteFrameOnUINode(target: Node | null | undefined): void {
6186     if (!target?.isValid) {
6187         return;
6188     }
6189     const rb = target.getComponent(RoundBox);
6190     if (rb) {
6191         rb.setDisplaySpriteFrame(null);
6192     }
6193     const spr = target.getComponent(Sprite);
6194     if (spr) {
6195         spr.spriteFrame = null;
6196     }
6197     target.active = false;
6198 }
6199 ''
```